Stratocracy — Prototype GDD

Hex turn-based strategy · Unreal Engine 5.8 · 7-week AI-agent jam

---

1. Executive Summary

Concept. Two commanders battle across a small skirmish-sized hex map. Units move over terrain that shapes movement and defense; players capture factories to reinforce and win by destroying the enemy's **flag unit**. Matches run **~10–15 minutes typical**, with the turn cap (§2.8) putting the **outer bound near 20 minutes** at a normal move pace against the AI — so 10–15 usual, ~20 worst-case if a match goes the distance to the cap. That's an expected duration, not a wall-clock guarantee (a PvP hotseat, a stretch feature, would add a per-turn timer). The pacing discipline is deliberate, see Lineage.

The deliverable is the game; AI agents are the method. The graded, shipped output is a complete, playable prototype — deliberately systems-forward and art-light so that AI agents can author, test, and tune the bulk of it, with the human acting as director. The genre was chosen for exactly this reason: a deterministic hex wargame is almost entirely **rules + data** — the surface agents author most reliably and can self-verify with tests. How much the agents actually did is reported honestly as a per-system provenance ledger (§3), as supporting evidence of the method — but the thing that ships and is judged is the game. (Accepted tradeoff: the art-light choice leaves visual-craft strengths on the bench, by design.)

Lineage. *Conflict* (Vic Tokai, NES, 1989) — the first NES game to render a hex battlefield. We inherit its structure (hex map, terrain modifiers, capturable cities/factories, destroy-the-flag win) and explicitly fix the flaw its own reviewers named: **Famicom Tsūshin (Famitsu) faulted it for battles that "took too long"** (Western outlets were kinder — Nintendo Power 4/5 — so the criticism is specifically about pacing, not the design). Everything here is tuned toward short, decisive play.

Design pillars. 1. **Legible, deterministic rules** — agent-authorable and unit-testable; no hidden RNG (seeded if any). 2. **Short and decisive** — small maps, punchy damage, a turn cap. Never an hour-long slog. 3. **Terrain + unit rock-paper-scissors** — the only depth needed; it comes from data, not feature count. 4. **Minimal art, maximal system.** 5. **Tests are a deliverable** — every core system ships with agent-generated tests and self-play balance data.

Scope at a glance. One polished scenario, four units, six terrains + the capturable Factory tile, capture + production, a heuristic AI opponent, and a functional UI is a *complete, shippable game.* Everything beyond that is garnish (see §2 scope table). Core playable in weeks 1–3; the remaining four weeks are content, balance, UI polish, and documenting the pipeline.

Success = a shipped, playable prototype. The primary bar is the game itself: the §2 core loop running, tuned, and fun in a 10–15 minute match. Agent contribution is tracked as a **per-system provenance ledger** (§3) — an honest, row-by-row record of which systems agents authored and verified — reported as supporting evidence of the method, *not* as the pass/fail bar. Anything a human hand-authored (UI polish, glue) is logged as human, so the ledger under-claims rather than over-claims.

Design decisions (all resolved). 1. ~~Scenarios in the prototype: one polished, or three?~~ → **RESOLVED: one polished scenario.** Content is the most agent-scalable axis; one hand-built map proves the pipeline without spending human review budget on volume. 2. ~~Economy: production only, or add the "fame" scoring spine?~~ → **RESOLVED (updated): a unified Fame currency.** Production points, combat rewards, and the win-score collapse into one **Fame** pool inherited from *Conflict* — earned from held factories and kills, spent to build, tallied to decide a capped match (§2.7). This is now core (not a separate stretch spine); flag-kill remains the primary, decisive win. 3. ~~Opponent: heuristic-only, or attempt the LLM commander?~~ → **RESOLVED: heuristic-only ships (§2.9); LLM commander stays a stretch toggle.** 4. ~~2-player hotseat: in scope or cut?~~ → **RESOLVED: stretch only, off the critical path (§2.10). Nearly free on IGOUGO, but doesn't serve the agent-authorship thesis, so it never blocks core.** 5. ~~Working title.~~ → **RESOLVED: the game is titled *Stratocracy*.**

1.5 Revision Notes — First Draft → Final Draft

No external feedback was returned on the First Draft in time for this revision, so rather

than wait, I **stress-tested the draft with an agent review crew** — the exact technique from the course's GDD-anatomy stress-test method (an Exploit-Hunter / Consistency / Pacing board run over the document, then human-adjudicated). Applying the course's own agent method to my own GDD *is* the growth here: the crew surfaced concrete, exploitable logic gaps on paper, and every one below was fixed before a line of code. Changes are listed as **finding → change → why it's better**.

#	Finding (agent review crew)	Change	Why it's better
1	**Turtle exploit in the tiebreak.** Original cap tiebreak ("most factories held, then total HP") *rewarded* stalling: grab one factory, wall the flag in a corner, run out the clock, win without fighting.	Reordered the cap tiebreak around **combat Fame earned** (damage dealt), excluding passive factory income; added a **mutual-passivity → draw** guard; put **multiple factories** on the map and a **live standings scoreboard** (§2.7–2.8, §2.11).	Turtling now *loses* the cap. Pillar 2 ("short & decisive") is enforced by the grading itself, not just by the turn cap.
2	**Success metric was unmeasurable.** "Agents did X% of the work" was a hero-number that couldn't be checked or defended.	Replaced it with a **per-system provenance ledger** — one row per system, marked by author and agent-verified status (§3).	It's *checkable* (re-runnable tests + commit trace) and deliberately under-claims (human work logged as human).
3	**Economy was unpriced.** Production points, combat rewards, and win-score were three separate uncosted pools; unit costs were undefined; built units had "nowhere to go / uncapped points."	Collapsed everything into one **Fame** currency with explicit costs (Inf 100 / Recon 150 / Arty 200 / Tank 300), factory +100/turn, kill ≈ ½ cost, flag +500 ends the match, and **space-throttled** spawning (§2.7).	One legible pool the player and the AI both reason about; the uncapped-points gap is closed by throttling on board space, not an arbitrary ceiling.
4	**The AI ignored the economy.** A re-review found the baseline opponent never built or captured — it would fight with only its starting force, making the whole economy one-sided.	Gave the baseline AI a two-phase turn: an **economy (build) phase** plus a **capture step** before the unit phase (§2.9).	Keeps production, capture, and the "objectives held" tiebreak live on *both* sides — the economy is actually exercised in play.
5	**Scoring could invert the win.** Raw-Fame scoring let a long cap match's accumulated kill-Fame "outscore" an actual flag kill, and a mutual-zero-combat cap had no defined result.	Made victory a set of **categorical tiers** (Decisive > Marginal > Draw); combat Fame is only the **sort key inside** the tiebreak, never the grade; mutual-zero-combat = **draw** (§2.8).	A flag kill is *always* the best outcome — the win condition can never be out-piled by attrition points.
6	**Framing risk (design judgment).** Positioning the *agent method* as the deliverable would have misaligned with a game-first rubric that grades the playable product first.	Reframed the identity: **the game is the deliverable**; agent-driven development is the *method*, reported as supporting evidence via the ledger (§1, §3).	Aligns the document with how the work is actually judged — the shippable game leads, the pipeline substantiates it.

1.6 Revision Notes — Final Draft → Production Draft

The Final Draft closed the *logic* gaps §1.5 found, but it left the document thin in four

specific places that external feedback and the review board had already named. This revision closes them, and it was authored the same way the game is being built: a **four-author crew** — rules, UX/onboarding, scenario, and technical — writing in parallel against a frozen snapshot of this document, each confined to its own section and forbidden to edit the master. A separate **continuity gate** audited every draft against the live GDD for contradictions, stat drift, dead references, and invented numbers, and merge was blocked until it returned PASS. It did not pass on the first attempt: the gate filed four violations (a production-menu mock that greyed a unit the player could afford, a scoreboard chevron marking the wrong leader, and two dead references in the technical draft), the two authors responsible were re-spawned with those violations, and only the corrected drafts merged. That failure-and-correction loop is the same one §3 documents for T-COMBAT-07 — applied to prose instead of code. Changes are listed as **finding → change → why it's better**.

#	Finding (source)	Change	Why it's better
1	**No stated player-experience goals** *(rubric feedback, −0.5)*. The document said what the design *valued* (§1's pillars) but never what the player should *feel*, so nothing downstream could be checked against an intent.	Added **§2.0**, six goals PX-1..6, each carrying a rule source **and an observable check** — e.g. PX-4 "fighting is always better than hiding" is checked by T-CAP-02/03, "a side with zero combat Fame never wins a capped match."	A goal that cannot be checked is a preference, not a goal. Self-play reports and playtest notes now cite goals by ID, so intent is testable rather than asserted.
2	**No onboarding plan** *(review board open item)*. The GDD assumed a player who already understood hexes, Fame, capture, and the RPS triangle — with no manual and no tutorial mode in scope, nothing explained how they would learn them.	Added **§2.11.6**: three teachers (constraint, the forecast, and one-shot event tips), a scripted four-turn guided opening (turns 1–4) delivered through a directive strip, nine one-shot strings plus the two cap-approach banners, a pre-match briefing overlay, and a twelve-row concept ledger mapping every concept to where it is first taught.	The one genuinely unowned risk in a game with no manual is now a specified, buildable surface with a named cut line (§2.11.8) rather than an assumption.
3	**The ~3-match replay cliff** *(review board open item)*. The game is deterministic end-to-end, so any line that beats the AI once beats it forever; one mirrored map is a solved puzzle by match ~4.	Added **§2.13** in full — the shipped scenario *Ferrum Crossing* specified hex by hex, plus **§2.13.4**, which reframes the replay unit as a **configuration = map × seat × difficulty handicap** rather than as new mechanics.	Moves the cliff from match ~3 to match ~6 on shipped scope alone, using scenario data and one menu affordance — no new systems, no new rules, and the stretch maps stay explicitly cuttable.
4	**Eight `*pending*` ledger rows had no route to a gate** *(§3 provenance ledger)*. Combat cleared a real test gate; the other eight systems had no spec, no test IDs, and therefore no way to flip the same way.	Added **§4.7**'s eight gateable spec stubs (Inputs / Formula / Invariants / Determinism / Acceptance, the shape that certified Combat), **§4.8–§4.10** for data, integration and save-replay, and **§4.11**'s build order with an explicit critical path.	Every pending row now has a named acceptance suite and a dependency position, so the ledger flips on evidence rather than on assertion — and where a rule was missing, it is filed as a numbered open question in §4.7's register instead of being invented.

#	Finding (source)	Change	Why it's better
5	**§2 had drifted after the Conflict fold** *(continuity audit)*. Bridge and Factory had been folded into §2.3's terrain set, but §1 and §2.10 still said "five terrains," and §2.8's tiebreak had grown an apparatus nobody had re-justified.	Consolidated **§2.1–§2.10**, reconciled the terrain count across §1/§2.3/§2.10, and subjected the whole Fame/tiebreak apparatus to a **delete-test** — every piece removed in turn to see what breaks (§2.8's closing block).	The tiebreak now survives on evidence rather than inertia: the one piece that failed its delete-test (the floated per-turn Fame decay) was cut, and what remains is a guard, a sort, and an enum over state the game already tracks.

1.7 Revision Notes — Production Draft → the ruling cycle

§1.6's four-author crew did not only add sections. Wherever a gate needed a rule the document did not state, the crew was forbidden to invent one and filed it instead as a numbered open question in **§4.7's register**, with the conservative reading it would ship under while the question stayed open. This revision is the pass in which those questions were **ruled** — one row at a time, each ruling written into the register beside the question it answers, together with every site the answer reached. §4.7's register is the authoritative record of which rows exist and what each was ruled; the five entries below are the ones that changed the *game* or its evidence, rather than a gate's wording.

What they have in common is the growth worth reporting: **a ruling is not finished when the row is answered — it is finished when every site that depended on the answer has been found and corrected.** Four of the five cost more edits outside the row than inside it, and in two cases the ruling was carried out, re-checked, and then amended. Changes are listed as **finding → change → why it's better**.

#	Finding (source)	Change	Why it's better
1	**Income the document did not have** *(§4.7 register, Q8)*. §2.7 twice asserted that both sides have income from turn 1 — "both players have income from turn 1" and "plus home-factory income from turn 1" — while the shipped scenario priced its own opening the other way: §2.13.2 buys East's turn-1 Infantry with "100 of the 200 starting Fame," not of 300. Two sentences and a map disagreed about the first turn, and two gates (T-FAME-02, T-FAME-04) were written-and-blocked on the disagreement.	Ruled: income accrues at the **start** of the owner's turn and is spendable in that same turn's economy phase, but there is **no accrual on turn 1** — turn-1 buying power is the side's starting Fame alone. The correction rewrote **four §2.7 bullets** (factories, Income, build-and-spawn, starting Fame), §2.9's economy phase, T-FAME-02 and T-FAME-04, and the starting-Fame and build lines in `kb/rules.md`.	The reading chosen was the one the map was **already priced on**, so **no map number moves** — the edit lands on prose that had drifted, not on balance. Both blocked gates now assert, and the shipped scenario's opening turn means what §2.13.2 says it means.
	Two sections, two schedules *(Q23, Q30), §4.4*	Ruled on one principle in opposite directions: the vertical slice moved **later** (week 3; week 2 delivers move + attack only), and the §4.10 **format + headless replayer** moved	§1, §2.10, §4.4 and §4.11 now describe one schedule, and the rule that settles the next scheduling

#	Finding (source)	Change	Why it's better
2	*Q20)*. §4.4 promised a working vertical slice *with* the baseline AI in week 2, while §4.11's critical path (`1 → 3 → 4 → 5 → 6/8`) and §2.10 both put capture and Fame production in week 3. Separately, §4.10's format sat in week 5 although the week-2 integration gate's input file **is** a save.	**earlier** (week 2), leaving only the save-slot UI and slot I/O in week 5. The principle is now stated once, in §4.4: **a format is a test instrument; slot I/O is a feature.** The Q20 ruling was then **amended** — it had been written into §4.4 without re-reading §4.11's dependency table — and repaired by **scoping each gate to the command set of the log it replays**, without moving a week number.	argument is written down instead of re-derived each time. The amendment is the more useful half: the third disagreement between those two sections was closed by naming a gate's scope rather than by moving work again — and what a partially-scoped gate may claim was registered (Q29) rather than assumed.
3	**A new invariant refused the shipped map** *(Q22, Q28)*. Gating §2.13.1's promise that the guided lane is "uncontested, not merely reachable" produced **T-SCN-11**, and measuring it across all three maps *before* writing it found five of six lanes clear and one exact tie: West's lane to the South factory cost 5 MP, and East's second Infantry at (9,5) reached that same factory in 5 MP flat.	**The map was corrected rather than the rule loosened.** East's second Infantry deploys at **(9,1)**; no terrain, factory or town count, lane cost, home-factory rule or turn estimate moved with it, and *Ferrum Crossing* now reports **5 against 6 in both seats**. The pre-fix deployment is retained as T-SCN-11's fixture (b).	The suite now owns "a failing case that was actually authored, that passes every other invariant in the suite" — a negative fixture the project produced rather than one a test author constructed, which is the difference between a test that can refuse a real scenario and one that agrees with the repo. It also marked the single row where the register's conservative-reading convention could not hold, and the register now states that limit once.
4	**An unpriced term inside the primary sort key** *(Q6)*. §2.7's "small bonus" for an undamaged strike had no number, yet combat Fame is the **primary** sort key of §2.8's cap tiebreak — a number nobody had chosen was helping decide capped matches.	Ruled **cut**, not priced: kills already pay half cost and the positional triangle already rewards a clean standoff strike with tempo, so the bonus was paying twice for one thing. The cut reached **six sites that never cited Q6** — §2.8's tally definition, §2.11's standings row and its tooltip, the §2.11.6 one-shot toast, the concept ledger's RPS row (whose receipt is now the range-2–3 one-shot), and two lines in `kb/rules.md`.	Cutting removes the unpriced term from the tiebreak instead of leaving an unchosen number inside it, and it is cheaper than the alternative, which would have made the document carry two Fame totals — one for the pool, one for the cap tally — in both the kb economy block and the T-CAP- suite. The six uncited sites are the transferable lesson: grepping the identifier alone would have found none of them.
	The token budget priced one thing twice *(Q4.6)*. The	Re-derived every	Both faults were

#	Finding (source)	Change	Why it's better
5	**twice** *(§4.6)*. The Opus escalation is a *delta* between two models on the same task. An earlier draft folded it into the dev-time subtotal and then applied it a second time to reach **$225** — which is why three figures in one table disagreed. A second fault in the same table scaled the 1.5× overrun off a *rounded* subtotal.	figure from the two rate lines and the task count, and quoted the escalation delta **unrounded** ($1.035) at every site that uses it. The subtotal is **$178.02**; the overrun case is re-derived rather than scaled, landing at **≈ $266**, with **$303** stated as the ceiling.	visible **only because the table is fully re-derivable from its inputs** — the property is what caught them, so it is now the table's stated discipline rather than an accident. A budget that can be recomputed is auditable; one that quotes a total is not.

2. Game Mechanics

2.0 Player-experience goals

The pillars (§1) state what the design values; these goals state what the player should experience, phrased so each is observable in the test suite or the balance harness (§4.1). A goal that cannot be checked is a preference, not a goal — so each row carries its check. Self-play reports and playtest notes cite goals by ID.

ID	The player experiences	Rule source	Observable check
PX-1	Matches feel short and decisive: ~10–15 minutes typical, ~20 worst-case at the cap. A match never becomes the hour-long slog the lineage was criticized for.	§1, §2.8	Self-play turn-length distribution: the median match ends before the turn cap; cap-tiebreak matches are a minority of results.
PX-2	The rules can be trusted completely. The forecast shown before committing an attack is exactly what resolves — a whole turn can be planned in advance.	§2.6	Every attack's forecast equals its resolution; identical inputs give identical results (determinism invariants, T-COMBAT suite).
PX-3	Depth comes from where units stand, not from a memorized counter chart. The triangle is read off movement and range on the board.	§2.4	The type-effectiveness table ships all-1.0; no counter-multiplier is load-bearing at ship.
PX-4	Fighting is always better than hiding. There is no line of play where sealing a corner and running the clock wins.	§2.7, §2.8	A side with zero combat Fame never wins a capped match (T-CAP-02, T-CAP-03 — both gated as **T-TURN-05**).
PX-5	The player always knows who is currently winning and how close the cap is. The tiebreak is never a hidden win condition.	§2.8, §2.11	The standings scoreboard displays every tiebreak input (combat Fame, objectives held *X/N*, surviving HP) plus the turn counter against the cap.
PX-6	The economy is one thought: earn Fame, spend Fame. The player never converts between currencies or tracks parallel pools.	§2.7	Every income, build cost, and combat reward mutates a single per-side Fame pool; no second resource exists in the data schema.

2.1 Core loop

```
Player turn:
  for each of your units (any order; a unit may be given its remaining
command
    with other units' commands in between — the rules module has no
current unit):
     select → move (within range, terrain-costed) and/or act (attack), in
either order → done
              (two independent flags: at most one move and at most one act
per
               unit in its own turn, and neither is required — gated as
               T-TURN-01; capture and build set no act flag, §2.7)
  end turn
Opponent turn: same, driven by AI
repeat until a flag unit dies, an objective is met, or the turn cap
triggers a tiebreak
```

I-GO-U-GO alternation. No simultaneous resolution.

2.2 Hex grid

The battlefield is a field of pointy-top hexagons. The player reads adjacency and distance at a glance — every hex has six equal neighbours, so no direction is unfairly cheap the way

diagonals are on a square grid — and each unit occupies exactly one hex. Line-of-sight blocking (a unit can't see or fire through a ridge) is a stretch goal.

2.3 Terrain (prototype set: 6 movement terrains + the capturable Factory tile)

Terrain is the first thing the player weighs before moving: some hexes cost more of a unit's movement allowance to enter, some make a unit standing on them harder to kill, and some are closed to certain movement classes entirely — the player is always trading speed against cover and reach. Move cost is per movement class. All values are starter/tuning targets.

Terrain	Move cost	Defense	Passable
Plains	1	0%	land, air
Woods	2	+20%	land, air
Mountains	3	+40%	land (slow), air
Water	—	0%	sea, air
Town	1	+10%	land, air; capturable
Bridge	1	−10%	land, air; **the only hex a land unit crosses Water**
Factory	1	+15%	land, air; capturable; build/spawn + repair point

Bridge turns Water from dead space into a chokepoint system: land routes must funnel across bridges, and the negative defense makes a unit caught mid-crossing an exposed target — a contested kill-zone, not a safe shortcut. *(Tuning fallback: −10% → 0% if too punishing.)* **Factory** is the economy's anchor (§2.7): occupiable by either side, capturable by Infantry, a defensive anchor, and the build/spawn + repair point.

2.4 Units (prototype set: 4)

Rock-paper-scissors, not a roster. Starter stats.

Unit	HP	Move	Atk	Def	Range	Cost (Fame)	Notes
Infantry	10	3	4	2	1	100	Cheap; the only unit that captures towns/factories
Tank	20	5	8	5	1	300	Line breaker; **Flag Unit** is a Tank variant (not producible)
Artillery	8	3	10	1	2–3	200	Strong ranged, fragile, no melee counter
Recon/Air	12	7	5	3	1	150	High move; ignores some terrain cost

Cost is in Fame, the single currency (§2.7). All values are tuning targets, scaled down from *Conflict*'s cost ladder for four units and ~20-turn matches (§2.12).

Flag Unit: a designated Tank. Its death = loss. Not producible.

The triangle is positional, not a counter chart. Stratocracy's rock-paper-scissors lives in movement and range, an invariant worth naming: **Artillery beats Tank** (fires from range 2–3, takes no melee counter), **Recon beats Artillery** (move 7 runs it down into a fragile melee), **Tank beats Recon** (higher atk/def wins the range-1 fight). Infantry sits outside the triangle as the capture/objective unit. A type-effectiveness multiplier exists in the combat formula (§3) but ships **defaulted to 1.0 everywhere** — populated only if self-play shows the positional triangle too weak, so depth stays in positioning rather than in a lookup table (Pillar 3; PX-3).

2.5 Movement & pathfinding

The player selects a unit and every hex it can truly reach this turn lights up, terrain costs already accounted for — the highlight is the real move set, not an estimate. Clicking a lit

hex sends the unit by the cheapest route. Only one unit fits in a hex, so lanes, chokepoints, and blocking are part of the plan. Zones of control — enemy units freezing anything that moves next to them — are cut from the prototype; they return only if matches feel too fluid.

2.6 Combat resolution

Before an attack is committed, the game shows the outcome up front: the predicted damage dealt, and the counterattack the defender strikes back with if it survives and the attacker is within its reach (attack forecast, §2.11). Combat is a pure function of attacker, defender, the defender's terrain, and the attacker's remaining health — the same matchup always resolves the same way, so a player can plan a whole turn and trust it plays out as shown (PX-2). Any randomness added later is **seeded**, so a given fight stays reproducible: the forecast the player sees is exactly what resolves, with no hidden roll.

2.7 Economy & capture — the Fame currency

Stratocracy runs on a **single currency, Fame** (inherited from *Conflict*, §2.12): earned from held objectives and combat, spent to build, tallied to decide a capped match. Production points, combat rewards, and the win-score are **one pool, not three** (PX-6). *All numbers are starter/tuning targets, scaled down from Conflict's for four units and ~20-turn matches.*

- **Factories & starting layout:** the map ships with **multiple factories** — a **home factory per side** (owned at start, so both players draw income from **turn 2**, the first accrual — Q8, §4.7) plus **two or more neutral factories** in contested ground; a typical small skirmish map has **~4 factories total**. The spread is what makes expansion worth fighting for and "objectives held" (§2.8) a real 0–N measure instead of a 1–0 coin-flip.
- **Income:** each **factory** held pays **+100 Fame/turn**; each captured **town** pays **+25/turn**. Income accrues **at the start of your turn and is spendable in that same turn's economy phase** (§2.9) — with **no accrual on turn 1**, so a side's turn-1 buying power is its **starting Fame alone** — 200 for both sides at Normal, and for the **player** 350 on Easy or 100 on Hard, since §2.9's handicap moves the player's opening Fame only and the AI opens on 200 at every tier — and the first income lands on turn 2 (Q8, §4.7). More objectives held = a faster army, so the neutral factories are the mid-game prize.
- **Build & spawn:** spend Fame at a factory to produce a unit from the buildlist (§2.4 costs: Infantry 100, Recon 150, Artillery 200, Tank 300). **One build per factory per turn**, for the player and the AI alike (§2.9). The unit **spawns on the factory hex if it's free, otherwise an adjacent free hex; if the factory is boxed in, the build waits.** A waiting build **holds that factory's slot until it spawns**, and its Fame is **committed when the build is queued, not when the unit appears, and is not refundable** (Q8, §4.7). The slot and the per-turn limit are **two rules, not one**: the slot is held only while a build waits and clears the moment the unit spawns, while the per-turn limit binds for the rest of that turn either way — a factory whose build spawns at once has still spent its build for the turn. The **player cannot currently reach the waiting case**: §2.11.5 disables the Build buttons while a factory is boxed in, so for the player queue time and spawn time are the same instant, and the waiting build is an **AI-only path** today (§2.9 builds without the UI). Whether the player should be able to queue into a boxed-in factory is **Q31** (§4.7). Fame has no hard cap — deployment is throttled by board space, not a point ceiling.
- **Combat Fame:** destroying an enemy unit pays **exactly half its §2.4 cost — Infantry +50, Recon +75, Artillery +100, Tank +150** (Q5, §4.7; every value an integer). Destroying the enemy **flag pays a flat +500 and ends the match** — the flag award **replaces** the victim's ordinary kill award rather than stacking with it, so a flag Tank pays 500, not 650 (Q5). There is **no undamaged-strike bonus**: it was cut rather than priced, because kills already pay half-cost and the positional triangle already rewards a clean standoff strike with tempo (Q6, §4.7). These feed the same Fame pool and the cap tiebreak (§2.8).
- **Capture:** move an Infantry (the only capturer) onto a town/factory and hold to capture over N turns (**N = 1** on the shipped scenario; N is per-scenario data). Capture progress is **held by the tile and resets to zero the moment the capturing Infantry leaves the hex or dies** — it never transfers to another unit and never survives an interruption (Q4, §4.7). A captured objective flips its Fame income to the new owner.
- **Starting Fame:** each side opens with **200 Fame** at Normal — enough for two Infantry or one Artillery on turn 1, and the whole of turn-1 buying power, since income first accrues on turn 2 (Q8, §4.7). Home-factory income adds +100/turn from turn 2 onward. Single-player difficulty is a starting-Fame handicap — see §2.9 — and it moves **the player's side only**, so the 200 is a **baseline, not a constant, for the player**: the player opens on 350 on Easy and 100 on Hard, while the AI opens on 200 at every tier. Any statement here about what turn 1 can afford is a Normal-tier statement.
- **Repair:** a unit that ends its turn on an **owned Town or Factory and is not adjacent to**

any enemy heals **+25% of max HP** (rounded down, min 1, capped at max) at the start of its next turn — free for the prototype. This is the game's only HP-recovery path; without it every unit is a one-way asset. The **not-adjacent clause is the anti-fortress lock**: a unit must break contact to repair, so a factory next to the front never becomes unkillable — and because repairing earns zero combat Fame, a repair-turtle still loses the cap tiebreak (§2.8). *(Heal %, and a possible small Fame cost, are tuning levers.)*

2.8 Turn structure & victory

- **Primary win — decisive victory:** destroy the enemy flag unit. Ends the match immediately.
- **Secondary win — territorial domination:** control **every factory on the map** at the start of your turn. Ends the match immediately and is ranked a **Decisive win**, equal to a flag kill. Because taking the last factory means capturing the enemy home factory deep in their territory, a flag kill is usually already available by then — this is an **active backstop that closes out a flag-turtle stalemate before the cap**, not a common win. Factories only (towns excluded), so it stays hard-won.
- **Loss:** your own flag unit is destroyed, or the enemy dominates all factories.

Turn cap → attrition tiebreak. If neither flag has fallen by the turn cap (**20 turns** on the shipped scenario — the cap is per-scenario data, set in the scenario file's `turnCap` field, §2.13.2 and §4.7 Stub 7), the match resolves as a battle of attrition. The full procedure is one guard, one three-key comparison, and one grade. Every input is a value the game already tracks for the economy (§2.7) and already shows on the standings scoreboard (§2.11): the tiebreak adds no new state, only an ordering over existing state.

Tally definition. A side's **combat Fame** is the Fame it earned from unit kills (§2.7) — the undamaged-strike bonus this key once also counted was cut unpriced (Q6, §4.7), so kills are its only source. It **excludes** passive factory and town income — counting income would let a staller re-earn the turtle exploit (§1.5 #1) by sitting on objectives. The flag bonus (+500) can never appear in a capped tally: destroying the flag ends the match immediately, so no match that reaches the cap contains one.

Resolution procedure, in order:

1. **Mutual-passivity guard.** If *both* sides' combat Fame is zero — nobody engaged — the match is an immediate **draw**. It does not fall through to the keys below, because "objectives held" would otherwise re-crown a turtle who simply sat on more factories.
2. **Lexicographic comparison** — higher wins at the first key that differs:
    1. **Combat Fame earned.** The anti-turtle lever (PX-4): a player who sealed a corner and refused to fight scores zero here and loses the cap to anyone who dealt damage. (The guard already covers the both-zero case, so no separate "both sides fought" precondition is needed on the later keys — if this key ties at a nonzero value, both sides fought by construction.)
    2. **Objectives held** — the factories and captured towns a side owns at the cap, as *X of N*. Ownership only: a capture in progress (§2.7) counts for nobody until the objective flips. Towns count here even though the domination win is factories-only — domination must stay hard-won; this key merely breaks an exact Fame tie. With ~4 factories plus towns on the map (§2.3, §2.7) this is a genuine spread.
    3. **Surviving strength** — total remaining HP of a side's units. Deliberately last: alone it would re-reward turtling, but by the time it applies both sides have earned identical nonzero combat Fame, so an HP edge measures trading efficiency between two sides that both fought. It is free to compute (the scoreboard already sums it) and decides matches that would otherwise be draws.
3. All three keys equal → **draw**. Acceptable for the prototype; a competitive build would add a sudden-death overtime.

Victory quality. A result is graded as a **tier**, not a number:

Tier	Trigger
Decisive win	Enemy flag destroyed, or territorial domination
Marginal win	Led the attrition comparison at the cap
Draw	The passivity guard fired, or all three keys tied

Tiers rank categorically: a **decisive win always outranks a marginal win**, regardless of how much Fame either side piled up. Combat Fame is only the sort key *inside* the tiebreak, never the grade — keeping the two separate prevents a long capped match's accumulated kill-Fame from "outscoring" an actual flag kill (§1.5 #5). For self-play tuning, log the tier plus the Fame breakdown per match. The result: a flag kill is always the best outcome, so Pillar 2 ("short and decisive") is enforced by the grading itself, not just by the turn cap.

Invariants. `T-CAP-01..08` is §2.8's own numbering for the procedure §4.7 Spec Stub 5 gates within `T-TURN-01..10`, so there is one suite, not two. The map below names, for each invariant, the ID or IDs that gate it; one row names a gate outside the `T-TURN-`

numbering.

1. **T-CAP-01** — Flag destruction on any turn at or before the cap yields a Decisive result; the tiebreak procedure is never evaluated.
2. **T-CAP-02** — Both sides at zero combat Fame at the cap → Draw, regardless of objectives held or surviving HP.
3. **T-CAP-03** — Passive income never decides the cap: a side holding 4 factories with zero kills loses the cap to a side holding 1 factory with one Infantry kill.
4. **T-CAP-04** — No capped tally contains the +500 flag bonus.
5. **T-CAP-05** — A capture in progress at the cap contributes zero to "objectives held" for either side.
6. **T-CAP-06** — Tier order is Decisive > Marginal > Draw for *any* pair of Fame totals.
7. **T-CAP-07** — Determinism: identical end state at the cap → identical result and tier.
8. **T-CAP-08** — Controlling every factory at the start of your turn ends the match immediately as a Decisive win; towns do not count toward domination.

Alias map — the ID or IDs that gate each invariant above; one row names a gate outside the `T-TURN-` numbering:

§2.8	Aliases to	Why
T-CAP-01	**T-TURN-02**	flag death ends the match at once; the tiebreak is never evaluated
T-CAP-02	**T-TURN-05**	the mutual-passivity guard
T-CAP-03	**T-TURN-05**	T-TURN-05's fixture is 4 objectives + zero kills losing to 1 objective + one 50-Fame kill. That combat Fame excludes passive income is **T-FAME-01**
T-CAP-04	**T-TURN-02**	no capped tally can contain the +500, because a flag kill ends the match before the cap
T-CAP-05	**GATE-CAP-PARTIAL**	not a `T-TURN-` ID; see below
T-CAP-06	**T-TURN-07**	tiers are categorical
T-CAP-07	**T-TURN-09**	determinism
T-CAP-08	**T-TURN-03**	domination, factories only

T-CAP-05 is the exception. No `T-TURN-` ID asserts it. It is discharged *structurally* by T-FAME-05 and T-FAME-06 — an objective's owner does not change until the capture completes, and the tally counts owners — and it is asserted end to end by **`GATE-CAP-PARTIAL`**, which is named in Spec Stub 8's acceptance set (§4.7). Nothing blocks that gate. **It has since run, and §3 records the run:** it passed on a fixture configured with `captureTurns = 2`, a state *Ferrum Crossing* cannot reach at N = 1 (Q4). **That closes T-CAP-05** — ruled 2026-08-04; the reasoning is recorded once, at §4.7's Q14.

> **Why this shape (the delete-test).** Every piece of the apparatus was tested by deletion. Delete key 1 → the documented turtle exploit (§1.5 #1) returns whole. Delete the guard → a four-factory turtle beats a one-factory turtle without a shot fired. Delete key 2 → contesting the neutral factories (§2.7's mid-game prize) stops mattering at the cap. Delete key 3 → more draws for zero savings, since the HP sum already exists for the scoreboard. Delete the tiers → a capped grind's tally can read as "beating" a flag kill in tuning logs — the inversion §1.5 #5 closed. Delete the domination backstop → a walled-in flag forces every such match to run the full cap for a Marginal result instead of ending early and Decisively (Pillar 2). The one piece that failed the test — the floated per-turn Fame decay — is cut in this revision (§1.6 row 5). What remains is a guard, a sort, and an enum, all over state the game tracks anyway.

On the turn cap vs. real time. The cap bounds *turns*, which guarantees the match terminates; it does not bound wall-clock minutes. Against the shipping single-player AI (which moves instantly) that is a non-issue, so "~20 minutes" (§1) is an expected duration at a normal move pace, not a hard real-time ceiling. The only mode that would need a per-turn timer is **PvP hotseat**, a stretch feature (§2.10) — if it ships, add a move clock there.

2.9 Opponent AI (runtime gameplay system)

- **Baseline (ships): simple objective-seeker.** Two phases each turn, so the AI actually *uses* the economy (§2.7), not just the map:
  - **Economy phase.** Runs first, on the income that has just accrued for this turn (§2.7 — none on turn 1). At each factory it holds: if it can afford a unit, build one

— the single build that factory gets this turn (§2.7) — from a default buildlist (mostly Infantry, an occasional Tank), spawning per §2.7. Ties between affordable units break by the fixed priority **Infantry > Recon > Artillery > Tank** (Q9, §4.7). It spends Fame and replaces losses instead of hoarding.
    - **Unit phase, per unit:** (1) an idle **Infantry** adjacent to or near an uncaptured, **undefended** factory/town moves onto it to capture — the AI contests objectives, keeping capture, production, and the "objectives held" tiebreak live on *both* sides; (2) if an enemy is within reach after moving, attack — prefer the enemy flag, else the best expected-damage target, ties broken by the target's canonical hex order (Q9, §4.7); (3) ranged units (Artillery) fire from maximum standoff so they don't eat a counter; (4) otherwise advance along the cheapest path toward the enemy flag.
    - One cheap guard keeps it from looking broken: **skip a strictly-losing attack** (the unit would die and trade down).

  Deliberately un-clever — decisive, readable, and fully testable. This is the shipping opponent.
- **Difficulty = a starting-Fame handicap, not a smarter AI** (mirrors *Conflict*). The baseline routine is identical at every tier; only the economy shifts: **Easy** = player +150 opening Fame; **Normal** = even (200/200, §2.7); **Hard** = player −100. Deterministic and trivially tunable, with no AI-quality risk. *(Stretch dial, documented only: also scale AI income ±25%/tier.)*
- **Stretch: AI second pass (utility + threat map).** Upgrade the baseline to score candidate actions on attack value (expected damage vs. counter), objective proximity, and safety (a threat map of enemy reach). Week-3 work, only if the baseline proves too exploitable in self-play.
- **Stretch: LLM commander.** Serialize the board to text, enumerate legal moves, ask the model to rank one, then **validate and apply** (never trust an unchecked move). Behind a toggle, heuristic as fallback. This is the one place an AI *agent* appears in the shipped product — see §3.

2.10 Scope table

	Contents
IN (core; the playable core phased wk 1–3, the remainder scheduled in §4.4)	Grid; 4 units; **6 terrains + the Factory tile**; move + attack; **multiple factories** (home-per-side + contested neutrals, ~4) + capture + Fame production *(these land wk 3, not wk 1–2)*; heuristic AI; **one hand-built scenario**, with the scenario file and headless validator it loads through (§4.7 Stub 7); win-by-flag; functional UI; the **§2.11.6 guided opening** — four beats, first match only *(onboarding, wk 5)*; the **§4.10 save/replay format + headless replayer** — *instrument, not feature (wk 2)*; minimal single-slot save/load *(wk 5)*
STRETCH (if ahead)	2nd–3rd scenario authored on that format — *Longwater March* (P1, wk 4) and *The Causeway* (P2, wk 4) — shipping under the conditions §2.13.7 states, which that section states alone; LLM commander; fog/recon; sea/air units; map-gen MCP toolset (§4.2) — the in-editor wrapper only, not the validator it wraps; 2-player hotseat
CUT	All 16 original scenarios; campaign/meta; zones of control; elaborate art; anything real-time

Reading this table. Four notes, so that a scope question and a schedule question are answered on the same page:

- **§1's *Scope at a glance* and this table are not the same line.** §1 names the smallest set that is already a complete game — §4.5's hard MVP line. This table names what is **scheduled**, which is a superset of it.
- **Instrument, not feature.** The §4.10 format and headless replayer are IN because two gates run on them, not because a player asks for them: T-INT-02's input file is a save, and the week-4 self-play logs T-SAVE-07 validates are the same format (§4.4, §4.11 row 10). §4.4 states the rule this turns on — *a format is a test instrument; slot I/O is a feature* — so the format is core, the save-slot UI is the ordinary week-5 UI half, and no player-facing replay is scoped.
- **Off the critical path is not the same as stretch.** §4.11 says nothing in its chain waits on row 7, and §4.7 Stub 7 is nonetheless core: §4.4 has the one scenario loading, validating and rendering in week 2; §4.11 row 8 (UI binding) depends on row 7

because the snapshot needs full state; row 10(b)'s replayer reads the `scenarioId`/`scenarioHash` it produces; §2.8's turn cap is its `turnCap` field; and §2.11.6's guided opening is driven by its `guidedOpening` entries. What is stretch is the **second and third scenarios authored on that format**, and the in-editor toolset that wraps the validator — the manual fallback stands (§4.11 row 7).
- **A STRETCH row is a promise about cost, not about intent.** Where a section already states the condition an item ships under, that section owns it and this table does not restate it — for the two stretch maps that section is §2.13.7.

2.11 UI/UX

Stratocracy's interface has one job: make a deterministic game *look* deterministic. Every rule in §2 is knowable before the player commits, so the UI's standard is not "pretty" but **no surprises** — if the player is ever surprised by an outcome, the UI failed, not the player. Three principles govern everything below:

1. **Hover is information, click is commitment.** Hovering never changes game state; it only reveals (terrain lines, path previews, the forecast). A single left-click commits. Cancel is always right-click or Esc.
2. **Every element earns its pixels.** Each HUD element is audited against the decision it supports (§2.11.2); anything supporting no decision is cut. Two deliberate cuts up front: **no minimap** (the one shipped scenario is a small skirmish map, §2.7 — it fits on screen at default zoom; pan/zoom covers the rest) and **no combat log** (determinism plus the forecast make a history redundant for the prototype; a log rides the replay format if that ships, §4.10).
3. **Rules-critical information is never transient-only.** Toasts and banners are receipts; the durable version of every fact lives on a persistent or hover-recallable surface.

UI is budgeted as core work from week 2 (§4.4, §4.5 — "UI underestimated" is a named risk) and everything below is scoped to what one person can build in UMG on top of agent-scaffolded widget skeletons (§3, UI Scaffolder role).

2.11.1 Control scheme

Selection state machine. The core loop (§2.1) maps to four UI states. The loop's own sequence, and its list of what counts as an act, are §2.1's to state; this machine is the input surface for them and deliberately does not reprise them:

```
IDLE ──LMB on own unacted unit──▸ SELECTED (reachable hexes lit, §2.5;
attack targets lit)
IDLE ──LMB on own factory──────▸ PRODUCTION MENU (§2.11.5)
SELECTED ──LMB on lit hex──────▸ MOVED (targets relit from the new hex;
Wait available)
SELECTED ──hover enemy target──▸ forecast card shown (§2.11.3)
SELECTED ──LMB on lit target───▸ attack resolves as forecast → unit DONE
(move unspent)
SELECTED ──Space (Wait)────────▸ unit DONE without moving or acting
SELECTED ──RMB / Esc───────────▸ IDLE (nothing committed)
MOVED ──hover enemy target─────▸ forecast card shown (§2.11.3)
MOVED ──LMB on lit target──────▸ attack resolves exactly as forecast →
unit DONE
MOVED ──Space (Wait)───────────▸ unit DONE without acting
MOVED ──RMB / Esc──────────────▸ unit DONE where it stands (move already
spent)*
```

* Under a ruling that grants move-undo (**Q11**, §4.7 — currently unruled, and no gate assumes it), RMB/Esc in MOVED instead reverts the unit to its pre-move hex and returns to IDLE. Until that rule is adjudicated, the shipping semantics are the conservative ones shown: a completed move stands. The two behaviors share a UI; only the rules module differs.

The machine is narrower than the rule, and that is deliberate. `T-TURN-01` (§4.7 Spec Stub 5) owns the per-unit move and act flags and the orderings they permit; read them there, not off this diagram. The machine above reaches an attack two ways — after a move, or straight from SELECTED with the move unspent — and it retires the unit at the act either way, whatever that unit has left. What it does not offer is the reverse, a unit returning to the board after acting: an ordering the invariant permits is not on screen. This is a UI restriction and nothing more: no line here asks the rules module to refuse a command it would otherwise accept, and AI turns, headless runs and replays are untouched. It is also not provisional the way the move-undo note above it is — that note waits on an unruled question, this one scopes a ruled rule down to what one screen teaches. The cost is paid to keep the per-unit vocabulary at exactly two verbs (*attack* or *wait*, below) and the board at one per-unit state: the SELECTED attack is precisely the case that retires a unit with a flag to spare, and a unit that could come back for it needs a third pip state, a rule for what **Tab** does with it, and an explanation — in a first session already spending its attention on hexes, capture and the forecast (§2.11.6). Nothing announces the restriction to the player, by §2.11.6's first principle: the option is never

offered, so there is nothing to un-learn.

Standing still is never a move. The unit's own hex does not join the lit reachable set (`T-MOVE-03`: a move never ends on an occupied hex), so SELECTED's two lit sets — reachable hexes and attack targets — are disjoint, and **LMB** on a lit hex is never ambiguous between moving and attacking. A unit that attacks from SELECTED has not moved; it has spent its act and been retired by this machine, not by the rules module.

What DONE means, and what binds to it. DONE is this machine's own per-unit bit — *this unit takes no further command this turn* — and it is not the act flag. It lives in the **view-model** — the rules-produced snapshot plus a declared **presentation block** — as a member of that block, alongside the guided opening's `lockedThisTurn` (§2.11.6-B, turn 1a), which is per-unit and per-turn and which the guidance layer owns rather than the selection machine. The rules module has no DONE bit and no Wait command; no snapshot field mirrors this bit and none is asked for. It is per-turn: it clears when the owner's next turn begins. **Space** (Wait) and RMB/Esc in MOVED both reach DONE without acting, so the two bits come apart in ordinary play. Every surface in §2.11 that says a unit *has not acted* binds to the machine's bit: the `unacted unit` entry into SELECTED, the **Tab** cycle, the **Enter** confirm's count and its `3 units have not acted. End turn?` string, the idle count and the unacted pip (§2.11.2), and the spawned unit's pip in §2.11.6-D. Bound that way each of those stays literally true, because the machine retires a unit the instant it acts — so every unit still carrying a pip has in fact not acted. Bound to the act flag instead, a waited unit would keep its pip, answer **Tab**, and be counted in the End Turn confirm the player had just dismissed it from: a surprise the player caused and the UI then denied, which is the one thing §2.11 does not ship.

Capture and build need no extra verbs. Capture is by presence (§2.7: an Infantry that ends its move on a capturable tile begins capturing — a progress pip appears, no button). Building is the factory's own interaction, not a unit's. This keeps the per-unit action vocabulary to exactly two: *attack* or *wait*.

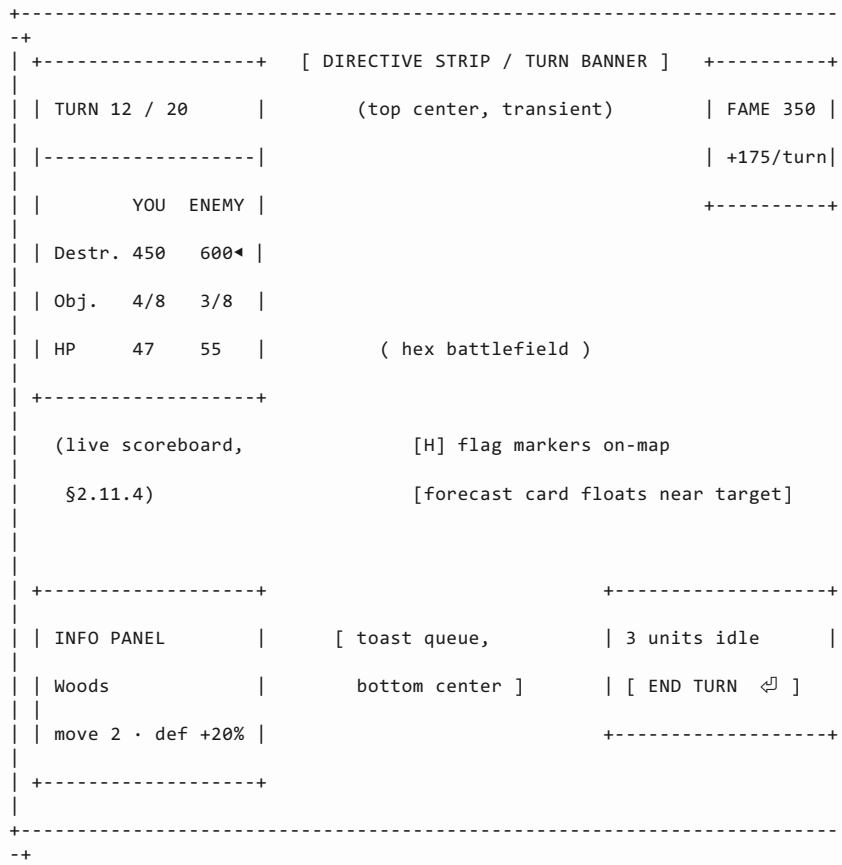
Input reference.

Input	Effect
Hover hex	Terrain line in the info panel (§2.11.2); if a unit is SELECTED, dotted **path preview** along the cheapest route (§2.5) with the terrain-cost tick per hex
Hover unit	Unit stats in the info panel; if an own SELECTED or MOVED unit has it in reach, the **forecast card** (§2.11.3)
LMB	Select own unit / commit previewed move / commit forecast attack / open production menu on own factory / activate buttons
RMB / Esc	Cancel: close menu or forecast, deselect, back out one state (see machine above)
MMB drag or **WASD / arrows**	Pan camera
Mouse wheel	Zoom (two or three fixed steps, not continuous — readability over cinematography)
Tab	Cycle to the next unit that has not acted
Space	Wait — mark the selected/moved unit done without acting
F	Snap camera to your flag unit
B	Open the production menu at the selected owned factory (or your home factory if none selected)
Enter	End turn — with a confirm dialog if any unit has not acted: `3 units have not acted. End turn?`
Z	Undo move *(only if Q11 (§4.7) is ruled to grant undo; otherwise unbound)*
Any click / Esc during enemy turn	Skip AI playback to its end state (§2.11.2, turn banner)

No hidden double-functions, no drag-to-move, no context menus. A first-session player can complete a match knowing only: hover, LMB, RMB, Enter.

2.11.2 HUD layout & information architecture

Screen layout.



Three information layers, strictly tiered:

1. **Persistent** (always on screen): scoreboard + turn counter, Fame pool + income rate, End Turn + idle-unit count, on-map flag `H` markers and unacted-unit pips. These are the four standing decisions of every turn — *am I winning the cap, can I afford to build, is my turn actually finished, what must I protect.*
2. **Contextual** (selection-driven): reachable-hex highlight, path preview, attack-target highlight, info panel content, forecast card, production menu, capture pips. Appears with a selection or hover, gone when it ends.
3. **Transient** (event receipts): toasts (`+100 Fame — Factory`, `+[N] HP — repaired`, `+150 Fame` on a kill), turn banner, one-shot onboarding lines (§2.11.6), cap-approach banners (§2.11.4). Each transient fact has a durable home: income toasts restate what the Fame widget's `+X/turn` already shows; kill toasts restate the scoreboard's Destroyed row.

Earn-your-pixels audit — every persistent/contextual element, the decision it supports, and the rule it surfaces:

Element	Decision it supports	Rule surfaced
Scoreboard (turn + 3 rows)	Force combat or hold; how close is the cap	§2.8 tiebreak order, turn cap
Fame pool + `+X/turn`	Build now vs. save; which neutral factory is worth a fight	§2.7 income (+100 factory / +25 town), costs — the `+X/turn` figure is the snapshot's per-side `incomePerTurn`, not a figure derived from the scoreboard's *X of N*, which does not separate factories from towns; and `incomePerTurn` is the standing rate, so on turn 1 the widget shows the rate that will pay at the start of turn 2 (Q8, §4.7) rather than the 0 that pays on turn 1
End Turn + idle count	Is my turn genuinely spent	§2.1 per-unit loop
Flag `H` marker (both sides, always visible)	What to protect, what to hunt	§2.4 flag death ends the match; *Conflict*'s `H` convention
Unacted pip on own units	Which units I can still give an order to	§2.1 per-unit loop, via the DONE bit of §2.11.1's machine, carried in the view-model's presentation block
Reachable-hex highlight	Where can this unit truly go	§2.5 — "the real move set, not an estimate"
Path preview with cost ticks	Which route; exposure en route (e.g. a turn spent on the Bridge at −10%)	§2.3, §2.5 cheapest-route
Attack-target highlight (incl. Artillery's dark range-1 hole)	Who is actually hittable from here	§2.4 ranges; the Artillery dead zone
Forecast card	Commit this attack or not	§2.6 — the whole section (§2.11.3)
Info panel	Speed-vs-cover terrain trade; matchup reading	§2.3 table, §2.4 stats
Production menu	What to buy, where it will spawn	§2.7 build & spawn rules (§2.11.5)
Capture progress pip	Hold or abandon the capture	§2.7 capture over N turns
Repair eligibility pip (owned tile, damaged unit selected, no adjacent enemy)	Retreat to heal or keep fighting	§2.7 repair + anti-fortress clause

Info panel (bottom-left, hover-driven, ~3 lines, never modal): - Hovered hex: terrain name, move cost, defense bonus, and status if capturable — `Factory · move 1 · def +15% · yours (+100/turn)` or `· neutral` or `· enemy`. - Hovered unit: name, HP as `12/20`, Atk/Def/Move/Range, and `ready` or `done` — the machine's DONE bit (§2.11.1), read from the view-model's presentation block and not from a snapshot flag: a waited unit reads `done` while its act flag is unspent. The flag unit's panel is red-edged and appends `FLAG — its loss ends the match.` - Empty when nothing is hovered. It never covers the board's lower-center where fighting happens.

Turn banner & AI playback. A brief `YOUR TURN` / `ENEMY TURN` banner marks the IGOUGO handoff (§2.1). The headless AI resolves instantly (§2.8); the presentation layer **replays its action list at a watchable fixed pace** (~0.5 s per action, camera stepping to each) so the player can read what the AI built, captured, and attacked — this is presentation pacing only, no rules change. Any click or Esc skips to the end state. First-session value: watching the AI's economy phase is how the player learns the enemy shares the same Fame economy (§2.9).

2.11.3 The attack forecast — the game's centerpiece display

Combat is a pure function (§2.6, §3 spec), so the forecast is not an estimate — it is the resolution, shown early. It appears on **hover** over any lit target from either the SELECTED or the MOVED state (§2.11.1); **LMB commits**, RMB/Esc cancels. A commit from SELECTED spends the act with the move unspent and retires the unit all the same. The card:

```
+----------------------------------------+
| ATTACK FORECAST                        |
| Artillery  →  Tank      (Woods +20%)   |
|                                        |
|  You deal    3 dmg      Tank 20 → 17   |
|  Counter     0          out of range   |
|                                        |
|  [LMB] Commit        [RMB / Esc] Cancel |
+----------------------------------------+
```

(Values follow the §3 formula: Artillery atk 10 at full HP vs Tank def 5 in Woods → round(10 × 1.0 × 0.8) − 5 = 3; counter 0 because the attacker sits outside the Tank's range 1.)

Hard rules for the card:

- **The defender's terrain bonus is named inline** (`Woods +20%`), every time — terrain defense must never read as hidden dice. (Per the §3 invariant, it is always the *defender's* hex; the card's placement of the modifier next to the defender teaches that for free.)
- **The counter line is never omitted, and always states its reason**: a number, `out of range`, or `defender destroyed`. This line is where the positional RPS triangle (§2.4) becomes visible — there is no type chart to consult, deliberately, so `Counter 0 — out of range` *is* the Artillery-beats-Tank lesson and `Counter 5` on an adjacent brawl *is* the Tank-beats-Recon lesson.
- **HP is shown before → after** for the defender (and for the attacker whenever the counter is nonzero), so the HP-scaling term of the formula (§2.6 — a damaged attacker hits softer) is observable across fights rather than asserted.
- **Lethal forecasts state their reward**: if the defender dies, the card appends `Destroys Tank · +150 Fame` (kill ≈ half cost, §2.7) — the tiebreak's combat-Fame criterion (§2.8) is priced on the commit card, not discovered at the cap.
- **Flag warning band**: if the forecast is lethal to *either* flag, the card gains a band — `FLAG AT RISK — this attack ends the match` (own flag: red; enemy flag: gold, with `+500 · Decisive victory`). No player can end a match, theirs or the enemy's, without having been told on the card they clicked.
- **Determinism is restated once**, via the first-attack one-shot (§2.11.6): `Check the forecast. It is exact — what you see is what resolves.` After that the card carries no reassurance text; the outcomes matching the card, every time, are the proof.

Selecting Artillery renders its attack ring as range 2–3 **with the range-1 hole drawn visibly dark** — the dead zone on the map is the "Recon runs it down" lesson in pixels, using the attack-target highlight already budgeted.

2.11.4 The live Fame scoreboard

The scoreboard exists because of revision §1.5-#1: the tiebreak must never be a hidden win condition. It is persistent (top-left), compact, and its **rows are ordered top-to-bottom in exact tiebreak order (§2.8)** — the layout *is* the rule, read passively all match:

```
+---------------------------+
| TURN 12 / 20              |
|---------------------------|
|             YOU     ENEMY |
| Destroyed   450     600 ◂ |
| Objectives  4/8     3/8   |
| Unit HP     47      55    |
+---------------------------+
```

- **Turn counter** against the cap, always. (`/ 20` is *Ferrum Crossing*'s cap, §2.13.2; the cap is per-scenario data, so the widget reads `turnCap` from the scenario rather than hardcoding a number.)
- **Destroyed** = combat Fame earned (kills and the flag bonus, §2.7; there is no undamaged-strike bonus — Q6, §4.7) — **passive income is excluded**, exactly as the tiebreak excludes it. Hover tooltip: `Fame from kills. Factory income does not count at the cap.` This row deliberately does *not* equal the spendable Fame pool (top-right widget), and the tooltip on each names the difference — the one place the single-currency design (§2.7) needs a disambiguating sentence.
- **Objectives** as *X of N* over all factories + capturable towns (§2.8 criterion 2), N supplied by the scenario (§2.13) — **N = 8** on *Ferrum Crossing* (4 factories + 4 towns), as the mock shows.
- **Unit HP** = surviving strength (criterion 3), listed last because it *is* last.
- A **chevron (◂)** marks the current attrition-tiebreak leader, evaluated in criteria order, and flips visibly when the lead changes. It is drawn beside the leading side's value — in the mock above the enemy leads at criterion 1, 600 combat Fame to 450 (§2.8: higher wins), so the chevron sits on the enemy column. If both Destroyed values are zero, the chevron is replaced by `— no engagements —` spanning the row: the mutual-passivity draw (§2.8) made visible before it bites.

- **Cap-approach banners** (transient, once each): at cap−5, `5 turns to the cap. The scoreboard decides a capped match.`; additionally, if both sides are still at zero combat Fame, `No engagements. A capped match with no combat is a draw.`

End-of-match screen. The result is the tier first (§2.8 — Decisive / Marginal / Draw), then the same three rows in the same order, so the verdict is always a restatement of what was on screen all match. Beneath the tier, one **faction-voiced result line**, written to the setting and voice guide (`kb/setting.md`), which supplies all three constraints on it: faction voice appears only in result-screen text, a result line is **≤ 30 words**, and the register is field-manual plain — terse tactical briefing, substance over drama, no melodrama or fantasy filler. The same file's two faction blocks supply the voices the samples below are written in — the **Directorate** cold, doctrinal, bureaucratic-military, framing every outcome as a matter of record; the **Vanguard** terse, pragmatic, defiant, measuring everything by ground held — and its pipeline note retrieves a faction block *only* for this screen, which is why faction voice appears nowhere else in the UI. Samples, one per case, generated content to follow these:

- Directorate, decisive: `Command directive fulfilled. The enemy flag is struck from the record. Order is restored.`
- Directorate, marginal: `The cap is reached. The ledger favors the Directorate. The record stands.`
- Vanguard, decisive: `Their flag is down. We hold the ground. That's the whole report.`
- Vanguard, marginal: `Cap hit. We did the damage; they held the rear. The ground says we win.`
- Draw, neutral system voice: `Turn cap reached. Attrition equal. Recorded as a draw.` / mutual passivity: `Turn cap reached. Neither side engaged. Recorded as a draw.`

2.11.5 Production menu & match-flow surfaces

Production menu — opens on LMB on an own factory (or `B`), anchored beside it:

```
+-------------------------------------+
| FACTORY — BUILD          Fame: 250  |
|-------------------------------------|
|  Infantry   100   [Build]           |
|  Recon      150   [Build]           |
|  Artillery  200   [Build]           |
|  Tank       300    ----   (need 50) |
|-------------------------------------|
|  Spawns here, or adjacent if        |
|  occupied.                          |
+-------------------------------------+
```

- Unaffordable rows are greyed with the shortfall named (`need 50` at the mocked 250-Fame pool against the Tank's 300 cost, §2.4), never hidden — the price list is also the strategy lesson (§2.4 costs).
- The spawn rule (§2.7: factory hex if free, else an adjacent free hex) is one static line in the footer. If the factory is fully boxed in, the footer swaps to `Boxed in — build waits for a free hex.` and Build buttons disable: the space-throttle (§2.7) explains itself at the moment it applies. Both the swap and the disable read the snapshot's per-factory `spawnBlocked` — board geometry, computed by the module, no board scan the widget runs for itself. They do not read `buildWaiting`, which is the narrower §2.7 fact that a build is holding this factory's slot until it spawns: a boxed-in factory need not have anything queued.
- When any unit is affordable and the factory has not built this turn, the factory tile shows a small `BUILD` pulse — the nudge that connects hoarded Fame to an army (a first-session failure mode, §2.11.6 ledger). The second half of that condition is read rather than inferred: the factory's own `hasBuiltThisTurn` — the per-factory build record the rules module already holds — out of the snapshot's per-factory block `{hex, owner, hasBuiltThisTurn, buildWaiting, spawnBlocked}`.
- The row hover shows the unit's stat line in the info panel, so buying is done with the §2.4 table in view.

Pre-match: the static briefing overlay (three callouts: flag, Bridge, factories — §2.11.6-A). **Post-match**: the end screen above. That is the complete screen list for the prototype: title/menu, briefing, match, result. No settings screen beyond volume + resolution is budgeted (Enhanced Input remap is a polish item).

2.11.6 Onboarding — the first session

Philosophy. No manual, no tutorial mode, no modal text walls. Three teachers:

1. **Constraint** — the first turn removes every option except the one being taught.
2. **The forecast** (§2.11.3) — deterministic combat means every attack is a free, truthful

lesson in terrain defense, range, and HP scaling. The plan makes the player *read* the forecast once, early; after that the game teaches itself.

3. **One-shot event tips** — a single system-voice line (neutral system voice per the tone bible in `kb/setting.md`; ≤ 30 words, borrowing that file's result-line ceiling — one-shot tips are not a length category it names), fired the first time a concept becomes relevant, never repeated (boolean flags in the save slot, §4.1).

The first match runs on the one shipped scenario at **Easy** by default (player +150 opening Fame, §2.9) with a **guided opening**: four scripted directives inside a fixed four-turn window — the first appears on turn 1, the strip and every beat behind it are gone for good at the end of turn 4, then hands-off. Any completed match on the save skips all guidance automatically; a `Skip guidance` control kills it instantly for anyone, and kills the guided opening's board state with it — the objective ring and the turn-1a unit marker clear in the same frame as the strip.

A. Pre-match briefing — a dimmed board, three anchored callouts, click-through (~5 s): **your flag** (`If it falls, you lose. It cannot be rebuilt.`), **the Bridge** (`The only land crossing.`), **factories** (`Hold them for Fame. Fame builds units.`). Why only three: the flag is the win condition, factories are the economy, and the Bridge is the map-reading trap — a player who misses the single Water crossing misplans the whole match.

B. Guided opening (turns 1–4), via a one-line directive strip at top center, one instruction at a time, each retiring on completion:

Turn	Constraint	Directive	Teaches	Retires when
1a	Only one marked Infantry selectable; others dimmed (hover: `Locked this turn.`). The two states behind that surface read from two places. *Marked* — whether this unit is the scenario's `guidedOpening.infantry` — is the snapshot field `isGuidedMarked`, which the turn-1a unit marker reads out of the snapshot. `lockedThisTurn` is per-unit and per-turn, owned by the guidance layer, so the dimming and its hover string read it out of the view-model's presentation block. End Turn is inert until that Infantry has moved (hover: `Move the marked Infantry first.`), and while 1a is outstanding that Infantry cannot retire itself without moving: its attack targets are not lit, so the SELECTED → attack transition (§2.11.1) is closed to it, and **Space** is inert for it on the same footing as End Turn and for the same reason. Those are the machine's only two routes from SELECTED to DONE that do not pass through MOVED, so both are closed for that one unit and nothing the player can do leaves End Turn inert with no move left to satisfy it — this is what makes 1a retire inside turn 1 in every branch, and it is the only guided-opening constraint that gates a player *input* rather than a selection, adopted under **Q27** (§4.7), ruled — it was registered rather than assumed because it gates an input	`Select the marked Infantry. Lit hexes are its true reach. Click one to move.`	Selection; the highlight is the real move set (§2.5)	Move completes
		`End turn.`	IGOUGO	

Turn	Constraint	Directive	Teaches	Retires when
1b	End Turn locked	`The enemy moves; then you.`	(§2.1) — the player watches a full AI turn	End Turn ends
2 *(standing)*	None on selection. The scenario's designated neutral factory (`guidedOpening.objective`, §2.13.1) is ringed from turn 1; its info-panel line appends `Only Infantry captures.`	`Move the Infantry onto the ringed Factory. Only Infantry captures.`	Capture; the Infantry-only rule (§2.7)	A capture pip appears — on whatever turn that happens. *Standing* means it stays **outstanding**, not that it holds the strip. How many turns it actually holds the line is decided by rules 1–2 and by when the pip lands, and the schedule table's three branches are exhaustive: **twice** — turn 2, then a turn-4 rule-2 last call, if the pip has still not landed (wandered); **once** — turn 2 only, retiring on a turn-2 or turn-3 pip; or **never** — the pip lands on turn 1 and beat 2 retires before rule 1 can select it (fast lane). On every turn it does not hold the line it runs on the ring and the unit marker. Hard-expires at end of turn 4
3	None. Fame ≥ 100 whenever this beat is outstanding, and not because of income: builds are Fame's only sink (§2.7), and the beat retires on the first spawn — so an outstanding beat 3 means nothing has been spent and the player still holds **at least their opening Fame**, which is 350 at §2.11.6's default Easy tier and never below Infantry's 100 at any tier the player can pick (Normal 200, Hard 100 — §2.7, §2.9). Turn-1 income is not assumed anywhere in this beat: there is none (Q8, §4.7)	`Spend Fame at your Factory. Infantry costs 100.`	Fame → factory → unit	A unit spawns — on whatever turn that happens, including turn 1

The Turn column is the beat's **order index** — the turn it takes the line in the common case — not a floor. Rules 1–2 below assign the line, so a beat moves up whenever a lower-numbered beat has already retired: beat 3 takes turn 2 in the fast-lane branch of the schedule table. The strip shows **one directive at a time**, and the line is assigned at the

start of each turn by two rules in order:

1. the lowest-numbered **outstanding** beat that has **not yet held the line on an earlier turn**;
2. if every outstanding beat has already had its turn on the line, the lowest-numbered outstanding beat — a **last call**.

A beat gives up the line either the instant it retires (which is how 1a hands off to 1b inside turn 1) or at the end of the turn it first appeared, whichever comes first. **Giving up the line is not retiring.** An outstanding beat that has yielded keeps running on its board state: for beat 2 that is the ringed objective and the marked Infantry, which are the same instruction in spatial form and have been on screen since turn 1. The text line introduces the objective; the ring is what reminds.

The consequence that matters: **no beat expires unheard.** Each beat either retires on its own event — its lesson landed in play, which is exactly what beat 2 does on turn 1 in the fast-lane branch, before it has ever held the line — or it holds the line at least once before the window closes, which is what rule 2 exists to force. A beat 2 that hangs — the *Ferrum Crossing* 1-MP-slack case below — cannot starve beat 3, whose Fame → factory → unit lesson is the one the §2.11.6-D ledger confirms with a bought unit on the board. Rule 2 then makes turn 4 a last call for whatever is still outstanding, so the strip's final turn is spent on the player's actual gap rather than on order-of-arrival:

Turn	Common case — pip lands turn 2	Wandered case — pip lands turn 3 or 4	Fast lane — pip lands turn 1 (*The Causeway*, 3 MP lanes)
1	1a, then 1b when 1a retires	1a, then 1b	1a, then 1b; the pip lands inside this turn, so beat 2 retires **without ever holding the line**
2	beat 2 (rule 1) — retires on the pip	beat 2 (rule 1) — holds, then yields	beat 3 (rule 1)
3	beat 3 (rule 1)	beat 3 (rule 1)	beat 3 — rule 2 last call, **untagged**
4	beat 3 — rule 2 last call, tagged	beat 2 — rule 2 last call, tagged	beat 3 — rule 2 last call **again**, tagged
end of 4	strip gone; anything still outstanding expires	strip gone; anything still outstanding expires	strip gone; anything still outstanding expires

How to read a cell. Each cell names the beat that rules 1–2 select for that turn, given the column's pip timing *and* beat 3 still outstanding when its cell is reached. Beat 3 retires on a spawn **event**, not on turn 3, and Fame ≥ 100 from turn 1, so a player who builds early retires it early. Every cell is therefore read as: *this beat — or, if it has already retired, the next beat rules 1–2 select — or nothing, if none is outstanding.* Turn 1 is identical in all three columns because 1a and 1b cannot outlive it: End Turn stays inert until the marked Infantry moves (beat table, row 1a), and 1b's retire condition **is** the turn boundary.

Is the strip ever quiet before the end of turn 4? No — there is no live-but-blank strip in this system. Rule 2 has no exit: while any beat is outstanding, some beat holds the line. The only empty state reachable before end of turn 4 is the strip being **gone**, and its condition is exact — *all four beats retired*: 1a and 1b inside turn 1, beat 2 on its pip, beat 3 on a spawn. The earliest that can occur is the end of turn 1, and only in the fast-lane branch with a turn-1 build; it is permanent, per the disappearance rule below, not a pause. In particular, fast-lane turn 4 is **not** unconditionally quiet: a player who has not bought a unit still has beat 3 outstanding, and rule 2 returns it to the line, exactly as the common-case column does at its own turn 4. That is the guarantee working, not an exception to it.

A **turn-4** last-call line is the beat's own text with a dim right-hand tag, so the player is never dropped mid-instruction without warning. The tag states a fact about the *window*, not about rule 2, so it renders on turn 4 only: an earlier rule-2 last call (fast lane, turn 3) shows the same line untagged, because guidance does not end that turn and the strip must not say it does:

```
+-------------------------------------------------------------------------
-+
|  Move the Infantry onto the ringed Factory. Only Infantry captures.
|
|                                                    guidance ends this
turn|
+-------------------------------------------------------------------------
-+
```

The strip disappears for good once all four beats have retired, and unconditionally at the end of turn 4; every beat, the objective ring, and the turn-1a marker expire with it.

Why beat 2 is a standing directive, not a turn-2 deadline. Its target is guaranteed by §2.13.1's **opening-capture reachability** invariant: for each seat, at least one Infantry deployment hex has a Bridge-free land path to a **neutral factory** costing ≤ 6 movement points — two turns at Infantry Move 3 (§2.4) — and the scenario file names that unit and that factory in `guidedOpening.infantry` and `guidedOpening.objective`. The directive strip reads exactly those two fields: `guidedOpening.infantry` is the Infantry marked in beat 1a, `guidedOpening.objective` is the factory ringed from turn 1. Nothing is measured at runtime and no "nearest objective" heuristic is used — the lane is authored, machine-validated, and recorded as a number by `validate_scenario` (§4.2), so the onboarding and the map can never disagree about which factory the player was told to take.

What the invariant does *not* buy is a safe turn-2 deadline, and that is why beat 2 retires on an event rather than a turn number. Beat 1a hands the player a free move in any direction — that is its whole lesson, and the onboarding must not punish the player for using it. On *Ferrum Crossing* both lanes cost **5 movement points**, not 5 and 4 hexes: West (1,5) → South (5,7) is 5 hexes of Plains at cost 1, and East (9,3) → North (6,2) is 4 hexes but one is a mandatory Woods ring hex at cost 2 (§2.3). Against the 6 MP budget that is **1 MP of slack**, the tightest of the three maps and the only one that is tight at all: both stretch maps were redrawn on even row counts (*Longwater March* 13 × 8, *The Causeway* 9 × 8, both `rot180`), and their lanes price at **4 MP / 4 MP** and **3 MP / 3 MP**, so they carry **2** and **3** MP of slack respectively (§2.13.1's table). No lane on either 8-row map carries 4. A single turn-1 step spent walking off the lane therefore pushes the pip to turn 3 — and a hard turn-2 retire condition would strand the strip on a directive the player had already been made unable to satisfy that turn. The standing directive absorbs precisely that: on *Ferrum Crossing*, where a turn-1 pip is impossible at 5 MP per lane against Move 3, it takes the line on turn 2, stays outstanding until the pip appears, and hard-expires at the end of turn 4 — the last turn of the guided window, not one turn past it. It persists as an *objective*, not as a line of text: it holds the strip for turn 2, yields the line to beat 3 for turn 3, and returns for a turn-4 last call only if the pip still has not landed. That is **two turns on the line at the outside** — one in the common case, where the turn-2 pip retires it, and none at all on a map whose lanes let the pip land on turn 1 (the fast-lane column). That is the whole price of "standing", and it is paid in ring and marker rather than in strip time, so the Fame → factory → unit lesson is never the thing that gets crowded out by a slow walk. Ringing the objective from turn 1 biases beat 1a's free move onto the lane without constraining it, so in the common case the slack is never spent and the pip lands on turn 2 as designed.

"Capturing" by turn 2, never "captured." The invariant promises the Infantry is *standing on* the factory at the end of turn 2, not that the tile is yours: under Q4's N = 1 reading the tile flips at the start of turn 3 (§2.7). So the directive reads `Move the Infantry onto the ringed Factory` — never "capture it" — and it retires on the **capture pip**, the arrival receipt, not on the ownership flip. The flip gets its own confirmation one turn later via the `+100 Fame – Factory` toast, which is where the concept ledger's Capture row already ends. This wording is also correct if N is ever ruled 2: the pip is the arrival event either way.

C. Event-driven one-shots (once each, at first relevance): first attack hover → `Check the forecast. It is exact – what you see is what resolves.`; first Artillery strike at range 2–3 → `No counter at this range. Artillery strikes beyond reply.`; first Recon-vs-Artillery forecast → `Recon closes fast. Artillery cannot answer at range 1.`; first Tank-vs-Recon forecast at range 1 → `Armor wins the adjacent fight.`; first Water hover with a unit selected → `Impassable to land. Cross at the Bridge.`; first Bridge hover → `The only crossing. -10% defense – do not stall on it.`; first combat Fame → `+150 Fame. Kills count at the cap. Income does not.` *(amount = actual award)*; first repair blocked by adjacency → `No repair – enemy adjacent. Break contact to repair.`; first repair tick → `+[N] HP – repaired at Factory.`; plus the two cap-approach banners (§2.11.4).

D. Concept ledger — every concept, its failure point, its teach moment, its confirmation:

Concept	Where a first-timer gets lost	Taught at	Confirmed landed when
Hexes & movement	Clicks the map with nothing selected	Turn 1a: single selectable unit + reach highlight	Move completes; directive retires
IGOUGO turns	Expects simultaneous movement	Turn 1b: pulsing End Turn, then watching the AI's turn	Player ends turn 2 unprompted
Terrain move cost	Lopsided highlight (shallow into Woods/Mountains) reads as a bug	The asymmetric highlight + info-panel hover line	Player routes around a Mountain, or hovers 2+ terrains in one selection
Terrain defense	Same attack, different damage → suspects hidden dice	Forecast names the defender's bonus inline, every time (§2.11.3)	Resolution matches forecast, every time
The Bridge	Reads Water as decoration; "half the map is unreachable"	Briefing callout + Water never lights in any reach set + hover one-shots	First previewed or executed path crosses the Bridge
Forecast / determinism	Veterans hedge for RNG; novices fear committing	First-attack one-shot: `It is exact.`	Player commits; outcome equals card; usage becomes habitual
Positional RPS triangle	No counter chart exists — the triangle is emergent from range and move, invisible until experienced	The counter-reason line (§2.11.3), the three edge one-shots, the Artillery dead-zone ring	One uncountered Artillery strike lands — the range-2–3 one-shot (`No counter at this range.`) is the receipt, the undamaged-strike toast having been cut with the bonus (Q6, §4.7)
Capture	Parks a Tank on the factory and waits	Turn 2 directive; non-Infantry on a capturable tile shows disabled `Capture — Infantry only.`	Pip → tile recolors → next turn's `+100 Fame — Factory` toast
Fame income & build	Fame is abstract; player hoards, never connects factories → army	Beat 3, which holds the strip on turn 2 or turn 3 in every branch where the player has not already built — and where they have, the lesson landed without it (B); income toasts; `BUILD` pulse when affordable; greyed rows with shortfall (§2.11.5)	A bought unit spawns and stands on the board carrying its unacted pip — the **event**, not the directive, so the row also confirms for a player who skipped guidance
Flag = the game	Loses one specific Tank in a "fair trade"; match ends out of nowhere	Briefing callout; persistent `H`; red-edged info panel; flag warning band (§2.11.3)	Any lethal-to-flag forecast has been shown before any match can end
Turn cap & tiebreak	Assumes "most stuff wins"; income exclusion and passivity-draw are the least guessable rules	Scoreboard rows in tiebreak order (§2.11.4); first-combat-Fame one-shot; cap−5 banners	Chevron flips noticed in play; end screen matches the board watched all match
Repair	Heal silently fails next to an enemy; reads as a bug	Eligibility pip; the `enemy adjacent` one-shot fires at the *blocked* case first if it occurs	`+[N] HP — repaired` toast at turn start

2.11.7 Art (unchanged)

Flat/low-poly color-coded hexes, simple unit meshes or billboarded icons, generative/agent-assisted. Feel comes from subtle tweens and clear feedback, not VFX. One readability addition, cheap and worth naming: ownership is **double-coded** (faction color *and* a shape/border difference), so faction reading never depends on hue alone.

2.11.8 Build ranking (solo dev, UMG)

Must-have for a playable first session: selection state machine + input table (§2.11.1); reachable highlight and path preview (the path is already computed by §2.5 pathfinding — rendering it is cheap); forecast card with terrain-inline, counter-reason, HP before→after, kill-Fame line, and flag band; scoreboard in tiebreak order with turn counter and chevron; Fame pool + income widget; production menu with grey/shortfall/boxed states; info panel; End Turn + idle confirm; turn banner + paced AI playback with click-to-skip; toast queue; flag `H` markers and unacted pips; directive strip with the four beats; the one-shot system (string table + boolean flags); pre-match briefing overlay; end-of-match screen with tier + rows + one faction line.

Polish: chevron flip animation; Artillery dead-zone shading (plain target highlight conveys most of it); cap banners' timing tuning; camera smoothing and zoom-to-action; hotkey remap via Enhanced Input; colorblind palette variants beyond the default double-coding; hover-delay tuning; a dedicated micro-tutorial map (explicitly stretch — it would occupy a stretch scenario slot, §2.10).

2.12 Lineage extraction — what we kept, diverged, and cut from *Conflict*

The prototype was digested against the original *Conflict* (Vic Tokai, NES, 1989) manual so the lineage is a deliberate set of decisions, not an accident. **Kept and mapped:** hex board, terrain move/defense, capturable towns/factories, factory production, a single Fame currency, held-objective income, and destroy-the-commander victory. **Extracted as new pillar-fitting features:** the *Bridge* terrain (§2.3), *Repair* at owned objectives (§2.7) — the one survivable slice of *Conflict*'s logistics — *starting Fame + difficulty-as-Fame-handicap* (§2.7, §2.9), the *territorial-domination* secondary win (§2.8), and an all-1.0 *type-effectiveness* lever (§3 spec). **Deliberately diverged/cut** (recorded so the choice is on the record, not implemented): the real-time **battle mini-game** (NORMAL/AUTO + weapon/maneuver menus + evasion %) → replaced by the deterministic forecast (Pillar 1); **fuel/ammo logistics** and their supply units → cut for pacing; the **16-map password campaign** → one polished scenario; and *Conflict*'s **combat Fame penalties** (retreat/base/flag) → omitted so a flag kill can never be out-piled by attrition (Pillar 2, §1.5 #5). **Declined levers:** the per-turn **activation cap** (3-Units mode) and **one-type-per-turn** production — both lean against Pillar 2 or duplicate the existing board-space throttle.

2.13 Scenario & map design

> *All layout values are starter/tuning targets, per the GDD's standing convention (§2.3, §2.7). Nothing here adds a unit, terrain, or rule: the palette is the six movement terrains + the capturable Factory tile (§2.3) and the four-unit roster (§2.4), exactly.*

2.13.1 Layout conventions (shared by every map)

- **Coordinates:** `(col, row)`, col 0 = west, row 0 = north; pointy-top hexes (§2.2), odd rows offset +½ hex east (odd-r). ASCII glyphs: `p` Plains · `w` Woods · `m` Mountains · `~` Water · `B` Bridge · `T` Town · `F` Factory.
- **Factories:** each side owns exactly one **home factory** at start; all others are **neutral** (no income until captured, §2.7). The factory set is also the §2.8 territorial-domination win set, so factory count and placement is the primary match-length dial (see 2.13.5).
- **Starting force (standard): 5 units per side — 1 Flag Tank (§2.4, not producible), 2 Infantry, 1 Artillery, 1 Recon** — plus 200 starting Fame (§2.7) ± the §2.9 difficulty handicap. Rationale: one of each producible system is live from turn 1, so the positional RPS triangle and capture are in play before the first build; two Infantry means the standard opening (one to a town, one to a neutral factory) doesn't consume the only capturer; the producible force is worth 550 Fame — about two turns of mid-game income — so losing the opening force is recoverable, not match-ending. *No section outside §2.13 sizes the starting force; this count is pending Director approval (Q15, §4.7), not silently adopted.*
- **Home factory hex starts empty** in every deployment, so the turn-1 build spawns on the factory itself (§2.7 spawn rule) instead of scattering.

- **Validation invariants** (for the §4.2 `validate_scenario` tool — schema per §4.7 Stub 7): every land-passable hex reaches every factory (Bridges are the only Water crossings, §2.3, and there is no sea unit — connectivity is a build-time check, not a hope); all deployment hexes are free and land-passable; factory count in the map file equals the count the domination check uses; declared symmetry is machine-verified **in axial** (§4.7's T-SCN-05 forbids loaded state from holding `(col, row)` at all, so the check has no other place to run). The declarable values are **`rot180` or `none`** — `mirror` is not one of them, because no odd-r rectangle has a *vertical* mirror axis at any dimension and no map in the set is drawn to the *horizontal* one, which exists only on an odd row count (see the symmetry note at the end of this section); and `rot180` is well-formed only on an **even row count**. Both constraints are **ruled** (Q24, §4.7) and are the shipped rule, not an assumption awaiting one. Whether the horizontal mirror was worth a third enum value — it being available and merely unused rather than impossible — was **answered no at Q26**: the enum stays at two values and T-SCN-10 stays unwritten.

- **Opening-capture reachability** — the invariant the guided opening (§2.11.6-B, turn 2) rests on, and the reason it can cite a scenario guarantee at all. For **each seat**, at least one Infantry deployment hex must have a land path to a **neutral** factory costing **≤ 2 × Infantry Move = 6 movement points** (§2.4 Move 3) and crossing **no Bridge**: two turns of movement then put that Infantry on the factory hex, and the capture pip appears without making a contested crossing the first lesson. The scenario file names that unit and that factory (`guidedOpening.infantry`, `guidedOpening.objective`, §4.7 Stub 7) so the turn-1a *marked* Infantry is the one already standing on the lane. Measured on the three maps as drawn (cost in movement points, cheapest legal path, Bridge-free):

Map	West lane	East lane
Ferrum Crossing	(1,5) → South **(5,7)**, **5 MP**, all Plains	(9,3) → North **(6,2)**, **5 MP** (4 hexes, one mandatory Woods ring hex)
Longwater March	(1,2) → **(4,1)**, **4 MP** (all Plains)	(11,5) → **(8,6)**, **4 MP** (all Plains)
The Causeway	(1,2) → **(3,2)**, **3 MP** (via the town at (1,1))	(7,5) → **(5,5)**, **3 MP** (via the town at (7,6))

  All three pass. Three things the invariant deliberately does *not* promise:

  1. **"Capturing by turn 2", never "captured by turn 2."** With capture N = 1 (§2.7, Q4) the Infantry stands on the factory at the end of turn 2 and the tile **flips on turn 3**. The turn-2 directive therefore retires on the *pip*, which is exactly what §2.11.6-B already specifies.

  2. **Slack is not uniform.** *Ferrum Crossing* carries only 1 MP of slack against the 6, so a turn-1 move spent walking away from the lane pushes the pip to turn 3 — still inside the guided window, which runs **turns 1–4** (§2.11.6-B). *Longwater March* carries 2 MP of slack on both lanes, *The Causeway* 3 on both; being symmetric, each carries the *same* slack in both seats, which is what makes them usable as controlled tests. Both stretch lanes are also clear of that seat's own four other starting units. Since the Q22/Q28 fix moved East's second Infantry to (9,1), which lies on none of the eight routes *Ferrum Crossing* prices, **no map's numbers move under either reading of Q21 (§4.7) as drawn** — Q21 is ruled terrain-only, and the shipped map is simply the one where a *future* deployment edit landing in a priced route would flip a gate.

  3. **Uncontested, and exactly what that buys.** The designated lane is the seat's *own* neutral — West → South, East → North on the shipped map — and under Q22/Q28 that is a **measured inequality**, not an authorial claim: the opposing seat's cheapest Infantry route to the same objective, ranging over **every** Infantry that seat deploys, must cost **strictly more MP** than the owning seat's lane (T-SCN-11, §4.7). Measured on the set: *Ferrum Crossing* **5 against 6** in both seats, *The Causeway* **3 against 5**, *Longwater March* **4 against 8**.

     An earlier draft of this note said "the first lesson is not a race" and stopped there. That was wrong on both counts. It was unmeasured; and on the shipped map as first drawn, East's *second* Infantry at (9,5) tied West's South lane at 5 MP flat — a race under any reading, and the reason §2.13.2's deployment now reads (9,1). What is true, stated at the resolution the number supports:

     - **No opposing Infantry can arrive for fewer MP.** That is the whole of the inequality, it holds on all six lanes in the set, and it is the property the gate checks.
     - **On the shipped map it does not buy a turn.** 5 MP and 6 MP are *both* two turns at Infantry Move 3 (§2.4). A 1 MP margin is daylight in movement

points, not in turns. What converts it into "you get there first" is **player-first IGOUGO** (§2.1): the player's seat moves before the AI inside every turn, so on the shared arrival turn the player stands on the factory hex first and an opposing Infantry walking the same lane finds the hex occupied and the capture pip already placed.

- **Deployment carries the rest of the load, and is meant to.** After the correction neither East Infantry has a cheaper errand in the south: both sit 5 MP from East's *own* North objective and 6–7 MP from West's South one. The southern contest is not merely lost on distance, it is uneconomic — which is a stronger guarantee than the gate can express and a weaker one than the gate can enforce, so the gate keeps the inequality and this note keeps the reasoning.
- **The stretch maps do buy a turn**, and that is the difference an honest note has to show. *The Causeway*'s 3-against-5 is a one-turn lane against a two-turn one; *Longwater March*'s 4-against-8 is two turns against three. The shipped map is the tightest in the set, on slack (1 MP, note 2) and on contest (1 MP) alike, and both facts have the same cause: it is the only map in the set that is not drawn to a symmetry.

Declared symmetry does imply equal lane cost — and that is what makes it worth declaring. A 180° rotation of a hex board is an isometry: it preserves hex distance, and because it maps every hex to a hex of identical terrain it preserves *path cost* too. On a genuinely symmetric map the two seats' lanes therefore cost exactly the same, and a 1 MP split is not an offset artefact — it is proof the layout is not symmetric. An earlier draft of this section argued the opposite from the odd-r row offset; that confused a storage convention with a geometry. §4.7's T-SCN-05 forbids loaded state from holding `(col, row)` at all, so the symmetry check necessarily runs in axial, where no offset exists and a declared symmetry is an isometry or it is nothing.

Two facts about offset rectangles follow, and both stretch maps are drawn to them:

1. **No odd-r rectangle has a *vertical* mirror axis, at any dimension — but one with an odd row count has a horizontal one.** Even rows occupy columns 0… W−1 and are centred on (W−1)/2; odd rows sit ½ hex east and are centred on W/2. The centres differ by ½ hex for every W, so a single **vertical** axis cannot serve both parities, at any dimension. The horizontal axis is a different question with a different answer: c ↔ c, r ↔ H−1−r is exact whenever H−1 is even — i.e. whenever **H is odd** — because r and H−1−r then share a parity, so every row keeps its ½-hex offset and no column moves at all. In axial it is one parity-free affine map, **μ(q, r) = (q + r − (H−1)/2, H−1−r)**, integer-valued exactly when H is odd. That is the precise complement of fact 2's even-row-count precondition, so an odd-r rectangle admits *at most one* of the two symmetries and its row parity chooses which: even H → 180° rotation, odd H → horizontal mirror, never both, never a vertical mirror. 11 × 9 *Ferrum Crossing* (§2.13.2) sits on a geometry that has such an axis and is deliberately not drawn to it (§2.13.4) — an asymmetry that is a design choice, not a geometric accident. Whether a horizontal `mirror` therefore becomes a declarable value alongside `rot180`, with an odd-row-count precondition, was **answered no at Q26** (§4.7): the enum stays at two values and T-SCN-10 stays unwritten. No map in the set declares one, and a later ruling could add it purely additively.
2. **An odd-r rectangle admits a 180° rotation only when the row count is even.** The rotation pairs row r with row H−1−r. With H odd those two rows share a parity, so each must land on a row carrying the same ½-hex offset — and a rotation is one rigid motion with one centre, while fact 1 has just shown the two parities' centres differ by ½ hex. Put that centre on the even rows, at (W−1)/2, and every odd row reflects as c ↔ W−2−c, which sends column W−1 to column −1. Put it on the odd rows, at W/2, and every even row reflects as c ↔ W−c, which sends column 0 to column W. Either placement throws one column of every second row off the board. In axial the same fact is pure arithmetic: the rotation constant W − H/2 is a half-integer on odd H, so *no* hex has a hex image — on 9 × 9 the constant is 4.5 and (1,1) rotates to column 6.5, which is why §4.7's T-SCN-09 refuses the file with a reason instead of reporting failed comparisons. With H even the parities alternate and the board closes exactly under **ρ(c, r) = (W−1−c, H−1−r)**. This is why *Longwater March* is 13 × **8** and *The Causeway* is 9 × **8** rather than nine rows each.

The validator still **measures and records each seat's cost as a number** (T-SCN-08) rather than reading it off the symmetry flag. Not because the flag is untrustworthy in principle, but because it is an *authored declaration* and measurement is the only thing that catches it being wrong. That is exactly what it caught here: the flag said mirrored, the numbers said 3 and 4, and the numbers were right.

2.13.2 The shipped scenario — *Ferrum Crossing* (§2.10 IN)

Spec	Value
Dimensions	**11 × 9 = 99 hexes**
Starting units per side	**5** (standard force, 2.13.1)
Factories	**4** — one home per side + 2 neutral (verbatim the §2.7 "~4 total" layout)
Towns	**4**
Turn cap	**20 turns** (the §2.8 per-scenario cap, Stub 7 `turnCap`)
Symmetry	**Asymmetric** — handicap story below
Estimated match length	**12–16 turns** (reasoning below)

Layout.

```
r0:  p  p  p  p  p  ~  p  p  p  p  p
r1:   p  p  p  T  p  B  w  p  T  p  p
r2:  p  p  w  p  p  ~  F  w  p  p  p
r3:   p  p  p  w  p  ~  w  p  p  p  p
r4:  p  F  p  p  p  B  w  w  p  F  p
r5:   p  p  w  p  p  ~  p  p  p  p  p
r6:  p  p  p  p  m  p  m  T  p  p  p
r7:   p  p  T  p  p  F  p  p  p  p  p
r8:  p  p  p  p  p  p  p  p  p  p  p
```

Key coordinates. Water (5,0)(5,2)(5,3)(5,5) · Bridges **(5,1)** north, **(5,4)** center · Home factories: West **(1,4)**, East **(9,4)** · Neutral factories: **North (6,2)** on East's bank, **South (5,7)** in the mountain pass · Towns (3,1)(8,1)(2,7)(7,6) · Pass mountains (4,6)(6,6) · Woods bands: East bank (6,1)(7,2)(6,3)(6,4)(7,4), West approaches (2,2)(3,3)(2,5).

Terrain distribution (99 hexes): Plains 75 · Woods 8 · Mountains 2 · Water 4 · Bridge 2 · Town 4 · Factory 4. Three-quarters open ground is deliberate: movement stays fast (§2.3 cost 1), contact comes early, and the match fits Pillar 2's 10–15-minute envelope — cover is a *purchase*, not the default.

Starting positions (all on Plains; home factory hex left free):

Unit	West	East
Flag Tank	(0,4)	(10,4)
Infantry ×2	(1,3), **(1,5)**	**(9,3)**, (9,1)
Artillery	(0,3)	(10,5)
Recon	(0,5)	(10,3)

Guided opening (§2.13.1). `guidedOpening.infantry` = **(1,5)** West / **(9,3)** East; `guidedOpening.objective` = **South (5,7)** West / **North (6,2)** East. Distinct objectives (T-SCN-07); **5 MP each**, Bridge-free (T-SCN-06); 1 MP of slack against the 6 MP ceiling, in both seats. This map previously named its guided units only in §2.13.1's lane table; they are declared here now, with the map they belong to.

Non-contention, measured (T-SCN-11; Q22 ruled, Q28 ruled). Q28 rules that "the opposing seat's Infantry" means **every** Infantry that seat owns, not only its marked guided unit — so this map is checked on **eight** routes, not two. All eight, priced identically: Stub-3 cheapest path, terrain alone with occupancy excluded (Q21), every hex entered counted including the objective, Bridges permitted on opposing routes and forbidden on the two guided lanes.

Infantry	→ North (6,2)	→ South (5,7)
West (1,3)	**6** — (2,3)(3,2)(4,2)(4,1)(5,1)B(6,2)	**6** — (2,4)(3,4)(3,5)(4,5)(5,6)(5,7)
West **(1,5)** *guided*	7 — (2,4)(2,3)(3,2)(3,1)T(4,1)(5,1)B(6,2)	**5** — (2,6)(3,6)(3,7)(4,7)(5,7)
East **(9,3)** *guided*	**5** — (8,3)(7,3)(6,3)w(6,2)	**6** — (9,4)F(8,5)(8,6)(7,7)(6,7)(5,7)
East (9,1)	5 — (9,2)(8,2)(7,2)w(6,2)	**7** — (9,2)(8,3)(8,4)(7,5)(7,6)T(6,7)(5,7)

Both entries pass, and both report the same pair: 5 against 6. West's South lane costs 5; East's cheapest Infantry route to (5,7) costs 6, from the guided hex (9,3). East's North lane costs 5; West's cheapest route to (6,2) costs 6, from (1,3) over the north Bridge. Strictly longer in both seats, which is the ruled comparison — equality fails, and equality is what this map used to report.

Two structural facts the table makes visible. **Every route to North costs exactly 1 MP more than its hex distance**, because the Water column forces West onto a Bridge and the Woods ring — (6,1)(7,2)(6,3), the only land approaches to (6,2) that are not the Bridge —

forces East through cover. **Every route to South is a plain geodesic at 1 MP per hex**, because rows 6–8 are open Plains that the two pass Mountains do not close. That is the map's economy in two lines: the north is priced by terrain, the south is priced by distance alone, and the south is therefore the half where deployment is the *only* thing standing between two Infantry and the same factory.

Why East's second Infantry sits at (9,1) and not (9,5). At (9,5) it was 5 MP from *West's* South objective — a dead tie with West's own lane, which T-SCN-11 refuses, correctly: a tie is a race and a race is what the invariant exists to prevent. It could not be nudged and stay southern. Every **free** hex in East's southern quarter measures 4–5 MP to (5,7); the only two hexes down there that clear 5 are (10,5) and (10,4) — the Artillery's and the Flag Tank's — so staying southern means pushing a fragile ranged unit or the flag itself onto the outer column for 1 MP of margin. The cause is geometric and worth stating: East's south town (7,6) is only 2 hexes from South (5,7), so by the triangle inequality anything within 3 MP of that town is within 5 MP of that factory. A capturer that covers East's southern town is *necessarily* a racer for West's southern factory.

So the fix is a change of flank, not a shuffle — and it costs East nothing in opening income. East's second Infantry was ever only the early capturer for **one** town; at (9,1) it is adjacent to the north town **(8,1)** and captures on turn 1 exactly as it did at (9,5) for (7,6). One town in the opening either way; north instead of south. What East actually gives up is the southern picket: town (7,6) now waits for a turn-1 Infantry build — 100 of the 200 starting Fame (§2.7), spawning on the deliberately empty home factory (9,4), 3 MP and therefore one turn from (7,6) — and East's opening south flank is Artillery (10,5) behind the Flag Tank (10,4). That is a one-turn, 25-Fame-per-turn delay on one town, paid for out of starting Fame, against a guided lane that stops being a race. It is also the deployment finally agreeing with the sentence this section has always printed: *East's prize is North.*

Terrain as economics, hex by hex. The river spans rows 0–5 only, crossed at two Bridges (−10% defense, §2.3): a unit forcing a crossing under Artillery (range 2–3, no counter, §2.4) buys ground with HP — the intended price. The river deliberately does **not** bisect the map: the southern pass (row 6) is a bridge-free route priced by two Mountains (cost 3, +40%), so it is the *slow* flank, never a free one. The one-hex river also means opposite banks are distance 2 — inside Artillery range, and the prototype ships without line-of-sight blocking (§2.2) — so bank control is contested by fire even before anyone crosses. East's bank carries the Woods band (+20%): cover in exchange for the longer road south. West gets open Plains: speed in exchange for fighting uncovered. The South neutral factory sits between the pass Mountains — +15% defense and a spawn point anchoring the southern flank; the North neutral sits on East's bank, so West must win a bridge to contest it. Rear towns (3,1)/(2,7) west and (8,1)/(7,6) east are the repair points (§2.7): the anti-fortress clause (no repair adjacent to an enemy) keeps the forward ones from healing a frontline garrison.

The tactical question this map asks: *which neutral factory do you race, and which bridge do you contest — knowing the two answers pull a 5-unit army in opposite directions?* West's fast prize is South (open Plains approach); East's is North (no crossing needed). Committing to both splits the army into halves that lose either fight.

Asymmetry and its handicap story. Starting Fame and forces are identical; the asymmetry is purely spatial and intended to be self-balancing. Each seat is closer to a *different* neutral factory — measured, and by exactly **1 MP in each seat**: West's cheapest Infantry route is **5 MP to South against 6 to North**, East's is **5 MP to North against 6 to South** (all four figures in the eight-route table above). Earlier drafts asserted that sentence without pricing it, and for East it was simply false: with an Infantry at (9,5), East was **equidistant** — 5 MP to South and 5 MP to North — so the seat the map called the northern one had no measurable preference at all, and West's southern lane was a tie rather than a lane. The deployment above is what makes the sentence true, and the four numbers, not the sentence, are the claim. Each seat's advantage (West: tempo and open approaches; East: cover and a crossing-free path to its prize) is the other's problem. If §4.1 self-play shows either seat above ~55% win rate, the corrective is the existing §2.9 dial — a per-seat starting-Fame offset — never a terrain rework. This is also the replay lever: the two seats are two different deterministic puzzles (see 2.13.4).

If one side holds both bridges: the other side is not locked out — the southern pass exists precisely so bridge control here is *tempo*, not a topological wall. A double-bridge holder who then sits earns zero combat Fame and loses the §2.8 cap tiebreak (or draws under the mutual-passivity guard); meanwhile the cross-river Artillery duel and the open southern pass keep combat Fame available to both sides. Full lockout as a map premise is reserved for *The Causeway* (2.13.6), where the tiebreak rules carry the whole anti-turtle load by design.

2.13.3 Match-length reasoning — and the dial it exposes

Estimate: 12–16 turns; cap at 20. Not just the number:

- **Contact turn 3–4.** Homes are 8 columns apart over mostly cost-1 Plains; Tank (move

5) reaches a bridge or the pass mouth in 2–3 turns, Infantry (move 3) in 3–4.
- **Economy online turn 3–4.** Each side's near Infantry reaches its closest neutral factory in ~2 turns; capture at N=1 (fixed from §2.7's "start N=1–2" range — Q4, §4.7) flips income the following turn. "The following turn" counts the **capturing side's own turns from the turn ownership flips** — not from the turn its Infantry moves onto the tile, and not the next turn in the I-GO-U-GO alternation (§2.1), which is the opponent's. Income ramps 100 → ~225–250 Fame/turn: a reinforcing unit every 1–2 turns, continuously feeding the fight without flooding it (board space is the throttle, §2.7).
- **Kill speed.** Damage is punchy against 8–20 HP pools (§2.4): units die in 2–3 hits, so bridge and pass fights *resolve* rather than accumulate. Mid-game runs turns 5–12; the flag hunt closes 12–16.
- At ~45 s/player-turn that is ~10–13 minutes — inside §1's "10–15 typical, ~20 at the cap".

The match-length dial is factory count × home separation. More neutral factories → steeper income ramp → bloodier, faster mid-game and a wider "objectives held" spread at the cap; fewer → slower armies and more stall risk. Each column of home separation moves the contact turn back roughly one-for-Tank-move. Scenarios tune these two numbers first; unit and terrain stats are never per-map (those belong to rules and balance, §3).

2.13.4 Replayability — configurations, not mechanics

Stratocracy is deterministic end-to-end (Pillar 1, §2.6, §2.9's tier-invariant AI), so any line that beats the AI once beats it forever; by match ~4 a single mirrored map is a solved puzzle. The replay unit is therefore a **configuration = map × seat × difficulty handicap**, each a distinct deterministic puzzle:

Matches	Configuration	Ships in
1–3	*Ferrum Crossing*, West seat, Easy → Normal → Hard (§2.9 Fame ladder)	Core
4–6	*Ferrum Crossing*, **East seat**, same ladder	Core (seat-select = scenario data + one menu affordance)
7–8	*Longwater March*, Normal → Hard	Stretch P1 (§4.4 wk 4)
9–10	*The Causeway*, Normal → Hard	Stretch P2 (wk 4)

If no stretch lands, the shipped scope alone moves the cliff from match ~3 to match ~6 — that is what the asymmetric map buys, and why *Ferrum Crossing* is not mirrored. Honest ceiling: determinism means every configuration is eventually solved; this multiplies puzzles, it does not make them unsolvable. The long-term fixes (AI second pass, map-gen MCP toolset) already sit in the GDD's stretch column and are not re-proposed.

2.13.5 Stretch scenario — *Longwater March* (P1, §4.4 wk 4)

Spec	Value
Dimensions	**13 × 8 = 104 hexes** — eight rows, not nine, because a 180° rotation only closes on an even row count (§2.13.1)
Starting units per side	5 (standard force — one variable at a time; this map's variable is factory count)
Factories	**6** — one home per side + 4 neutral *(above §2.7's "typical ~4", inside its "two or more neutral"; pending Q19, §4.7)*
Towns	4
Symmetry	**180° rotational**, ρ(c, r) = (12−c, 7−r) — fair and admittedly dull; chosen so the factory-count dial is the only variable under test. *Mirrored* was the earlier declaration and is withdrawn: no odd-r rectangle has a vertical mirror axis at any dimension, and the horizontal one exists only on an odd row count — this map is 8 rows (§2.13.1). Rotation costs this map nothing it wanted from mirroring — it is equally distance-preserving, so both seats' lanes cost 4 MP and the only asymmetry left is the one under test.
Estimated match length	**16–20 turns, frequently reaching the cap**

```
r0:  m  p  p  p  p  p  T  p  p  p  p  p  m
r1:   p  p  p  p  F  p  p  p  F  p  p  p  p
r2:  p  p  p  p  p  p  p  p  p  p  p  p  p
r3:   p  F  p  T  p  w  w  p  p  p  p  p  p
r4:  p  p  p  p  p  p  w  w  p  T  p  F  p
r5:   p  p  p  p  p  p  p  p  p  p  p  p  p
r6:  p  p  p  p  F  p  p  p  F  p  p  p  p
r7:   m  p  p  p  p  p  T  p  p  p  p  p  m
```

Key coordinates. Home factories: West **(1,3)**, East **(11,4)** — 10 hexes apart, unchanged from the 13 × 9 draft, so §2.13.3's contact-turn arithmetic is untouched. Neutral factories **(4,1)(8,1)(4,6)(8,6)** · Towns **(6,0)(3,3)(9,4)(6,7)** · central Woods knot **(5,3)(6,3)(6,4)(7,4)** · corner Mountains **(0,0)(12,0)(0,7)(12,7)**.

Every one of those is a ρ-pair under ρ(c, r) = (12−c, 7−r), and the pairing is the map's whole warrant: (1,3)↔□(11,4) homes, (4,1)↔□(8,6) and (8,1)↔□(4,6) neutrals, (3,3)↔□(9,4) and (6,0)↔□(6,7) towns, (5,3)↔□(7,4) and (6,3)↔□(6,4) woods, (0,0)↔□(12,7) and (12,0)↔□(0,7) mountains. Rows 2 and 5 are all Plains and map to each other. Note that ρ has no fixed hex on an even-row board, so the homes sit on *different rows* (3 and 4) — that is the rotation working, not a drafting slip.

Terrain distribution (104 hexes): Plains 86 · Woods 4 · Mountains 4 · Water 0 · Bridge 0 · Town 4 · Factory 6. Even more open than the shipped map (83% Plains vs. 76%), which is the point: this is the maneuver map, and the only terrain that slows anyone is the four-hex Woods knot standing between the two home rows.

Starting positions (all on Plains; home factory hex left free; East is the exact ρ-image of West, which is what "one variable at a time" means here):

Unit	West	East
Flag Tank	(0,3)	(12,4)
Infantry ×2	**(1,2)**, (1,4)	**(11,5)**, (11,3)
Artillery	(0,2)	(12,5)
Recon	(0,4)	(12,3)

Guided opening (§2.13.1): `guidedOpening.infantry` = **(1,2)** West / **(11,5)** East; `guidedOpening.objective` = **(4,1)** West / **(8,6)** East — distinct objectives, 4 MP each, no Bridge on the map at all. 2 MP of slack against the 6 MP ceiling, identical in both seats.

No Water at all — the map that teaches what the chokepoint map can't: open-field maneuver, Recon (move 7) flanking wide, and expansion tempo deciding who holds 4-of-6 **factories** at the cap. Criterion 2 counts factories *and* captured towns (§2.8), so this map's denominator is **N = 10** — 6 factories + 4 towns, against N = 8 on *Ferrum Crossing* (§2.11.4). The factory half of that spread is what the extra neutrals buy: a **0–6 factory swing inside a 10-objective sort**, where the shipped map can swing only 0–4.

Match-length reasoning: 4 neutral factories scale income toward 600+ Fame/turn (§2.7) — losses are replaced almost as fast as they land (a Tank kill pays +150 while the victim's

economy rebuilds the Tank in under a turn at full income). Attrition drags, the flag hides behind rebuilt lines, and the match leans on the §2.8 tiebreak. That is the point: this is the scenario that *exercises* the combat-Fame tiebreak and the §2.11 scoreboard, which on the shipped map rarely fire.

Tactical question: *how many factories can you hold with the army those factories pay for?* Over-expansion strands Infantry on capture duty while the center Woods fight is lost; under-expansion loses the income race and, at the cap, the objectives-held sort.

2.13.6 Stretch scenario — *The Causeway* (P2, wk 4)

Spec	Value
Dimensions	**9 × 8 = 72 hexes** — eight rows, not nine, because a 180° rotation only closes on an even row count (§2.13.1)
Starting units per side	5 (standard force)
Factories	4 — one home per side + 2 neutral (conforms to §2.7 ~4)
Towns	2
Symmetry	**180° rotational**, ρ(c, r) = (8−c, 7−r) — fair, and rotation puts each seat's near bridge on the opposite flank (West's is north at (4,2), East's is south at (4,5)), so seat-swap stays non-cosmetic even on a symmetric map. Rotation is not a preference here but the only symmetry available to an odd-r rectangle with an even row count — the vertical mirror exists at no dimension, the horizontal one only on odd H (§2.13.1); the row count is even so that the rotation actually closes.
Estimated match length	**8–12 turns**

```
r0:  p  p  p  p  ~  p  p  p  p
r1:   p  T  p  w  ~  p  p  p  p
r2:  p  p  m  F  B  p  p  p  p
r3:   p  F  p  w  ~  p  p  p  p
r4:  p  p  p  p  ~  w  p  F  p
r5:   p  p  p  p  B  F  m  p  p
r6:  p  p  p  p  ~  w  p  T  p
r7:   p  p  p  p  ~  p  p  p  p
```

Water fills column 4 end to end except Bridges **(4,2)** and **(4,5)** — **a full bisection, the deliberate opposite of Ferrum Crossing.** Column 4 is its own ρ-image, which is why the bisection survives the rotation intact. Homes **(1,3)**/**(7,4)** — 6 hexes apart, unchanged. Neutral factories **(3,2)** and **(5,5)** each guard the approach to their adjacent bridge from their own seat's bank: +15% defense *and* a spawn point at the chokepoint (§2.7 build-and-spawn makes a held bridge-factory a reinforcement faucet exactly where reinforcements matter). Woods **(3,1)(3,3)** flank the north bridge on West's bank and **(5,4)(5,6)** flank the south bridge on East's; each seat's own crossing is the one it defends from cover, and each seat's *far* landing — (5,2) north, (3,5) south — is bare Plains, so attacking a bridge is always the uncovered half of the trade. Single Mountains **(2,2)**/**(6,5)** are Artillery perches — range 2–3 covers their bridge hex from +40% cover with no counter (§2.3, §2.4), and reaches the far landing at range 3.

ρ-pairs, exhaustively: (1,1)↔□(7,6) towns, (3,1)↔□(5,6) and (3,3)↔□(5,4) woods, (2,2)↔□(6,5) mountains, (3,2)↔□(5,5) neutral factories, (1,3)↔□(7,4) homes, (4,2)↔□(4,5) bridges, and the six Water hexes in pairs (4,0)↔□(4,7), (4,1)↔□(4,6), (4,3)↔□(4,4). Everything else is Plains.

Terrain distribution (72 hexes): Plains 52 · Woods 4 · Mountains 2 · Water 6 · Bridge 2 · Town 2 · Factory 4. The tightest board in the set, and the only one where a single terrain type — Water — decides the topology.

Starting positions (all on Plains; home factory hex left free; East is the exact ρ-image of West). *This map previously specified none, which left §2.13.1's lane figures without an antecedent and made T-SCN-06/07/08 unevaluable against it:*

Unit	West	East
Flag Tank	(0,3)	(8,4)
Infantry ×2	**(1,2)**, (1,4)	**(7,5)**, (7,3)
Artillery	(0,2)	(8,5)
Recon	(0,4)	(8,3)

Guided opening (§2.13.1): `guidedOpening.infantry` = **(1,2)** West / **(7,5)** East; `guidedOpening.objective` = **(3,2)** West / **(5,5)** East. Distinct objectives (T-SCN-07); 3 MP each, Bridge-free and confined to the seat's own bank (T-SCN-06); 3 MP of slack. The West lane is (1,2)→(1,1)→(2,1)→(3,2), through the town rather than over the Mountain at (2,2) — the 2-hex route costs 4 MP and the 3-hex route costs 3, which is the case T-SCN-08's "computes, never infers" wording exists for.

Note what the deployment does *not* do: neither seat starts within reach of a bridge. West's forward Infantry at (1,2) is 3 hexes from (4,2); the guided opening walks it away from the crossing, to a factory on its own bank. The first lesson on the bisection map is still capture, not crossing — the map earns its lockout premise from turn 3 onward, not turn 1.

Both bridges held — stated in full, because on this map it is the whole design. No naval unit, no other crossing: double-bridge control is a true lockout — the locked-out side cannot reach the enemy flag *or* home factory, so both the flag kill and territorial domination are out of its reach. The rules already make this a trap, and the map exists to prove it: the holder must still *cross* to win decisively; if it sits, it earns zero combat Fame and **loses the cap tiebreak to any opponent who dealt any damage at all** (§2.8 criterion 1); if both sides sit, the mutual-passivity guard calls a draw. Lockout on The Causeway is a tempo platform for a prepared crossing, never a victory condition — the map that demonstrates §1.5 finding #1 (the turtle exploit) is dead.

Match-length reasoning: homes only 6 columns apart, but every route crosses a Bridge; the map forces commitment, and the first successful bridgehead (turn 3–5) usually cascades into the flag kill within 4–6 turns because the defender's Fame went into defending crossings, not expanding.

Tactical question: *how do you buy a bridgehead when the crossing hex fights at −10% and the far bank fights at +20?* The §2.6 forecast makes the price exact before you pay it — the scenario where the forecast display earns its keep.

2.13.7 Scenario-set summary

Map	Status	Hexes	Units/side	Factories	Towns	Symmetry	Dial it turns
Ferrum Crossing	**Ships** (§2.10 IN)	11×9	5	4	4	Asymmetric (Fame-correctable)	baseline / seat asymmetı
Longwater March	Stretch P1 (wk 4)	13×8	5	6	4	180° rotational	factory count → cap pressure
The Causeway	Stretch P2 (wk 4)	9×8	5	4	2	180° rotational	bridge lockout – decisiven

The stretch condition — stated here, once. Neither stretch map may pull work forward of week 4 or block core; *The Causeway* is attempted only after *Longwater March* lands; and if week 4 is consumed by balance (its primary §4.4 purpose), the set stays on paper. Those four clauses are the whole test for whether either map gets built, and this is the only section that states it. Everywhere else the set is named — §2.13.4's configuration ladder, §2.13.5 and §2.13.6's headers, §2.10's scope table STRETCH row — carries the *labels* (P1/P2, week 4) as identification and defers here for the *condition*, so tightening or relaxing the scenario set is one edit in one place. Two sections stating one condition in their own words is exactly the drift §4.4 and §4.11 produced three times; the repair there was one owner and one pointer, and for the scenario set the owner is this paragraph.

---

3. AI Architecture — how AI agents are used (roles)

The project is run as a small **agent team directed by a human**. Each role has a clear responsibility, an instrument, and an output that is verifiable.

Role	Owner	Responsibility	Instrument	Verifiable output
Director	Human	Specs, scope line, final balance judgment, review gates, art direction, pipeline writeup	—	This GDD; merge approvals
Systems Engineer	Agent	Author the headless C++ rules (grid, movement, combat, capture, turn loop, win) from spec	Claude Code (headless module, no editor)	Compiling C++ modules
Test Engineer	Agent	Write + run the automation/unit-test suite; block merges on failures	Claude Code + test harness	Passing test suite (the merge gate)
Balance Analyst	Agent	Run headless AI-vs-AI self-play; report win-rate and turn-length; propose stat tuning	Self-play sim harness	Balance logs + tuning diffs
Content / Scenario Designer	Agent	Build maps/scenarios; generate + populate unit/terrain DataTables	Custom **MCP toolset** in-editor	Scenario assets, data tables
UI Scaffolder	Agent	Generate UMG widget skeletons + bindings for human polish	Claude Code + UE MCP	Wired UMG widgets
Opponent Commander *(stretch, runtime)*	Agent (LLM)	Play a turn in-product as the enemy	In-game LLM call + move validator	A validated legal move per turn
Documentation crew *(4 authors)*	Agent ×4	Draft assigned GDD sections in parallel against a frozen snapshot; write only their own file, never the master (§1.6)	Claude Code sub-agents over a synced read-only snapshot	Section drafts, one file per author
Continuity gate	Agent	Audit every draft against the live GDD for contradictions, stat drift, dead references and invented numbers; block merge until PASS (§1.6)	Claude Code	Per-section verdicts and violation counts in the gate's accept record

The last two rows are **document-side** rather than game-side: they authored and gated this GDD, not the build, and they are the crew §1.6 describes — including its recorded failure, where the gate filed four violations, the two authors responsible were re-spawned with them, and only the corrected drafts merged. They belong in this table because its contract is an I/O contract and those two were the only roles that had run without one. §1.5's **agent review crew** (Exploit-Hunter / Consistency / Pacing) gets this pointer rather than a row, for the opposite reason: its findings were human-adjudicated into §1.5's change table, so it has no machine-checkable artifact for the last column, and a role with no verifiable output does not belong in the one table whose purpose is verifiable outputs.

Pipeline. Claude Code is the agent client. Two surfaces: - **Headless harness** (Systems /

Test / Balance roles) — the rules module has **zero engine dependencies**, so these agents compile, test, and self-play in seconds without launching the editor. This is what makes agent authorship fast enough to dominate the build. - **Unreal MCP plugin** (Content / UI roles) — for in-editor operations (scenario building, UMG scaffolding).

Workflow — test-first. Director writes a spec → Test Engineer writes tests against it → Systems Engineer implements until tests pass → Balance Analyst self-plays and proposes tuning → Director reviews and approves. The agent verifies its own work against a spec, not against the Director's eyeballs.

Worked example — how a role is prompted and constrained. The role table names *what* each agent produces; the constraint is *how*. Every headless system starts as a structured **input spec** the Director hands the Systems Engineer — not a prose ask, but a contract of inputs, an exact formula, and machine-checkable invariants. For combat resolution:

```
SPEC: Combat resolution           (Director → Systems Engineer)
Inputs:  attacker{atk, type, hp, hpMax}, defender{def, type, hp},
         terrainDef% = defender's hex, attackerRange vs distance,
         eff = typeEffectiveness[attacker.type][defender.type]  (default
1.0 everywhere)
Formula: dmg = max(1, round(atk * eff * (hp/hpMax) * (1 - terrainDef%)) -
def)
Invariants (Test Engineer asserts each one):
  - terrain defense applies to the DEFENDER's hex only — never the
attacker's
  - counterattack fires ONLY if the defender survives AND the attacker is
    within the defender's range
  - Artillery (range 2–3) takes zero counter from a range-1 attacker
  - same (attacker, defender, terrain, hp) → identical result
(determinism; any RNG seeded)
Determinism: pure function of inputs, no unseeded RNG
TypeEff:     eff ∈ {0.5, 1.0, 1.5}; ships as an all-1.0 table, so RPS
stays
             positional (§2.4). Populate only if self-play shows the
triangle
             too weak. eff=1.0 everywhere → existing T-COMBAT-01..08
unaffected.
Acceptance: T-COMBAT-01..NN must pass before merge
```

The Systems Engineer is constrained to that spec; the Test Engineer turns each invariant into a gate *before* implementation exists. This is where the pipeline earns its keep — the gate catches rules the agent hallucinates. A real case:

```
T-COMBAT-07: Artillery counter-immunity
  Given  Infantry attacks Artillery from an adjacent hex (distance 1)
  Then   Artillery deals 0 counter damage        (its range is 2–3, so it
can't strike back at 1)
  First agent pass → FAIL: the Systems Engineer generalized "surviving
units counterattack"
  and wired a range-1 counter for Artillery. Merge blocked → the invariant
is re-fed →
  re-implemented → T-COMBAT-07 green → merged.
```

No human eyeballed the diff to catch that; the spec's invariant did, mechanically, at the merge gate. That is the difference between "an agent wrote some C++" and a constrained, self-verifying pipeline.

A second recorded case, at `c224825`. Movement did the same thing, and the block was wider:

```
T-MOVE-01/02/03: reachable-set exactness
  Pass 1  reachable set computed as hexDistance <= move — terrain never
          consulted. A plausible reading of "reachable"; not the §2.3
rule.
          Merge blocked on T-MOVE-01, T-MOVE-02 and T-MOVE-03 at once.
  Pass 2  Dijkstra over terrain cost, ties broken by canonical hex order →
          T-MOVE-01..06 green (6/6) → merged.
```

The load-bearing detail is what T-MOVE-01 compares against: an **independent** shortest-path pass written inside the test, not the module's own search. An invariant that re-runs the implementation agrees with it by construction and asserts nothing; this one could not, which is why a pass that was wrong *consistently* still failed it. Same shape as T-COMBAT-07, one layer deeper — there the spec's invariant caught a generalisation, here the test's own oracle caught a simplification.

This crew is the buildable deliverable. The three headless roles above — **Systems Engineer, Test Engineer, Balance Analyst** — are exactly the 3+ coordinating agents instantiated for the standalone agent-crew build: their *game-ready output* is compiling C++ systems, a passing test suite, and self-play balance data for *this* game, handed between them in a fixed order (spec → tests → implementation → balance). The role table's I/O columns are the contract that crew is written against, so the document and the crew stay

in lockstep.

Guardrails (non-negotiable). - Perforce checkpoint before any agent-driven edit session; agent permissions treated like production credentials. - The MCP plugin is **experimental, partly undocumented, and runs serially on the game thread** — it is **not on the critical path**; every editor op it does has a manual fallback. - Enable the **AllToolsets** companion plugin, or the MCP server connects but exposes nothing useful. - Claude Code is the documented client for editor-driving (not Claude Desktop).

Provenance ledger — how agent contribution is reported. The deliverable is the game (§1); agent authorship is reported honestly, not as a single hero percentage but as a **per-system ledger** — one row per system, marked by author and verification status, **each Verified row citing the commit and passing test IDs that back it** so the claim is auditable rather than asserted.

Status: live tracker — first rows populated 2026-07-26 from the Assignment-3 agent crew; ***Repair*** *and* ***Type-effectiveness*** *added and gate-verified 2026-07-29; the rest land as each system is built (wk 1–3, §4.4). This draft stands at 2026-08-06, at commit* `c2edae0` *in the crew repo and at* `fed8ae9` *in the Stratocracy UE project repo. In the crew repo,* `ec15be6`*,* `737f666`*,* `d837fc8`*,* `30a73f0`*,* `188f966`*,* `e19605e` *and* `1ee890e` *are each an ancestor of* `031ee20`*, measured with* `git merge-base --is-ancestor` *per sha rather than read off a branch. In the UE project repo,* `9dec48c` *is an ancestor of* `a13626f`*, measured the same way.* `c2edae0` *and* `fed8ae9` *are cited as commits, and this line makes no claim about how either stands to any branch. Measured at* `ec15be6`*, every crew-repo commit this GDD cited then is an ancestor of* `ec15be6`*, measured the same way per sha.* `9dec48c` *is cited as a commit, and this line makes no claim about how it stands to any branch. This line previously said that both commits had been verified with* `git ls-remote` *to be the head of their own repo's default branch. That was true when it was gated and false within the hour, a concurrent human push having moved one of them — a head expires and a sha does not — so the defect is recorded here rather than quietly overwritten. The landing this ledger records before them is* `b23823f` *in the crew repo, whose parent is* `e06c44b` *and whose grandparent is* `41a1452`*, with the UE project's half at* `99fcb84`*, the first commit in that repo since its Git LFS migration. §4.4's week-1 deliverable is §4.11 rows 1–3 — grid and hex math, the §4.8 tables, movement and pathfinding — and* ***week 1 closed two of the three****: rows 1 and 3 passed their full acceptance sets at that commit (T-HEX-01..07, 7/7; T-MOVE-01..06, 6/6) and flip in the table below.* ***Row 2 does not flip.*** *Its headless half is green — T-DATA-01..04 and 06, 5/5 — but T-DATA-05, the in-editor Unreal Automation parity pass, has not run, and Q29 requires the full acceptance set at one commit, so the row records a partial pass and stays unverified; the editor pass is not yet due, so that is the ordinary schedule and not §4.7's cut line firing. What week 1 did* ***not*** *close is everything after it: rows 4–8 held no code then, and* ***rows 4, 5 and 6 have since landed*** *— all three recorded at the end of this paragraph — so at* `d8284f1` *only rows 7–8 hold none; since §4.11's critical path runs 1 → 3 → 4 → 5 → 6/8, every row on that path has since landed,* ***row 8*** *last and on a partial pass that leaves its ledger row unflipped —* ***per acceptance ID as well as per row****:* `T-TURN-10` *closed at* `6ccd40b`*, and the path IDs still written and asserting without being green are* ***T-UI-03*** *and* ***T-UI-04****,* `T-UI-05` *having since closed at* `41a1452` *as recorded at the end of this paragraph, and row 8's other dependency is* ***row 7****, which has since landed at* `9086d6a` *on a partial pass, recorded at the end of this paragraph. §4.5's* Specification outruns the build* risk is therefore reduced and re-scoped rather than retired, and that row now states the arithmetic. **What landed after week 1 is a debug driver**, at `9f87ecd`: `cpp_reference/driver_main.cpp` builds a command REPL, `build/stratocracy_debug`, over the built modules. The binary is **not citable** — `build/` is untracked, as the paragraph below the table already records — so what is cited is its five tracked sources, each probed present at that commit: `spec/driver_spec.md`, `cpp_reference/Driver.h`, `cpp_reference/Driver.good.cpp`, `cpp_reference/driver_main.cpp`, `cpp_reference/test_driver.cpp`. **Seven** tracked sources defined `main()` at that commit — `cpp_reference/test_combat.cpp`, `cpp_reference/test_hex.cpp`, `cpp_reference/test_data.cpp`, `cpp_reference/test_move.cpp`, `cpp_reference/test_driver.cpp`, `cpp_reference/selfplay.cpp`, `cpp_reference/driver_main.cpp` — five test harnesses, one combat duel simulator, and the REPL; **row 4 has since added an eighth**, counted at the end of this paragraph. **The driver holds no rules of its own.** Reach, path and move delegate to `cpp_reference/Move.h`, damage and counter eligibility to `cpp_reference/Combat.h`, distance and adjacency to `cpp_reference/Hex.h`, and every stat to `cpp_reference/Data.h` over `data/units.csv`, `data/terrain.csv` and `data/effectiveness.csv` — the row 2 module, whose ledger row is still unflipped; `forecast` and `attack` call one computation, so §2.6's *the forecast the player sees is exactly what resolves* holds structurally at this surface. Where an answer would need §4.11 rows 4–8 — who owns a unit, whose turn it is, what a scenario file looks like — the driver at that commit **refused the command rather than deciding it**, so the build wrote no rule this document has not. Its gate is **GATE-DRV-01..07, 7/7 under clang++ and MSVC both**, and the checks that compare a value compute their expectation by calling the module directly rather than hardcoding it; those IDs are deliberately **not** `T-*`, because the driver is not a §4.7 stub and has no row in the ledger below, so §4.5's written-ID count does not move at this landing and **no ledger row flips on the driver's account**. A human can drive

units from that commit on: `move`, `attack`, `forecast` and `repair` are four of the sixteen commands `cpp_reference/Driver.good.cpp` dispatches at that commit. What there was no way to do at that commit is play a match: **no turn structure, no capture, no Fame, no production, no AI and no scenario file**, and the driver exposed none of it. On that record the Director **ruled, 2026-08-02, that §4.4's week-1 goal "Playable via debug commands" is met at `9f87ecd`, in its current state** — a ruling on a judgement rather than a result any check produced: the artifact exists and the match does not. **The Director amended that ruling the same day, qualifying it rather than retracting it:** the current state is acceptable **at this current time** and is **not a final call**, because "Playable via debug commands" should eventually include **all game features as they come online**. The goal is therefore **provisionally met** — met against the feature set that existed at the commit it was ruled on, **re-opened by each system that lands after it**, rows 5–8 included, and **re-opened equally by a rebuild of an already-landed row that changes what a human can reach at the REPL** (ruled 2026-08-04: a rebuild is not a landing, so the rule as first written did not reach one). On that second account the goal was **met again at `6ccd40b`**, where the move/act split into two independent per-unit flags made §2.1's *move and act, in either order* reachable at the REPL for the first time; before it the two commands spent one shared flag, so that sequence was unimplementable there. The bare word *met* is a current-state acceptance and not a permanent closure. **Row 4 then landed**, at `647d4df`: **Capture & Fame economy**, gated **T-FAME-01..09, 9/9 under clang++ and MSVC both**, cited by five tracked sources each probed present at that commit — `spec/economy_spec.md`, `cpp_reference/Economy.h`, `cpp_reference/Economy.good.cpp`, `cpp_reference/Economy.buggy.cpp`, `cpp_reference/test_economy.cpp`. The last of those is a sixth harness, so **eight** tracked sources define `main()` at `647d4df` — the seven listed above plus `cpp_reference/test_economy.cpp` — six test harnesses, one combat duel simulator, and one debug REPL. **No ID is left uncovered:** unlike row 2 there is no in-editor half, so the full acceptance set closes at one commit, Q29 is satisfied, and the row flips in the table below. **No new acceptance ID was written**, so §4.5's written-ID count does not move at this landing and its green count moves 18 → 27. §4.4 schedules rows 4–5 for **week 3**, so this row is **ahead of the milestone table, not behind it** — the opposite of the debt recorded above, and the two are different facts. Row 4 owns the economy and not the turn: it never advances a turn and never decides whose turn it is, taking the turn number as an argument, which is why it could land before row 5. **Four of its nine invariants execute a ruled question** rather than a stated reading — Q8 (no accrual on turn 1; Fame committed at queue time), Q4 (capture progress is tile-held, resets, and never transfers), Q5 (the flag award replaces rather than stacks, so 500 and not 650) and Q6 (no undamaged-strike bonus, asserted by absence) — so those four rulings are executed rather than only written. The driver reaches it: `objectives`, `fame`, `turn <n>`, `income <side>`, `build <side> <Type> <col> <row>` and `capture <side>` are new commands, plus a kill award paid through `attack`, and each is a call into `cpp_reference/Economy.h`, so the driver still holds no rules of its own; `turn <n>` is a **debug setter, not a turn structure**, and `GATE-DRV-01..07` is still 7/7 and still not `T-*`. What there is no way to do at `647d4df` is still play a match: there is **no turn structure, no AI and no scenario file**, and a setter for the turn number is not the first of those. **Row 5 then landed**, at `ad77b13`: **Turn loop & win/tiebreak**, gated, **as the acceptance set then read**, **T-TURN-01..09, 9/9 under clang++ and MSVC both** — `g++` is not installed on this machine — while its pass-1 implementation `cpp_reference/Turn.buggy.cpp` is blocked at 6/9, on T-TURN-05, T-TURN-06 and T-TURN-07, under both compilers. That set has since widened to `T-TURN-01..10` and row 5 was rebuilt against it; **this sentence records what ran at `ad77b13` and is not row 5's current evidence**, which is recorded at the end of this paragraph. Five tracked sources are cited, each probed present at that commit: `spec/turn_spec.md`, `cpp_reference/Turn.h`, `cpp_reference/Turn.good.cpp`, `cpp_reference/Turn.buggy.cpp`, `cpp_reference/test_turn.cpp`. The last of those is a seventh harness, so **nine** tracked sources define `main()` at `ad77b13` — the eight named above plus `cpp_reference/test_turn.cpp` — seven test harnesses, one combat duel simulator, and one debug REPL. **No ID is left uncovered:** row 5 has no in-editor half and no reserved-unwritten ID, so its full acceptance set closes at one commit — but that commit is no longer this one. The set was widened to `T-TURN-01..10` this revision, and it closes at the `6ccd40b` rebuild recorded at the end of this paragraph; Q29, read per acceptance ID, is satisfied there, and the row flips in the table below on that commit rather than on this one. **No new acceptance ID was written**, so §4.5's written-ID count does not move at this landing and its green count moves 27 → 36. §4.4 schedules rows 4–5 for **week 3**, so this row too is **ahead of the milestone table, not behind it**. Rows 3 and 4 declined the turn — row 4 takes the turn number as an argument — so row 5 is the first module to own alternation, per-unit act flags, the start-of-turn moment and the §2.8 result; it owns **no board**, since the §2.8 facts arrive as a caller-supplied `BoardSnapshot`, the same discipline row 4 uses. **T-TURN-04, 05, 06 and 07 encode §2.8's procedure exactly** — one guard, one three-key comparison, one grade — and **Q7 is executed rather than only written**: the cap is per-scenario data and the module refuses a cap below 1 rather than substituting a default, so no literal 20 exists in it. T-TURN-08 asserts only that the loop calls the already-verified `repairAmount` at the right moment with the right board facts, every expectation in it being a direct `cpp_reference/Combat.h` call; the heal values stay green at `5ffa8d6` under T-REPAIR-01..07 and are not re-asserted. Two readings are

recorded in `spec/turn_spec.md` as **documented choices, not rules**: a turn is one full I-GO-U-GO round shared by both sides, read off two sites that keep the turn number and the side to move as separate fields — §4.7 Stub 8's UI snapshot `match {turn, turnCap, sideToMove, resultTier or null}`, and §4.10's canonical state hash, which serializes `GameState` in a fixed field order beginning turn counter, side to move — and consistent with §2.7's *both players draw income from turn 2*; and the cap resolves at the **end** of round `turnCap`, since §2.11.4 displays the counter as `N / turnCap`, so turn `turnCap` is a playable turn. Neither is a new rule and neither is filed as a change request. The driver reaches the module: `match <firstSide> <turnCap>`, `endturn`, `standings`, `result` and `flag <side> <id>` are new commands, joined by act-flag and alternation enforcement on `move`/`attack` and active-side gating on `income`/`build`/`capture`, and each is a call into `cpp_reference/Turn.h`, so the driver still holds no rules of its own. `flag` is a **debug designation** standing in for Stub 7's `isFlag` — at that commit row 7 held no code, and Q10 is open on exactness — the human names the flag unit and the driver never picks one, and it is what makes the flag kill award reachable, paid as the flat 500 through `cpp_reference/Economy.h::killAward`. `turn <n>` remains a debug setter and is now **refused while a match runs**; with no match running the board is the same free sandbox it was at `647d4df`, which is why `GATE-DRV-01..07` are unchanged and still pass. The driver suite is now **GATE-DRV-01..09, 9/9 under clang++ and MSVC both**, and those IDs are still **not** `T-*`: the driver is not a §4.7 stub, has no row in the ledger below, and flips nothing. A **Claude Code session** authored row 5 from the Director-written stub `spec/turn_spec.md`, **not a live CrewAI run**. What `ad77b13` still does not have is **an AI or a scenario file** (rows 6–8): the driver's boards are built-in fixtures plus `place`/`remove`, and no file format is defined or read. **Row 6 then landed**, at `d8284f1`: **Opponent AI**, gated **T-AI-01..06 plus GATE-AI-SMOKE, 7/7 under clang++ and MSVC both** — `g++` is not installed on this machine — while its pass-1 implementation `cpp_reference/Ai.buggy.cpp` is blocked at 5/7, on T-AI-05 and T-AI-06, under both compilers. Five tracked sources are cited, each probed present at that commit: `spec/ai_spec.md`, `cpp_reference/Ai.h`, `cpp_reference/Ai.good.cpp`, `cpp_reference/Ai.buggy.cpp`, `cpp_reference/test_ai.cpp`. The last of those is an eighth harness, so **ten** tracked sources define `main()` at `d8284f1` — the nine named above plus `cpp_reference/test_ai.cpp` — eight test harnesses, one combat duel simulator, and one debug REPL. **No ID is left uncovered:** row 6 has no in-editor half and no reserved-unwritten ID, so its full acceptance set closes at one commit, Q29 is satisfied, and the row flips in the table below. **GATE-AI-SMOKE mints no acceptance ID.** The self-play smoke run is acceptance and §4.11 row 6 names it, but it carries no numbered ID in this document and `spec/ai_spec.md` declines to mint one, since a `T-AI-07` would move §4.5's count. The gate is therefore 7/7 while **six** IDs close: §4.5's written-ID count does not move at this landing and its green count moves 36 → 42. §4.4 schedules row 6 for **week 3**, so this row too is **ahead of the milestone table, not behind it**. This is **the shipping opponent** (§2.9) and not a stand-in: difficulty is a starting-Fame handicap and never a smarter routine, and nothing in the module reads a difficulty tier at all. **It decides and applies nothing** — it emits one ordinary command at a time and the caller applies it, in the gate through the debug driver's own `execute`, the same door a typed command uses, which is what makes T-AI-01's *validated like any player command* structural rather than asserted; T-AI-01's counter printed `129 AI commands issued across 6 games` **at that commit**. That figure is **commit-scoped and is not an invariant** — it counts what the routine emitted against the rules as they then stood, so an upstream rule change moves it without moving anything else in this row: re-run at `6ccd40b`, after row 5's rebuild, the same counter prints `120 AI commands issued across 6 games`, while row 6's commit, its 7/7 tally and its ledger row are unchanged. It holds no rules — routes to `cpp_reference/Move.h`, damage and counters to `cpp_reference/Combat.h`, stats to `cpp_reference/Data.h`, affordability and kill value to `cpp_reference/Economy.h`, act flags and alternation to `cpp_reference/Turn.h` — and it owns **no board**, reading a caller-composed `AiState` on which `spec/ai_spec.md` records **no field a player could not read off the screen**, which is where §4.7 Stub 6's *the AI cheats at nothing* is carried. **Q9 is executed rather than only written, and the gate caught pass 1 breaking it:** T-AI-06 fixes position and target ties to canonical hex order — for a target, the hex it occupies — and build ties to Infantry > Recon > Artillery > Tank, ascending §2.4 cost and **not** the order §2.4's table prints, which is the order pass 1 used. **T-AI-05 is a sweep rather than a fixture:** of **348** exchanges in the shipped stat table where the counter kills the attacker, the good build skips **338** and permits **10**, each permitted one checked not to trade down, while the pass-1 build permits **0** — its guard had collapsed to *do not attack if the counter kills you*, dropping §2.9's *and trades down* half, which is the failure the sweep exists to catch. Five readings are recorded in `spec/ai_spec.md` as **documented choices, not rules**: the buildlist is caller-supplied data, since §2.9 names a composition and no ratio; *undefended* (T-AI-03) excludes an enemy adjacent to the objective as well as one standing on it, since `cpp_reference/Move.h` already refuses an occupied hex; *near* (T-AI-03) is reachable this turn and cheapest to reach among those, ties by canonical hex order; *trades down* (T-AI-05) prices value dealt as the victim's kill award prorated by the damage share of its max HP and value lost as the attacker's own kill award unprorated, both read through `cpp_reference/Economy.h::killAward`; and with no flag designated the advance goal is the canonically first enemy unit, `isFlag` being Stub 7's placement field, which held no code at that commit, and Q10 being open on exactness. The driver reaches the module: `ai` plays the active side's turn and `ai buildlist` sets §2.9's list, and the start of a turn now

runs repair, income and capture tick in that order. GATE-DRV-01..07 are unchanged and still pass; the driver suite is now **GATE-DRV-01..10, 10/10 under clang++ and MSVC both**, GATE-DRV-10 being new at this commit — it replays the AI's printed command lines by hand and asserts an identical state hash — and those IDs are still **not** T-*: the driver is not a §4.7 stub, has no row in the ledger below, and flips nothing. A **Claude Code session** authored row 6 from the Director-written stub spec/ai_spec.md, **not a live CrewAI run**. No scenario harness and no UI harness is among the ten main() definitions above. **Row 7 then landed**, at 9086d6a: **Scenario file & validator**, gated **12/12 under clang++ and MSVC both** — g++ is not installed on this machine — while its pass-1 implementation cpp_reference/Scenario.buggy.cpp is blocked at 11/12, on T-SCN-11 alone, under both compilers: it minimised the opposing route over the opposing seat's guidedOpening.infantry alone rather than over every CanCapture-row unit that seat deploys, which is reading (b) at Q28 and the one that ruling refused. Six tracked sources are cited, each probed present at that commit: spec/scenario_spec.md, cpp_reference/Scenario.h, cpp_reference/Scenario.good.cpp, cpp_reference/Scenario.buggy.cpp, cpp_reference/test_scenario.cpp, data/ferrum_crossing.json. The fifth of those is a ninth harness, so **eleven** tracked sources define main() at 9086d6a — the ten named above plus cpp_reference/test_scenario.cpp — nine test harnesses, one combat duel simulator, and one debug REPL. **This row does not flip.** Q29 requires the full acceptance set at one commit, and it is applied **per acceptance ID as well as per row** — an ID closes only when its whole written fixture set has run. Of the twelve checks that passed, **T-SCN-01..07 ran in full and close**; GATE-SCN-PARSE and GATE-SCN-HASH gate a file format rather than a §4.7 stub and mint no numbered acceptance ID, on the GATE-AI-SMOKE precedent; and **T-SCN-08, T-SCN-09 and T-SCN-11 ran a part of their fixture sets and do not close** — T-SCN-08 on fixture (c) plus its measure-and-report behaviour on the shipped map, T-SCN-09 on its refusal branch, T-SCN-11 on fixtures (a) and (b), (b) being a required failure. So the row records a partial pass and stays unverified, which is the posture row 2 holds on T-DATA-05. **No new acceptance ID was written**, so §4.5's written-ID count does not move at this landing and its green count moves 42 → 49. **Four fixtures did not run, and each is named rather than absorbed** — each printed by name with its reason before the tally and recorded in the acceptance record: T-SCN-08 (a) *The Causeway* and (b) *Longwater March*; T-SCN-09's asserting branch; T-SCN-11 (c) *The Causeway*. Each needs a stretch map authored as a scenario file, and **none was replaced by a synthetic map**. Those three IDs are **written, unblocked and asserting**; what they lack is a map to run against, not a rule. **T-SCN-10 is a different state and is not one of them:** it is reserved and **unwritten** on Q26, so no invariant text exists for it, nothing asserts and nothing waits. **The shipped scenario file is a transcription, not an authoring**: all 99 terrain hexes of data/ferrum_crossing.json were diffed against §2.13.2's grid, zero mismatches, and the distribution is the one §2.13.2 states. **A change request out of the build is registered rather than acted on**: §2.13.1's validation-invariants bullet names three checks no T-SCN- ID asserts as written, and the build implemented T-SCN-02 and T-SCN-04 to their exact written text rather than widening them — registered as **Q32**, with no invariant text changed and no acceptance ID minted. The debug driver's suite is now **GATE-DRV-01..11, 11/11 under clang++ and MSVC both**, GATE-DRV-11 being new at this commit, and those IDs are still **not** T-*: the driver is not a §4.7 stub, has no row in the ledger below, and flips nothing. A **Claude Code session** authored row 7 from the Director-written stub spec/scenario_spec.md, **not a live CrewAI run**. **Row 7 is not on §4.11's critical path**: it was built because row 8 queues behind it and its own dependencies had landed. No UI harness is among the eleven main() definitions above. **Row 5 was then rebuilt**, at 6ccd40b, whose parent is 9086d6a, against three rulings whose **GDD half had already merged before any code satisfied them**: T-TURN-01's move/act split into two independent per-unit flags, the new T-TURN-10 build allowance, and T-FAME-04's dropped per-turn clause. This commit **modifies only** — git show --summary prints no create, delete or rename line, 14 files changed: README.md, crew/tools.py, spec/ai_spec.md, spec/driver_spec.md, spec/turn_spec.md, and in cpp_reference/, Ai.buggy.cpp, Ai.good.cpp, Driver.good.cpp, Driver.h, Turn.buggy.cpp, Turn.good.cpp, Turn.h, test_driver.cpp and test_turn.cpp. **No harness was added:** the same **eleven** tracked sources define main() as at 9086d6a. The gate is **T-TURN-01..10, 11/11 under clang++ and MSVC both** — g++ is still not installed on this machine — and **the 11 counts printed checks, not IDs**: ten IDs close, and T-TURN-01 prints two of the eleven lines, alternation-and-once-per-own-turn and two-independent-flags-in-either-order. Pass-1 cpp_reference/Turn.buggy.cpp is blocked at **5/11** — six FAIL lines over **five** distinct IDs: T-TURN-01 (both of its lines), T-TURN-05, T-TURN-06, T-TURN-07 and T-TURN-10 — under both compilers. **This is the commit at which row 5's acceptance set closes.** Because the set was widened to T-TURN-01..10 in the GDD half, row 5's ✓ rested on nine of its ten written IDs between that merge and this commit; Q29, read per acceptance ID, is satisfied at 6ccd40b and at no earlier commit, and the evidence cell below names it. **No new acceptance ID was written here** — T-TURN-10 was minted in the GDD half — so §4.5's written-ID count does not move at this landing, its green count moves 49 → 50, and its unclosed count moves 21 → 20. **Q8(b) is now executed rather than only written:** a Build naming a factory that has already taken its build this turn is refused, fameTotal is unchanged by the refusal, and both dispositions of the first build — one that spawned and one that waits holding the slot (T-

FAME-04) — count against the allowance. Two rulings carried in with the code are recorded here because neither is derivable from the invariant text alone: the allowance's **renewal boundary is per side-turn, not per round** — `beginTurn` clears both act flags and the allowance together, so a factory captured at the start of a side's half arrives with a clear record, and T-TURN-10 asserts that case inside one round number — and **command ordering is unconstrained**, move-then-attack and attack-then-move both completing and spending the same two flags, which is what T-TURN-01's second printed check asserts. The debug driver's suite is **GATE-DRV-01..11, 11/11 under clang++ and MSVC both**, the same ID range as at `9086d6a`, and those IDs are still **not** `T-*`, so nothing flips on the driver's account here either. **Row 6 was re-run and is not re-recorded:** its commit, its `T-AI-01..06` plus `GATE-AI-SMOKE` 7/7 and its ledger row are unmoved, and only T-AI-01's printed command counter moves, as recorded above. **How `6ccd40b` was authored is deliberately not stated:** no harness claim is made for it, because none was established, and reporting a harness that did not run is the exact failure this ledger exists to prevent. **Row 8 then landed**, at `7c36303`: **UI binding contract**, gated **`T-UI-01`, `T-UI-02` and `GATE-CAP-PARTIAL`, 14/14 under clang++ and MSVC both** — `g++` is still not installed on this machine — and **the 14 counts printed checks, not IDs**: those three IDs close over 14 printed lines, four of which are ungated snapshot-shape checks that carry no ID at all and are labelled as such in the run. Pass-1 `cpp_reference/Ui.buggy.cpp` is blocked at **10/14** under both compilers — **four FAIL lines over two distinct IDs**, `T-UI-02` on all three of its checks and `GATE-CAP-PARTIAL` on its differential — on two defects: the reachable highlight recomputed as a hex-distance filter rather than queried from the module, and partial credit toward `objectivesHeld` for a capture in progress, which is the reading **Q14** refuses. Five tracked sources are cited, each probed present at that commit: `spec/ui_spec.md`, `cpp_reference/Ui.h`, `cpp_reference/Ui.good.cpp`, `cpp_reference/Ui.buggy.cpp`, `cpp_reference/test_ui.cpp`. The last of those is a tenth harness, so **twelve** tracked sources define `main()` at `7c36303` — ten test harnesses, one combat duel simulator, and one debug REPL — a figure taken by enumerating them at that commit rather than by adding one to the count above. **This row does not flip. T-UI-03 and T-UI-04 have not run**: they are in-editor Unreal Automation over widget bindings, marked † in §4.11, and **no in-editor pass exists at this commit**. Both are printed by name with that reason before the tally and recorded in the acceptance record. Q29 requires the full acceptance set at one commit and is applied **per acceptance ID as well as per row**, so the row records a partial pass and stays unverified, which is the posture rows 2 and 7 hold. Both are **written, unblocked and asserting**; what they lack is a harness, not a rule — a different state from **T-SCN-10**, which is reserved and **unwritten** on Q26, and from **T-MOVE-07**, which cannot be written until Q2 is ruled. **No new acceptance ID was written**, so §4.5's written-ID count does not move at this landing, its green count moves 50 → 52 and its unclosed count moves 20 → 18; `GATE-CAP-PARTIAL` mints none, on the `GATE-AI-SMOKE` precedent. **`GATE-CAP-PARTIAL` ran on a fixture configured with `captureTurns = 2`.** *Ferrum Crossing* ships **N = 1** (Q4), so the shipped scenario cannot reach the state that gate asserts about at all; N is per-scenario data and the fixture sets it, so **no map was invented**, and the run states this rather than leaving it to be inferred. **Nothing was invented to fill a gap.** No buildlist query is offered, because whether `T-UI-04`'s buildlist reaches the UI as a snapshot field or as a query is stated nowhere in this document; and three known-absent snapshot fields were left absent — the per-factory record of builds taken this turn, the income rate, and §2.11.1's DONE bit. **All three have since been ruled into §4.7 Stub 8**, on 2026-08-04: the first two as snapshot fields — the per-factory block `{hex, owner, hasBuiltThisTurn, buildWaiting, spawnBlocked}` and the per-side `incomePerTurn` — and the DONE bit into the stub's **presentation block**, which the rules module does not produce and which is the second half of the view-model beside the snapshot. **No code implemented any of the three at this commit**, and the same rulings minted **`T-UI-05`**, a headless snapshot-fidelity check no code implemented either, so what row 8 lacked after this commit was no longer the in-editor pass alone. The two snapshot fields and `T-UI-05` were built at `41a1452`, recorded at the end of this paragraph; the presentation block is declared there rather than filled. The debug driver's `snapshot` command now prints the snapshot rather than refusing on row 8's account — the snapshot and not the whole view-model, whose other half, §4.7 Stub 8's presentation block, is produced by no rules field and was implemented by no code at that commit, and its suite is **`GATE-DRV-01..12`, 12/12 under clang++ and MSVC both**, `GATE-DRV-12` being new at this commit; those IDs are still **not** `T-*`: the driver is not a §4.7 stub, has no row in the ledger below, and flips nothing. **How row 8 was authored differs from rows 5, 6 and 7, and is not reported in their words.** A **Claude Code session** authored it, and the same session also authored `spec/ui_spec.md`, as an elaboration of §4.7 Spec Stub 8. Each of rows 5, 6 and 7 names a Director-written stub in `spec/` that predated its build; §4.7 Stub 8 is Director-written, and the crew-repo spec derived from it is not. **Row 8 was the last unbuilt link on §4.11's critical path.** No in-editor harness is among the twelve `main()` definitions above. **Row 8 was then rebuilt**, at `41a1452`, whose parent is `7c36303`, against the snapshot additions ruled on 2026-08-04 and the `T-UI-05` minted with them — the GDD half of all of them having merged **before any code satisfied them**, as row 5's rebuild did. The commit changes **nine** files: `cpp_reference/Ui.h`, `cpp_reference/Ui.good.cpp`, `cpp_reference/Ui.buggy.cpp`, `cpp_reference/test_ui.cpp`, `cpp_reference/Driver.h`, `cpp_reference/Driver.good.cpp`, `cpp_reference/test_driver.cpp`,

spec/ui_spec.md and README.md; no other file in the repo changed. The gate is **T-UI-01, T-UI-02, T-UI-05 and GATE-CAP-PARTIAL, 34/34 under clang++ and MSVC both** — g++ is still not installed on this machine — and **the 34 counts printed checks, not IDs**. Pass-1 cpp_reference/Ui.buggy.cpp is blocked at **21/34** under both compilers — **13 FAIL lines over three distinct IDs**, T-UI-02, T-UI-05 and GATE-CAP-PARTIAL — while T-UI-01 is green in both passes. The other eight harnesses are **unchanged** under both compilers — hex 7/7, data 6/6, move 6/6, combat 17/17, economy 9/9, turn 11/11, scenario 12/12, ai 7/7 — their sources being byte-identical to 7c36303. **T-UI-05 closes here and the row still does not flip.** It is headless, has no in-editor half and no unrun fixture, so Q29 is satisfied for it; **T-UI-03 and T-UI-04 did not run**, being in-editor Unreal Automation marked † in §4.11, and **no editor pass exists at this commit**. **Both are printed by name with that reason before the tally, and that is measured at this commit rather than inherited** — cpp_reference/test_ui.cpp is one of the nine files this commit changes, so the earlier run's behaviour would not establish it. Rebuilt from Ui.good.cpp at 41a1452, the two NOT RUN lines are output lines **76** and **82** and the 34/34 passed tally is line **101**; each names its ID, states that it is in-editor Unreal Automation marked †, and states that no in-editor pass exists at this commit. That is the clang++ run; the MSVC run was verified at 34/34 with zero FAIL lines. **What changed about row 8 is which IDs it lacks, not whether it flips:** before this commit the IDs the row lacked included T-UI-05, which was headless and unimplemented, and after it the IDs it lacks are the in-editor T-UI-03 and T-UI-04 — the row keeps the partial-pass posture rows 2 and 7 hold. **No acceptance ID is minted by this commit**, so §4.5's written-ID count does not move at this landing, its green count moves 52 → 53 and its unclosed count moves 19 → 18. §4.4 scheduled T-UI-05 for **week 3**, so this closure is **ahead of the milestone table, not behind it**. **What was built is the snapshot's ruled additions**: isGuidedMarked, the per-factory group {hex, owner, hasBuiltThisTurn, buildWaiting, spawnBlocked} and the per-side incomePerTurn. The field contract T-UI-05 runs over is **27 rows — 22 mirrors and 5 DECLARED DERIVED**. DriverUnit gained a placement field, because isGuidedMarked is a property of the placement rather than of the unit's current hex. **The presentation block is declared and deliberately NOT filled** by buildUiSnapshot: both its members have owners that are not the rules module, and clause (c) does not reach them because the block is not in this invariant's subject. **The implementation of T-UI-05's clause (b) was rewritten because the first one passed its own pass-1 variant**, and that is recorded rather than smoothed over; the clause's own text is unedited. Clause (b) requires each DECLARED DERIVED field to be recomputed *inside* the check; the first implementation recomputed by calling the **same helpers the projection called**, so for the wrong incomePerTurn and the wrong isGuidedMarked — both planted in those shared helpers — the check compared each value to itself and returned clean. The three derivations are now **written out a second time inside the check**, which is why a duplication sits there that would otherwise read as an oversight. **A discrepancy is registered rather than acted on.** spawnBlocked is implemented on **occupancy alone**, mirroring cpp_reference/Economy.h::resolveBuilds, which asks isOccupied and never consults terrain — the reading §4.7 Stub 8 states, whose derivation is *no hex at or adjacent to the factory is free*; §4.10's parenthetical instead describes the same field as a function of unit positions **and terrain**. The build followed the stub, no invariant text changed and no acceptance ID was minted; a passability filter would report a factory blocked at a hex the shipped spawner would place on. **GATE-DRV-12 was extended in the same commit, and the extension is measured to discriminate:** the version at 7c36303, rebuilt against the pass-1 module, passes **12/12** — it could not see any of the five defects that module carries — while the extended version fails it at **11/12**, the single FAIL being GATE-DRV-12. The driver suite is **GATE-DRV-01..12, 12/12 under clang++ and MSVC both**, the same ID range as at 7c36303, and those IDs are still **not** T-*: the driver is not a §4.7 stub, has no row in the ledger below, and flips nothing. The driver's snapshot render gained the per-factory group, incomePerTurn and the guided mark **because its own comment claimed to render the view model and had stopped doing so** once the snapshot widened. **The stateHash in cpp_reference/Driver.h and cpp_reference/Driver.good.cpp is the driver's own debug digest**, gated by GATE-DRV-06, and is a different thing from §4.10's canonical state hash, which is **not implemented at this commit**: row 10 held no code then. **How this commit was authored differs from every row above it and is not reported in their words.** The **Director** wrote the code, in the main session; it was **not** authored by an agent crew and it is not the two-pass agent pipeline that produced the earlier rows. The pass-1/pass-2 split was performed **by hand**: cpp_reference/Ui.buggy.cpp was rebased on the finished Ui.good.cpp and the two original row-8 defects re-injected, rather than preserved in place from 7c36303. **The gate runner was then repaired**, at e06c44b, whose parent is 41a1452. Before it, python run.py --week1 did not compile past §4.11 row 5: row 6 failed with .\Driver.h:18:10: fatal error: 'Ui.h' file not found, and rows 6, 7, 8 and the debug driver never ran. **The cause was in the runner and in no module:** 7c36303 added #include "Ui.h" to cpp_reference/Driver.h, and the string Ui occurred **zero** times in crew/tools.py, so the runner's registry never gained Ui.h, Ui.cpp or test_ui.cpp; row 8's and T-UI-05's gates had been run by ad-hoc compiles, so nothing exercised the runner after that commit. **The breakage was found by running the runner at 41a1452** — it was not reported by anyone and was visible in no record, and it is recorded here rather than repaired quietly. **No module source, header, harness or**

invariant was touched by the repair; it is registry and record text only. After it, from a clean tree under clang++, the whole suite ran — row 1 7/7, row 2 6/6, row 3 6/6, row 4 9/9, row 5 11/11, row 6 7/7, row 7 12/12, row 8 34/34, and the debug driver 12/12, with row 8's pass-1 fixture blocked at 21/34 on 13 FAIL lines over three distinct IDs — the same figures already recorded above, now produced by the runner rather than by hand. The three link lines the repair changed were also built and run under MSVC: row 8 34/34, row 6 7/7, and the driver 12/12. **No ledger row moves on a repair to the runner** and no acceptance ID closed here. **Build-order row 9's headless half then landed**, at `b23823f`, with the UE project's half at `99fcb84`. New in the crew repo: `sync_stratrules.py`, `spec/integration_spec.md`, `run_integration_gate_fn` in `crew/tools.py`, and `python run.py --integration`, which `--week1` now also runs at its end; the vendored module itself is described in §4.9. The gate is **`T-INT-01` and `T-INT-04`, 2/2 under clang++**, at `rulesCommit` `b23823f` over the vendored tree at `99fcb84`. `T-INT-02`, `T-INT-03` and `T-INT-05` **did not run** — no in-editor Automation harness exists, and the runner prints that sentence by name before its tally. **Row 9's acceptance set therefore does not close**, on the Q29 reading applied per acceptance ID as well as per row. It has no row in the table below to leave unflipped: §4.11 calls it a *proposed* ledger row, and this landing creates none, since what landed is the vendoring step and not the bridge §4.9 part 2 describes. Whether the ledger should gain the row was filed for the Director and has since been ruled, at the end of this paragraph. **No acceptance ID was minted** — this row wrote no invariant text — so §4.5's written-ID count does not move at this landing, its green count moves 53 → 54 and its unclosed count moves 18 → 17. **Compiler detection was measured rather than assumed:** `g++` is not installed on this machine, `clang++` is found first, and `cl` is present but never reached. §4.9's `T-INT-04` states that any one of the four compiling clean satisfies it and that it does not require all four, so a single-compiler run is the invariant's own terms and not a shortfall against them. **The check was shown to fail before it was believed** — known-bad inputs, each restored afterwards: a comment appended to a vendored source (`T-INT-01` FAIL, `T-INT-04` PASS); the same edit **plus the manifest's own recorded hash updated to match it** (`T-INT-01` still FAIL); a vendored header deleted (`T-INT-01` FAIL and `T-INT-04` FAIL, because `Ai.good.cpp` genuinely stops compiling); an extra `Sneaky.good.cpp` added (`T-INT-01` FAIL, and `T-INT-04` compiled eleven files rather than ten); and `#include "CoreMinimal.h"` injected (`T-INT-04` FAIL and `T-INT-01` FAIL). **Two of those results exceeded the prediction made before running them** — the deleted header also broke `T-INT-04`, and the added file moved `T-INT-04`'s compile count — and both are the checks being right rather than the deliberate breaks being wrong. **The manifest-hash control is the one that matters:** the check does **not** trust the manifest's own hashes and does **not** import the vendor script's file list, taking only `rulesCommit` and re-deriving both the expected file set and every expected hash from the crew repo at that commit, so a manifest-versus-disk comparison would have passed that edit and this one does not. **When the UE project is absent the gate SKIPS with its reason stated and asserts nothing**; it does not pass, because a vacuous green `T-INT-01` beside a vendoring that never happened is the failure this ledger exists to prevent. The module was first vendored at `e06c44b` and re-vendored at `b23823f`; `cpp_reference/` is byte-identical between those two commits — `git diff --stat` over that path prints nothing — so only the manifest's `rulesCommit` line changed. **The `.gitattributes` rule is load-bearing rather than housekeeping:** the UE repo runs with `core.autocrlf=true` and that path carried no `text` attribute, so a fresh clone would have rewritten the vendored files to CRLF while the crew blob stayed LF, failing a byte-for-byte check on another machine; working-tree bytes, staged index blob and crew blob were verified to hash identically for every vendored file. **What this landing did not build is stated so it is not inferred.** The module was **defined** by `StratRules.Build.cs` and was **not wired into any build target** — `Stratocracy.uproject` did not list it, nothing depended on it, and **no UBT build was run** — the Director's decision that session, on the ground that registering it would have claimed an in-engine build that round could not verify. Registration and the first UBT build are recorded below, at their own commits. **No bridge exists:** no load mapping, no command surface, no event list, no actor and no widget, so §4.9 part 2 is unbuilt. `T-INT-03`'s subject is §4.10's canonical state hash, which build-order row 10 had not built at that commit; the `stateHash` in `cpp_reference/Driver.h` remains the driver's own debug digest under `GATE-DRV-06` and is a different thing, as recorded above. **How `e06c44b` and `b23823f` were authored is deliberately not stated:** no harness claim is made for either, because none was established. **`T-INT-01`'s running check was then widened**, at `d837fc8` in the crew repo with the UE project repo at `9dec48c`. Its PASS line reads, verbatim: *T-INT-01 source identity: all 22 files in Source/StratRules/ are accounted for at d837fc8 — 20 sources and StratRules.Build.cs hash-match tracked blobs, and StratRules.manifest.json recomputes byte-for-byte.* **Nothing in that directory is exempt from it**, and what it must satisfy is stated in §4.9's invariant text, amended with it. **Two files became tracked in the crew repo to make that possible** — `ue_module/StratRules.Build.cs` and `ue_module/manifest_fields.json`. `StratRules.Build.cs` was previously a string literal inside `sync_stratrules.py` and is now vendored from the git object store exactly as the sources are, so it has a counterpart to be matched against where it had none. **The check reads both of those from the git object store and imports nothing from the vendor script**, because a check that called the generator's own constant would assert nothing about the generator. **The widening was shown to be a widening rather than asserted as one:** the **pre-change** check was run

against each new known-bad input first, and each passed it **undetected** — a comment appended to `StratRules.Build.cs`, the manifest's `note` altered, and the manifest's `generator` altered — while the post-change check FAILs all three. An extra `Sneaky.good.cpp` was run beside them as a **control** and FAILs both before and after, which is what makes those three results a widening rather than a changed disposition. **Two further controls behaved as designed:** editing `ue_module/StratRules.Build.cs` in the **working tree** moves no verdict, because the check reads the blob at the recorded commit; and tampering a vendored source *together with* its recorded hash in the manifest still FAILs. **`T-INT-01`'s green re-dated to this commit**, over the vendored tree at `9dec48c`, and was recorded here rather than left at `b23823f`: `ue_module/manifest_fields.json` did not exist at `b23823f`, so the amended text cannot have been satisfied there, and the closure convention §4.5 states governs where the green lands. **`T-INT-04` was unaffected at this landing** — its text did not change, and it passed at `b23823f` and again here. Both greens moved on at `e19605e`, each for its own reason, recorded below. **No acceptance ID was minted here** — the amendment widened an ID that already existed — so §4.5's written-ID count does not move at this landing, its green count moves **54 → 55** and its unclosed count moves **17 → 16**, which is the second half of the movement whose first half the `b23823f` landing above records. **The whole crew gate is green at `d837fc8` under clang++** — row 1 7/7, row 2 6/6, row 3 6/6, row 4 9/9, row 5 11/11, row 6 7/7, row 7 12/12, row 8 34/34, the debug driver 12/12 and the integration gate 2/2 — the same figures already recorded above, and **no rules code changed**: `cpp_reference/` is byte-identical between `b23823f` and `d837fc8`, `git diff --stat` over that path printing nothing. `spec/integration_spec.md` was updated at `d837fc8` to describe the widened check, having described the old one. `T-INT-02`, `T-INT-03` and `T-INT-05` **still did not run** — no in-editor Automation harness exists — so **build-order row 9's acceptance set still does not close**, on the Q29 reading applied per acceptance ID as well as per row; the vendored module was **still not registered** in `Stratocracy.uproject` at that commit, and **no UBT build had been run** by it. **The ledger table below deliberately gains no row for build-order row 9, and no row is added, removed or flipped by this landing.** That is the ruling on the question filed above, and it turns on what a row would claim: what has landed is the vendoring step and its identity check, while the bridge §4.9 part 2 describes is unbuilt, so a row written now would either name a system that half-exists or start a practice of one row per half. The row is deferred until the bridge is built, so the ledger later gains a row describing a whole system. **The Director has since named that row without creating it (ruled 2026-08-05):** it is **Headless → Unreal integration**, its acceptance set is `T-INT-01..05`, and it is created as **one** row when §4.9 part 2 lands — which is this deferral's own reason stated forward rather than a second decision. **The text-versus-check question this landing settled is registered as Q33** (§4.7), written already marked RULED. **How `d837fc8` was authored is deliberately not stated:** no harness claim is made for it, because none was established. **Build-order row 10's part (a) then landed**, at `737f666`, with the UE project repo unmoved at `9dec48c`. New in the crew repo: `spec/save_spec.md`, `cpp_reference/Save.h`, `cpp_reference/Save.good.cpp`, `cpp_reference/Save.buggy.cpp` and `cpp_reference/test_save.cpp`, registered as row `save` in `crew/tools.py`, so this row runs under the `python run.py` gate `T-INT-04` names rather than by an ad-hoc compile. The last of those sources is an eleventh harness, so **thirteen** tracked sources define `main()` at `737f666` — eleven test harnesses, one combat duel simulator, and one debug REPL — a figure taken by enumerating them at that commit and diffing that list against the twelve at `7c36303` rather than by adding one to a count above: the difference is exactly `cpp_reference/test_save.cpp`, and nothing was removed. The gate is **`T-SAVE-04` plus `GATE-SAVE-PARSE`, 25/25 under clang++ and MSVC both** — `g++` is still not installed on this machine. Pass-1 `cpp_reference/Save.buggy.cpp` is blocked at **18/25** under both compilers — **seven FAIL lines over two distinct IDs, identical under each, and predicted in full before the run**. Its three defects are mechanical edits of `Save.good.cpp`: `loadSave` parses into the caller's object and then validates; `checkHeader` compares only `formatVersion` and `rulesCommit`; `onlyKeys` tolerates an unknown key. Against them `T-SAVE-04` (a), (b) and (f) fail the *state untouched* clause alone, (c), (d) and (g) are not refused at all, and `GATE-SAVE-PARSE`'s unknown-key check is tolerated. **`GATE-SAVE-PARSE` mints no acceptance ID**, on the `GATE-SCN-PARSE`, `GATE-AI-SMOKE` and `GATE-CAP-PARTIAL` precedent. **No acceptance ID was written**, so §4.5's written-ID count does not move at this landing, its green count moves **55 → 56** and its unclosed count moves **16 → 15**. **Every other row's tally is unchanged from the pre-change baseline, under both compilers** — row 1 7/7, row 2 6/6, row 3 6/6, row 4 9/9, row 5 11/11, row 6 7/7, row 7 12/12, row 8 34/34 and the debug driver 12/12 — and **`T-INT-01` and `T-INT-04` still pass 2/2 at `rulesCommit` `d837fc8` after this module landed**, which is what makes the no-vendoring decision recorded in §4.9 safe rather than merely convenient. **Six of row 10's seven IDs did not run, and each is named in the runner with its reason** rather than absorbed: `T-SAVE-01`, `T-SAVE-02`, `T-SAVE-03` and `T-SAVE-05` needed the headless replayer of part (b), which was unbuilt at that commit; `T-SAVE-06` is the only † of the seven, is asserted jointly with `T-INT-02`, and had at that commit neither an in-editor harness nor a built subject; `T-SAVE-07` needed row 6's self-play, which part (c) reached. **Row 10's acceptance set therefore does not close**, on the Q29 reading applied per acceptance ID as well as per row. **It has no row in the table below to leave unflipped**: §4.11 calls row 10 a *proposed* ledger row, and this landing creates none. That is row 9's posture and deliberately not the partial-pass

posture of rows 2, 7 and 8, each of which has a ledger row that a partial pass leaves standing unflipped; a row that does not exist has nothing to leave. **What part (a) deliberately does not do is stated so it is not inferred.** No command is applied, so **§4.10's canonical state hash is not defined here** — that is part (b)'s, and part (a) carries `stateHash` as an opaque required string. `dataHash` and `scenarioHash` are **compared as opaque strings and never recomputed**, which is what keeps this part's dependency set empty; its link set is `Save.cpp`, `Hex.cpp` and `test_save.cpp` and nothing else. No in-editor harness is among the thirteen `main()` definitions above. **How `737f666` was authored is deliberately not stated:** no harness claim is made for it, because none was established. **Build-order row 10's part (b) then landed**, at `ec15be6`, with the UE project repo unmoved at `9dec48c`. New in the crew repo: `spec/replay_spec.md`, `cpp_reference/Replay.h`, `cpp_reference/Replay.good.cpp`, `cpp_reference/Replay.buggy.cpp` and `cpp_reference/test_replay.cpp`; modified: `cpp_reference/Save.h`, `cpp_reference/Turn.h`, `cpp_reference/test_save.cpp`, `crew/tools.py`, `crew/offline.py`, `spec/save_spec.md`, `spec/integration_spec.md` and `spec/turn_spec.md`. **Part (b) was run against part (c)'s closure conditions rather than after them, on a ruling (2026-08-05).** §4.11 splits row 10's dependencies because week 2's log carried only `{Move, Attack}`, and puts closure in part (c) on rows 4 and 5 completing the command set, row 6 completing `T-SAVE-02`'s determinism composition, and `T-SAVE-07` needing row 6's self-play besides; rows 4, 5 and 6 are green at `647d4df`, `6ccd40b` and `d8284f1`, so the calendar reason for the split is gone and the gate's log carries the **complete §4.9 command set** — `Move`, `Attack`, `Build`, `Capture` and `EndTurn` — which is what Q29 requires before an acceptance ID may close. **Part (c) is not complete at this commit**, `T-SAVE-06` and `T-SAVE-07` not having closed, for the two reasons recorded below. **A segment of the gate's log is generated by row 6's AI rather than hand-written:** at this commit the AI emits a move, an attack on the opposing flag Tank and an end of turn, so the log crosses a turn boundary under both sides and `T-AI-06` sits inside `T-SAVE-02`'s composition rather than beside it. The new module is registered as registry row `replay` in `crew/tools.py`, so it runs under the `python run.py` gate rather than by an ad-hoc compile, and it is a **second** registry row rather than a widening of the `save` row: §4.11 says part (a) has *no deps at all*, which is encoded as that row's link set — `Save.cpp`, `Hex.cpp` and `test_save.cpp` — so widening it would have falsified §4.11's part-(a) dependency cell silently. The `replay` row's own link set is `Replay.cpp`, `Save.cpp`, `Hex.cpp`, `Data.cpp`, `Move.cpp`, `Economy.cpp`, `Turn.cpp`, `Combat.cpp`, `Ai.cpp` and `test_replay.cpp`; `Scenario.cpp` is in neither link set, `scenarioHash` being compared as an opaque string and never recomputed. `cpp_reference/test_replay.cpp` is a twelfth harness, so **fourteen** tracked sources define `main()` at `ec15be6` — a figure taken by enumerating them at that commit with `git grep -l` and diffing that list against the thirteen at `737f666` rather than by adding one to a count above: the difference is exactly `cpp_reference/test_replay.cpp`, and nothing was removed. The gate is **`T-SAVE-01`, `T-SAVE-02`, `T-SAVE-03` and `T-SAVE-05` plus `GATE-REPLAY-*`, 29/29 under clang++ and MSVC both** — `g++` is still not installed on this machine. Pass-1 `cpp_reference/Replay.buggy.cpp` is blocked at **21/29** under both compilers — **eight FAIL lines over four distinct IDs, identical under each, and predicted in full before the run**. Its three defects are mechanical edits of `Replay.good.cpp`: `replayLog` applies to the caller's state in place rather than to a copy; the hash walks units in storage order rather than in canonical hex order; and it omits the two per-unit turn flags. Against them `T-SAVE-05` fails clauses (e), (f) and (g) alone — (a) through (d) PASS, because `applied` is a reported field rather than the state — and `GATE-REPLAY-BYTES`, `GATE-REPLAY-ORDER` and all three `GATE-REPLAY-FLAGS` clauses fail, while **`T-SAVE-01`, `T-SAVE-02` and `T-SAVE-03` all PASS against the defective module**, every clause they carry comparing two runs that share the defect on both sides. **`GATE-REPLAY-*` mint no acceptance ID**, on the `GATE-SAVE-PARSE`, `GATE-AI-SMOKE` and `GATE-CAP-PARTIAL` precedent. **No acceptance ID was written**, so §4.5's written-ID count does not move at this landing, its green count moves **56 → 60** and its unclosed count moves **15 → 11**. **Every other row's tally is unchanged from the pre-change baseline, under both compilers** — row 1 7/7, row 2 6/6, row 3 6/6, row 4 9/9, row 5 11/11, row 6 7/7, row 7 12/12, row 8 34/34, row 10's part (a) 25/25, the debug driver 12/12 and the integration gate 2/2. **`strat::GameState` is declared here (ruled 2026-08-05)**, in `cpp_reference/Replay.h`, and the type's NAME was already this document's: §4.9 calls it *the authoritative `strat::GameState`*, §4.10 defines the canonical state hash over it, and `T-UI-05`'s own invariant text names it. At `737f666` the string `GameState` occurred in the crew repo only inside comments citing this GDD, in `cpp_reference/Save.h`, `cpp_reference/Ui.h` and `spec/turn_spec.md`. **No invariant's text is amended by this landing, so no closure re-dates and no green attribution moves:** `T-UI-05` is green at `41a1452` before this commit and after it. The type holds the mutable state the rules modules own — board, units, economy, turn — and not the §4.8 tables or the Stub-7 scenario, which arrive as arguments; it is a **fourth composition** beside row 6's `AiState`, row 8's `UiWorld` and the debug driver's `Session`, and none of the four owns a rule. The flag designation lives on it as `flagUnit[side]` rather than as a bool on the unit, because a dead flag is absent from the unit list and a per-unit bool cannot then separate *"the flag died"* from *"this side designates none"*. **§4.10's canonical state hash is implemented here, and its field list was restated into the groups the modules hold (ruled 2026-08-05):** two groups moved — `cpp_reference/Economy.h` holds capture progress on the tile, as `CaptureProgress {hex, unitId, turnsHeld}`, on the stated

ground that progress can never transfer (Q4, T-FAME-05), and keys `PendingBuild {factoryHex, side, defIndex}` by the factory hex — and of §4.10's three per-factory names, `hex` survives as a field, **`hasBuiltThisTurn` stops being a field without ceasing to be carried** — the build-allowance group is walked over the turn module's built-this-turn set, so an entry exists for a factory that has taken its build this turn and none exists for one that has not, which is a change of encoding and not of coverage — and **`buildWaiting` alone stops being carried**, on §4.10's own stated omission rule, since it is recomputable from the pending-build group. §4.10 carries the implemented grouping. `canonicalStateBytes` is exposed beside the digest, so a check can assert the serialisation and not only the digest. **`Replay.h::openTurn` runs `beginTurn`, then start-of-turn repair, then income, then the capture tick** — the order the Director ruled on 2026-08-03, the tick after income — and `cpp_reference/Driver.good.cpp`'s `openActiveTurn` performs that same sequence over the driver's `Session`. **`Replay.h::openTurn` owns that sequence over `strat::GameState` (ruled 2026-08-05)**, and `openActiveTurn` is recorded as a **second implementation over the driver's own `Session`**, retained until the driver reads from `GameState`. That is a duplication of a ruled **order** and not of a rule a module owns, and the convergence stays **filed rather than done**: the change request in `spec/replay_spec.md` is not withdrawn, it gains an owner. **Two things this landing found in its own instruments are recorded rather than smoothed over.** The gate's first fixture board carried a single factory: §2.8's domination backstop (`T-TURN-03`) ends a match the moment one side holds every factory, and row 5 checks it at `beginTurn`, so that board's match ended before its first command and every later command was refused; the shipped fixture carries a factory per side. And **the `T-SAVE-05` check was written so that it could not fail** — its bad log replayed from a state in which the log's first entry had already been spent, so entry 0 was refused as a repeat and nothing was applied under any implementation, and it passed against the very defect it exists to catch; running the defective module is what exposed it. It now replays from the initial state, so entries 0 and 1 apply before the refusal at index 2, and it asserts that index by value. **Stale claims in the crew repo were repaired in the same commit**, found with a whitespace-collapsed, case-insensitive sweep: `cpp_reference/Save.h`, `cpp_reference/Turn.h`, `cpp_reference/test_save.cpp`, `crew/tools.py`, `spec/save_spec.md`, `spec/integration_spec.md` and `spec/turn_spec.md` each stated either that §4.10's canonical state hash had no implementation or that `T-SAVE-01/02/03/05` still awaited a replayer. `spec/integration_spec.md`'s is the load-bearing one: it said `T-INT-02` and `T-INT-03` *"have no subject even in an editor pass"*, which this landing makes false. **`T-SAVE-06` did not close.** It is the only † of row 10's seven, it is asserted jointly with `T-INT-02`, and no in-editor Automation harness exists; its other blocker — §4.10's canonical state hash having no implementation — is removed here, and among what it still waits on are the in-editor Automation harness and a vendored replayer: it is asserted jointly with `T-INT-02`, whose replay runs **in-engine** and so needs the replayer compiled into the engine and therefore vendored, and `Replay` is ruled out of vendoring until a bridge consumer exists. Both what *the editor pass* denotes and what running it does not supply are stated at §4.9. **`T-SAVE-07` did not close:** `cpp_reference/selfplay.cpp` is a combat-only 1v1 duel harness that prints a table of duel outcomes, and it writes no §4.10 command log. **Row 10's acceptance set therefore does not close**, on the Q29 reading applied per acceptance ID as well as per row, and its unclosed count moves **6 → 2**. **No ledger row is created, flipped or removed by this landing**: §4.11 calls row 10 a *proposed* ledger row, which is row 9's posture and deliberately not the partial-pass posture of rows 2, 7 and 8. **The Director has since named that row without creating it (ruled 2026-08-05), on Ruling K's precedent for row 9:** it is **Save & replay**, its acceptance set is `T-SAVE-01..07`, and it is created as **one** row when that full set closes — which under Q29 is the same condition as its flipping — so parts (a), (b) and (c) resolve to one ledger row rather than to several, and what it waits on is `T-SAVE-06` alone, `T-SAVE-07` having since closed at `1ee890e` on part (c). **How `ec15be6` was authored is deliberately not stated:** no harness claim is made for it, because none was established. **Build-order row 10's part (c) then landed**, at `1ee890e`, with the UE project repo unmoved at `9dec48c`. New in the crew repo: `spec/balance_spec.md`, `cpp_reference/Balance.h`, `cpp_reference/Balance.good.cpp`, `cpp_reference/Balance.buggy.cpp` and `cpp_reference/test_balance.cpp`; modified: `crew/tools.py` and `crew/offline.py`. **What landed is a producer of §4.10 command logs**, and the in-editor Automation harness `T-SAVE-06` waits on is untouched by it. The new module is registered as registry row `balance` in `crew/tools.py`, so it runs under the `python run.py` gate rather than by an ad-hoc compile, and it is row 10's **third** registry row beside `save` and `replay`: its link set encodes part (c)'s own dependency claim — rows 4, 5 and 6, the command set, the match that runs to a result, and the AI that plays it — while folding it into the `replay` row would have put row 6 inside part (b)'s stated claim. `cpp_reference/test_balance.cpp` is a thirteenth harness, so **fifteen** tracked sources define `main()` at `1ee890e` — thirteen test harnesses, one combat duel simulator, and one debug REPL — a figure taken by enumerating them at that commit with `git grep -l` and diffing that list against the fourteen at `ec15be6` rather than by adding one to a count above: the difference is exactly `cpp_reference/test_balance.cpp`, and nothing was removed. The gate is **`T-SAVE-07` plus `GATE-BALANCE-*`, 12/12 under clang++ and MSVC both**. Pass-1 `cpp_reference/Balance.buggy.cpp` is blocked at **6/12** under both compilers — **six FAIL lines, identical under each, and predicted in full at clause level before the run**:

GATE-BALANCE-TRANSLATE-ATTACK-IS-TARGET-HEX, GATE-BALANCE-TRANSLATE-BUILD-NAMES-THE-UNIT-BUILT, GATE-BALANCE-RUN-ENDS-WITH-A-TIER, GATE-BALANCE-COMMAND-SET-IS-THE-AIS-FOUR, GATE-BALANCE-LOG-HOLDS-ONLY-ACCEPTED-COMMANDS, and T-SAVE-07 clause (b). **What the defective module did not disturb is recorded rather than smoothed over: T-SAVE-07 clauses (a) and (c) PASSED against it.** The §4.10 format is agnostic to whether the rules accepted a command, so a log of refused entries still validates and still round-trips byte-identically; only clause (b), which replays the log, can see the difference, which is why this ID's discriminating power sits in one clause of three. The gate's own run emitted a **35-command log over a 6-turn cap**, printed by the runner. **GATE-BALANCE-* mint no acceptance ID**, on the GATE-SAVE-PARSE and GATE-REPLAY-* precedent. **No acceptance ID was written**, so §4.5's written-ID count does not move at this landing, its green count moves **60 → 61** and its unclosed count moves **11 → 10**. **The module owns the translation** between row 6's command vocabulary and §4.9's, and the discipline that only an **accepted** command enters the log; it chooses no move — `nextCommand` does — and applies no rule — `applyCommand` does. **A self-play log carries four command kinds, not five, and that is a property of the AI rather than of the format.** `cpp_reference/Ai.h` states that capture is deliberately outside the AI's vocabulary — a turn-boundary event the caller runs beside income, the AI's part of it being the move onto the objective (T-AI-03) — so `AiCommandKind` has four members where §4.9 has five, and a self-play match emits `Move`, `Attack`, `Build` and `EndTurn` and never `Capture`. The complete §4.9 command set was exercised by part (b)'s hand-authored log at ec15be6. T-SAVE-07 asserts **format compatibility, not command coverage**, so those four are its whole written fixture set, Q29 is satisfied over that set, and the gate asserts the four are present and the fifth absent rather than leaving the absence to be read as a shortfall. Part (c) drives its match through `applyCommand`, so the ruled start-of-turn order runs via `Replay.h::openTurn` here and not via the driver's `openActiveTurn`. **The module could not be named `Selfplay`, and the reason is a fact about the repo rather than a preference:** `cpp_reference/selfplay.cpp` is tracked and `crew/tools.py::ensure_workspace` copies it into `build/` on every gate call, the build filesystem is case-insensitive, so a `Selfplay.cpp` authored beside it is the same file — the duel harness's `main()` replaced the module, which surfaced as a linker error naming a duplicate `main` rather than as an overwrite — and §4.9 names `selfplay.cpp` in its enumeration of what is excluded from vendoring, so renaming that file would falsify a merged sentence. `Balance` is also the name this document already uses for the role that owns the artifact. **`cpp_reference/selfplay.cpp` is unchanged by this landing**: it is still tracked, still a combat-only 1v1 duel harness over `cpp_reference/Combat.h` that prints a table of duel outcomes and opens no file, still one of the files §4.9 excludes from vendoring because a UBT module cannot hold a second `main()`, and still not a producer of a §4.10 log. It did not become the new module and was not replaced by it. **`Balance` is deliberately not vendored** on the same ruling the other two unvendored modules stand on (§4.9), and T-INT-01 and T-INT-04 are unaffected — still 2/2 at `rulesCommit d837fc8`, run this session after the change. **Nothing was compiled by UBT at this landing, no editor was launched, and no UE project file was touched. T-SAVE-07 closes here. T-SAVE-06 did not close**: it is the only † of row 10's seven, it is asserted jointly with T-INT-02, and no in-editor Automation harness exists. **Row 10's acceptance set therefore does not close**, on the Q29 reading applied per acceptance ID as well as per row, and its unclosed count moves **2 → 1**, T-SAVE-06 alone. **No ledger row is created, flipped or removed by this landing**: §4.11 calls row 10 a *proposed* ledger row, which is row 9's posture and deliberately not the partial-pass posture of rows 2, 7 and 8, and the row the Director named without creating it is created as one row when its full acceptance set closes. **How `1ee890e` was authored is deliberately not stated:** no harness claim is made for it, because none was established. **The UBT landing then followed**, at 031ee20 in the crew repo — over 30a73f0, which added the `Build.cs` line and a spec repair, 188f966, which corrects a sentence `30a73f0` had written ahead of a landing that did not then happen, and e19605e — with the UE project repo at `a13626f`, where `Stratocracy.uproject`, `Source/StratRules/StratRules.Build.cs`, `Source/StratRules/StratRules.manifest.json` and `Source/StratRules/Turn.h` changed. `Stratocracy.uproject` now lists `StratRules`, and building the `StratocracyEditor` target compiles the vendored crew modules and links `UnrealEditor-StratRules.dll`. **These sources had never been compiled by UBT before.** The first attempt **failed**, on one diagnostic — *Driver.good.cpp(657,24): error C4456: declaration of 'r' hides previous local declaration* — because both targets set `DefaultBuildSettings = BuildSettingsVersion.V7`, and `BuildSettingsVersion.V2` onward raises `ShadowVariableWarningLevel` to `Error`, read from the engine's own `TargetRules.cs`; every other vendored crew module compiled clean. With `CppCompileWarningSettings.ShadowVariableWarningLevel = WarningLevel.Warning` in `StratRules.Build.cs`, scoped to that UBT module, the build succeeds, and C4456 and a C4457 that was never an error still print. **What this landing does not establish is stated so it is not inferred:** it compiled and it linked. **No editor was launched, nothing was executed, no runtime behaviour was observed, and the build mints no acceptance ID and closes none.** A green UBT build is not evidence for any acceptance ID — T-INT-04 asserts the **standalone** compile, outside UBT, while in-engine compilation is what the editor pass gates, and that pass does not exist. What the build does add is an MSVC compile **through UBT, under the engine's own flags** — a

toolchain configuration these sources had not been built under, and not a compiler they had not seen, the standalone gate having compiled them under clang++ and MSVC both throughout. **`T-INT-01`'s running check was then found defective, and repaired at `e19605e`.** The check derived its expected file set with `git ls-tree --name-only <rulesCommit>:cpp_reference` filtered to `*.h` and `*.good.cpp` — that is, every crew module — while the vendored set is the ten crew modules §4.9 enumerates, `Save`, `Replay` and `Balance` being ruled out of vendoring. Those two sets coincided only until `Save` landed at `737f666`. Measured: re-vendoring at `30a73f0` produced *FAIL T-INT-01 … missing 6: Balance.good.cpp, Balance.h, Replay.good.cpp, Replay.h, Save.good.cpp, Save.h*. **The consequence is what makes it worth a ledger entry: `rulesCommit` could not be advanced to any commit at or after `737f666`, for any reason.** The vendored manifest recorded `d837fc8` throughout, and `d837fc8` is an ancestor of `737f666`, so the collision had no occasion to show itself while three landings went by. `T-INT-01`'s written text quantifies over *every file in `Source/StratRules/`* — the vendored tree — and not over every crew source, so **the check asserted more than the text did**, which is the mirror image of Q33 and is ruled the mirror way: **the vendored set is declared, not inferred, and `T-INT-01`'s invariant text is unchanged**, the repaired check asserting what the text always said, so this is the check catching up rather than a widening. `ue_module/vendored_set.json` declares the set; both `sync_stratrules.py` and the check read it from the git object store at the commit in question, so neither imports the other's constants, and the declaration **must partition** the crew's modules — every crew module appears in exactly one of `vendored` and `excluded`. **`rulesCommit` moves `d837fc8` → `e19605e`**, where `T-INT-01` and `T-INT-04` pass **2/2**. The `T-INT-01` PASS line reads, verbatim: *T-INT-01 source identity: all 22 files in Source/StratRules/ are accounted for at e19605e — 20 sources and StratRules.Build.cs hash-match tracked blobs, StratRules.manifest.json recomputes byte-for-byte, and the declared vendored set partitions the 13 crew modules (10 vendored, 3 ruled out)*. The full `python run.py --week1` record is accepted, with every row's tally unchanged from the pre-change baseline taken before anything was edited — row 1 7/7, row 2 6/6, row 3 6/6, row 4 9/9, row 5 11/11, row 6 7/7, row 7 12/12, row 8 34/34, row 10 parts (a) 25/25, (b) 29/29 and (c) 12/12, and the debug driver 12/12. **The gate was shown to fail before it was trusted**, on known-bad inputs each restored afterwards: a comment appended to a vendored source FAILs `T-INT-01` on a hash mismatch; a deleted vendored header FAILs `T-INT-01` missing and FAILs `T-INT-04`; an extra `Sneaky.good.cpp` FAILs `T-INT-01` unexpected; a comment appended to the vendored `StratRules.Build.cs` FAILs `T-INT-01` on a hash mismatch; a crew module in neither declared list FAILs `T-INT-01` unaccounted; a module in both declared lists FAILs `T-INT-01` declared-both; and a declared module with no `cpp_reference` source FAILs `T-INT-01` no-source. One input is a **design control rather than a defect**: the declaration edited in the **working tree only** PASSes, the verdict unmoved, because the check reads the object store. The first attempt at the no-source control was **invalid** — `sync` aborts when a declared module has no source, so the manifest was never rewritten and the gate re-read the previous control's commit; it was redone against a distinct commit. Those are the inputs that were run and the verdicts they produced, and no claim of coverage beyond them is made here. **Both `T-INT` closures re-date to `e19605e`, for different reasons, and each is stated separately.** `T-INT-01` re-dates because it asserts identity **at `rulesCommit`**, and `rulesCommit` moved. `T-INT-04` re-dates because it compiles the **vendored copy**, and one vendored source's bytes changed: `Turn.h`, which changed at `ec15be6` with row 10's part (b) and is, measured file by file, the only vendored source whose bytes differ between `d837fc8` and `e19605e`. Neither ID's **text** changed, so this is a closure movement and not a widening. **No acceptance ID is minted and none closes**, so §4.5's written, green and unclosed figures do not move; what moves is the by-commit attribution of two already-green IDs. **No ledger row is created, flipped or removed by this landing**, and the register of open questions gains no row and loses none. `T-SAVE-06` remains the only † of row 10's seven, and among what it waits on are the in-editor Automation harness and a vendored replayer: it is asserted jointly with `T-INT-02`, whose replay runs **in-engine** and so needs the replayer compiled into the engine, and `Replay` is ruled out of vendoring until a bridge consumer exists. Both what *the editor pass* denotes and what running it does not supply are stated at §4.9. `cpp_reference/selfplay.cpp` is untouched. `Save`, `Replay` and `Balance` remain unvendored, and the ruling that keeps them out is now **stated in the declaration** rather than implied by a glob. **Two further rulings are recorded here.** The win-rate / turn-length **distribution reporter** is scheduled in the week that already owes the sims and the tuning themselves, and it gains **no build-order row and no acceptance ID**: it aggregates logs, defines no format and owns no rule, §4.10 already enumerates the Balance Analyst self-play log among the format's consumers, and the producer of that log has landed. The driver's `openActiveTurn` retirement condition stays **filed and deliberately unasserted**: the driver owns no rule, so a second implementation of a ruled *order* is duplication rather than unsoundness, and nothing green depends on it. **Two forms were considered and declined, with the reason recorded so neither is re-filed:** a document-wide reachability form that survives a landing, and a rule that a "has since" sentence must carry a commit pin. Both are ways of phrasing claims, and a closed list of legal phrasings has been adopted and withdrawn here before. **How `031ee20` was authored is deliberately not stated:** no harness claim is made for it, because none was established. **The in-editor Automation harness then landed**, at `c2edae0` in the crew repo — over `b1ea992`, which vendors the §4.8 CSVs into the UE project — with the Stratocracy UE project repo at `fed8ae9`. New in the UE project: `EUnitType`, a `UENUM` mirroring `strat::UnitType` with

the enumerator order pinned; the row structs `FUnitRow`, `FTerrainRow` and `FEffectivenessRow`, each deriving `FTableRowBase`; an `ImportStratData` commandlet that imports the vendored CSVs into `DT_Units` (4 rows), `DT_Terrain` (7 rows) and `DT_Effectiveness` (4 rows); and an in-editor Unreal Automation suite. **All of it lives in the `Stratocracy` module and none of it in `Source/StratRules/`**, which is the constraint §4.9 states: `T-INT-01` accounts for every file in that directory, so a test file placed there would fail a green ID. The gate is **`T-DATA-05`, 4/4, plus `GATE-DATA-VENDOR`, 5/5 in-editor** at `fed8ae9`, and **`T-DATA-01..04, 06` plus `GATE-DATA-HARDFAIL`, 6/6 headless** at `c2edae0`, where `python run.py --week1` prints **WEEK-1 GATE PASS** with every row at its full tally and **INTEGRATION GATE PASS 2/2**, under MSVC. **`GATE-DATA-VENDOR` and `GATE-DATA-HARDFAIL` mint no acceptance ID**, on the `GATE-AI-SMOKE`, `GATE-SCN-PARSE`, `GATE-CAP-PARTIAL`, `GATE-SAVE-PARSE`, `GATE-REPLAY-*` and `GATE-BALANCE-*` precedent. **The §4.8 CSVs are vendored, and their bytes are asserted rather than assumed:** they are copied from the crew object store into the UE project beside a manifest recording each file's sha256 and the crew commit it came from, and `GATE-DATA-VENDOR` asserts the bytes on disk are the recorded ones. That is what makes the two halves of row 2's acceptance set halves of one thing — the headless loader and the editor import read the same authored bytes. **Row 2 flips.** Its full acceptance set passes: the headless half at `b1ea992` and `T-DATA-05` over the UE tree at `fed8ae9`. Q29's *at one commit* is read across a repo **pair**, in the two-repo pinning form this ledger already uses for `T-INT-01`, registered as **Q34** (§4.7) and written already marked RULED. What answers Q29's concern on its own terms: no fixture went unrun, and `GATE-DATA-VENDOR` establishes that both halves ran against the same data bytes. **The headless IDs' greens do not move:** `T-DATA-01..04` and `06` stay credited at `c224825`, their run at `b1ea992` being a re-run on the precedent that a re-run is not a re-recording; what `b1ea992` supplies is the crew half of the pair the flip is cited against. **No acceptance ID was minted and none was re-dated**, so §4.5's written-ID count does not move at this landing, its green count moves **61 → 62** on `T-DATA-05`'s closure at `fed8ae9`, and its unclosed count moves **10 → 9**. **No ledger row other than row 2 changes state. The suite was proven able to FAIL on six known-bad inputs, each caught by the intended check and only that check**: a perturbed CSV value (parity and vendor both); a corrupted manifest hash (vendor only); an unrecorded file in the data directory (vendor only); a DataTable asset drifted from the CSV (parity only); a fifth enumerator (the runtime mirror check); and swapped enumerator values, which stop the build at a `static_assert`. **A defect in the previous round's registration was found by launching the editor, and is recorded rather than repaired quietly.** `Source/StratRules/` contains no `IMPLEMENT_MODULE`, so with `StratRules` listed in `Stratocracy.uproject`'s Modules array the editor aborted at startup with *"The game module 'StratRules' could not be successfully initialized"* and exited: at `a13626f` nothing in-editor could run — not a commandlet, not an Automation test, not the editor. The round that registered it gated that the module compiles and links, and never launched the editor. `StratRules` is **removed from the Modules array at `fed8ae9`** and remains a **link dependency of the `Stratocracy` module**, still built by UBT and still linked. The removal touches no vendored byte, so `T-INT-01` and `T-INT-04` do not re-date: both remain green at `e19605e`, and the running check reports all 22 files in `Source/StratRules/` accounted for. **§4.8's seven bool UStruct fields drop the `b` prefix** — `CanCapture`, `PassLand`, `PassAir`, `PassSea`, `Capturable`, `IsSpawnPoint`, `IsRepairPoint` — measured at engine source: `DataTableUtils::GetPropertyImportNames` accepts only `GetName()` and `GetAuthoredName()`, there is no metadata bridge, and `meta=(DisplayName=...)` was tried and failed identically. **The CSV column names and the headless field names are unchanged**, so one canonical CSV is still read by both sides with no transformation between them. **Stale claims in the crew repo were repaired at `c2edae0`:** the gate runner, its test harnesses, two specs and the README asserted that no in-editor Automation harness exists, at thirteen sites in five spellings, and each site now names what its ID still lacks. The records above of the runner's printed output at `b23823f` and `41a1452` describe those commits and are unmoved by the repair. **What this landing does not close is stated so it is not inferred.** It closes no acceptance ID other than `T-DATA-05`. `T-INT-02` still needs a vendored replayer — its replay runs **in-engine**, and `Replay` is ruled out of vendoring until a bridge consumer exists; `T-INT-03` still needs the command surface, which is part of the unbuilt bridge; `T-INT-05`, `T-UI-03` and `T-UI-04` still need real Stratocracy widgets; and `T-SAVE-06` is asserted jointly with `T-INT-02` and inherits that blocker. Build-order rows 9 and 10 therefore still do not close, and the ledger gains a row for neither. **How `b1ea992` and `c2edae0` were authored is deliberately not stated:** no harness claim is made for either, because none was established.* Legend: **Author** ∈ {agent, agent+human, human}; **Agent-verified** ∈ {✓, —}; **Evidence** cites the git commit + passing test IDs so any row is independently checkable.

System	Author	Agent-verified?	Evidence (commit · tests)
Combat resolution	agent	✓	`cpp_reference/Combat.good.cpp` + `cpp_reference/test_combat.cpp` @ `5ffa8d6` · COMBAT-01..10 (10/10)
Test suite	agent	✓	`cpp_reference/test_combat.cpp` @ `5ffa8d6` · 17/17 invariants; `cpp_reference/test_hex.cpp`, `cpp_reference/test_data.cpp`,

System	Author	Agent-verified?	Evidence (commit / tests)
			`cpp_reference/test_move.cpp` @ `c224825` · T[illegible] of the IDs that [illegible] testable tests) `python run.py`
Hex grid & math	agent	✓	`cpp_reference/Hex.good.cpp` + `cpp_reference/test_hex.cpp` @ `c224825` · T-HEX-01..07 (7/7)
Movement & pathfinding	agent	✓	`cpp_reference/Move.good.cpp` + `cpp_reference/test_move.cpp` @ `c224825` · T-MOVE-01..06 (6/6); T-MOVE-07 reserved-unwrit on Q2
Capture & Fame economy	agent	✓	`cpp_reference/Economy.good.cpp` + `cpp_reference/test_economy.cpp` @ `647d4df` FAME-01..09 (9/9)
Turn loop & win / tiebreak	agent	✓	`cpp_reference/Turn.good.cpp` + `cpp_reference/test_turn.cpp` @ `6ccd40b` · T-TURN-01..10, all ten closing — the runner prints PASS lines over those ten IDs, T-TURN-01 printi two of them, so the 11 counts checks and not IDs TURN-01..09 first closed at `ad77b13`; `6ccd40b` is rebuild at which the **full** set closes at one commit, which is what Q29 requires
Opponent AI	agent	✓	`cpp_reference/Ai.good.cpp` + `cpp_reference/test_ai.cpp` @ `d8284f1` · T-AI 01..06 (6/6) + GATE-AI-SMOKE
Data tables (units/terrain)	agent	✓	`cpp_reference/Data.good.cpp` + `cpp_reference/test_data.cpp` over `data/units.csv`, `data/terrain.csv`, `data/effectiveness.csv` · T-DATA-01..04, 06 (5/5) headless @ `c224825`, re-run green @ `b1ea9` **T-DATA-05 (in-editor) 4/4** over the imported DataTables and the `EUnitType` mirror @ `fed8ae9` the Stratocracy UE project repo, beside `GATE-DAT` `VENDOR`, which mints no acceptance ID. The full s closes across that repo pair, which is how Q29's *one commit* reads for a two-repo acceptance set (Q34)
Repair (owned-tile heal, §2.7)	agent	✓	`cpp_reference/Combat.good.cpp::repairAmou` @ `5ffa8d6` · T-REPAIR-01..07 (7/7)
Type-effectiveness (§3 spec)	agent	✓	`cpp_reference/Combat.good.cpp::effectiver` @ `5ffa8d6` · T-COMBAT-09..10 (neutral, 2/2)
Content / scenario	agent	—	**Partial pass — not a flip.** `cpp_reference/Scenario.good.cpp` + `cpp_reference/test_scenario.cpp` over `data/ferrum_crossing.json` @ `9086d6a` · T-SC 01..07 (7/7) headless, plus `GATE-SCN-PARSE` and `GATE-SCN-HASH`, which mint no acceptance ID. **T-SCN-08, T-SCN-09 and T-SCN-11 ran only part their fixture sets** — the four fixtures that did no run each need a stretch map authored as a scenari file — so the acceptance set is incomplete at this commit and Q29, read per ID, keeps the row unverified
UI	agent	—	**Partial pass — not a flip.** `cpp_reference/Ui.good.cpp` + `cpp_reference/test_ui.cpp` @ `41a1452` · T-UI T-UI-02 and T-UI-05 (3/3) headless — the runner prints **34** PASS lines, which count checks and not IDs — plus `GATE-CAP-PARTIAL`, which mints no acceptance ID and which ran on a fixture configu with `captureTurns = 2`, a state the shipped N = scenario cannot reach. T-UI-01 and T-UI-02 first closed at `7c36303`; `41a1452` is the commit at whic **T-UI-05** closes, the snapshot additions ruled besid having been built there. **T-UI-03 and T-UI-04 hav not run**: they are in-editor Unreal Automation over widget bindings and no in-editor pass exists at eith commit. The acceptance set is incomplete on that count and Q29, read per ID, keeps the row unveri

Ten rows carry a ✓ in the table above, and two more carry evidence without one.

Four came from the headless Combat module built for the Assignment-3 agent crew (github.com/jakemartin/stratocracy-crew, commit `5ffa8d6`): **Combat resolution** and its **Test suite**, plus **Repair** and **Type-effectiveness**, all agent-authored from `spec/combat_spec.md` (+ `spec/combat_spec_addendum.md`) and verified by the real compile+test gate — a live CrewAI run authored the module and the Test Engineer certified **17/17** on a live `g++`/`clang++` compile+run. **Hex grid & math** and **Movement & pathfinding** joined them at `c224825`, and their evidence sentence is deliberately not the one above: a **Claude Code session** authored the three week-1 modules against the Director-written stubs `spec/hex_spec.md`, `spec/data_spec.md` and `spec/move_spec.md` — **not a live CrewAI run**, since `crew/tasks.py` is still written against the Combat spec alone — and the Test Engineer certified them through the same `python run.py` compile+run pipeline, **18/18 on the IDs that ran, under clang++ and MSVC both**. The author is an agent either way; the two sentences differ because the harness differed, and reporting a harness that did not run is the exact failure this ledger exists to prevent. **Capture & Fame economy** joined at `647d4df` — T-FAME-01..09, **9/9 under clang++ and MSVC both** — and it leaves **no ID uncovered**: unlike row 2 it has no in-editor half, and unlike row 3 no reserved ID, so its full acceptance set closes at one commit and Q29 is satisfied rather than blocking. **It belongs to the second of those two sentences and does not start a third:** a **Claude Code session** authored it from the Director-written stub `spec/economy_spec.md`, again **not a live CrewAI run** — `crew/tasks.py` is **byte-unchanged from `5ffa8d6` to `647d4df`**, so the file the live crew runs from still describes the Combat spec alone. That check establishes that the file has not moved across those commits; it is not evidence about what any run did, and the harness is reported here because it is known and not because a diff proved it. **Turn loop & win / tiebreak** joined at `ad77b13` — T-TURN-01..09, **9/9 under clang++ and MSVC both** — and its acceptance set has since widened to **T-TURN-01..10**, green at the `6ccd40b` rebuild under clang++ and MSVC both, on **11 printed checks over those ten IDs**. It leaves **no ID uncovered**: no in-editor half and no reserved ID, so its full acceptance set closes at one commit — `6ccd40b`, not `ad77b13` — and Q29, read per acceptance ID, is satisfied rather than blocking. **It belongs to that same second sentence and does not start a third:** a **Claude Code session** authored it from the Director-written stub `spec/turn_spec.md`, again **not a live CrewAI run** — `crew/tasks.py` is **byte-unchanged from `5ffa8d6` to `ad77b13`**, so the file the live crew runs from still describes the Combat spec alone, and that check likewise establishes only that the file has not moved across those commits. **Opponent AI** joined at `d8284f1` — T-AI-01..06 plus GATE-AI-SMOKE, **7/7 under clang++ and MSVC both** — and it leaves **no ID uncovered** either: no in-editor half and no reserved ID, so its full acceptance set closes at one commit and Q29 is satisfied rather than blocking. Six of those seven checks are numbered IDs; GATE-AI-SMOKE mints none, for the reason the paragraph above gives. **It belongs to that same second sentence and does not start a third:** a **Claude Code session** authored it from the Director-written stub `spec/ai_spec.md`, again **not a live CrewAI run** — `crew/tasks.py` is **byte-unchanged from `5ffa8d6` to `d8284f1`**, so the file the live crew runs from still describes the Combat spec alone, and that check carries the same extent as the two before it: the file has not moved. **Data tables** joined at `b1ea992` with `fed8ae9` in the Stratocracy UE project repo — T-DATA-01..04 and 06 headless, **T-DATA-05 in the editor pass**, 4/4 — and it is the first row whose acceptance set closes across a **repo pair** rather than at a single commit, which is what Q34 rules on. Seven IDs are still recorded as **uncovered** rather than omitted, in **two states that are not the same state**. Two are **unwritten**: **T-MOVE-07**, reserved on Q2, and **T-SCN-10**, reserved on Q26 — no invariant text exists for either, so neither asserts and neither is waiting on a run. Five are **written and not green**: **T-SCN-08**, **T-SCN-09** and **T-SCN-11**, each written, unblocked and asserting, each having run only part of its fixture set; and **T-UI-03** and **T-UI-04**, written, unblocked and asserting, in-editor Unreal Automation for which no in-editor pass exists at either of row 8's two commits — among what those two wait on are real Stratocracy widgets, the in-editor Automation harness they also lacked having landed at `fed8ae9`, and they are the IDs row 8 still lacks. T-DATA-05 was why **Data tables** carried evidence without a ✓; it has since run, 4/4 in the editor pass at `fed8ae9`, completing that row's acceptance set across the repo pair Q34 rules on, and that row now carries one. **Two rows still carry evidence without a ✓, each recording a partial pass and staying unverified on Q29:** the three `T-SCN-` IDs are why **Content / scenario** does the same at `9086d6a`, Q29 there being read per ID, and T-UI-03 and T-UI-04 are why **UI** does the same at `41a1452`, for want of an in-editor pass at that commit and at `7c36303` before it. *(Every **crew-repo** commit this section **cited at** `ec15be6` — `d8284f1`, row 6's, included — is an ancestor of `ec15be6`, measured with `git merge-base --is-ancestor` per sha, so every commit link this section carried then resolves; each commit cited since is pinned at the landing that cites it, and the §3 status line above carries that pinning. That form was ruled on 2026-08-05, and the ruling is confined to this sentence: it matches the §3 status line above, whose substance is unchanged, and restating every reachability claim in this document against a named commit was considered in the same ruling and declined. **Not every cited commit is an object in the crew repo**: `99fcb84`, `9dec48c` and `a13626f` are not objects in it at all, measured with `git cat-file -e` per sha and controlled against `e19605e`, which does resolve there: each is a commit in the **Stratocracy** UE project repo, whose tree it pins, and this parenthetical makes no claim about how any of them stands to any branch there (ruled 2026-08-05). The claim this replaces was that each is reachable from the head of `master`; a head expires and a sha does*

not, which is the defect the §3 status line above records, so the form goes rather than being re-measured. That is Ruling S's move applied to the other half, and the ruling is confined to this parenthetical for the same reason Ruling S was: restating every reachability claim in this document against a named commit was considered when Ruling S was made and declined, and that declining stands. `99fcb84` *was the first citation this ledger made outside the crew repo, and the split is structural rather than incidental — the* `T-INT-01` *gate check lives in the crew repo while the vendored tree it asserts over lives in the UE project, so the evidence chain spans both and each commit is cited against the repo it is in. The* ***file*** *paths resolve too, which they previously did not: four evidence cells carried* ***five*** *bare citations between them —* `Combat.cpp` *three times and* `test_combat.cpp` *twice — and neither bare name has ever existed in this repository at any commit. Those five citations resolve to two tracked files,* `cpp_reference/Combat.good.cpp` *and* `cpp_reference/test_combat.cpp`*, which the cells now name in full; every path this table cites was probed at the commit its own row names. The* `build/` *directory is not tracked at all. That correction is what the "independently checkable" claim required, not a cosmetic one.)* The remaining rows fill in the same format — commit + passing test IDs — as each system clears the gate.

Rules that keep the ledger honest: - **Scope = the game's own source only.** The UE engine, the MCP/AllToolsets plugins, and the bought reference template (§4.3, not shipped) are excluded — they aren't the artifact we built, so they're not in the denominator. - **Author = human** for anything a human hand-wrote or substantially edited (UI polish, glue). Human work counts as human, so the ledger under-claims rather than inflates. - **Agent-verified = both bars:** (a) covered by agent-authored automated tests that pass in the headless suite (§4.1) **and** (b) signed off at the human review gate (§3). A system is "agent-verified" only if it clears both.

Because the game is deterministic rules + data, this ledger is *checkable*: the test suite is re-runnable and each system's provenance traces to its commits. It is supporting evidence of the method — the game itself is what ships and is judged.

---

4. Technical Strategy

4.1 Architecture

- **Headless rules module (C++)** — all of §2 mechanics, no engine deps. Enables sub-second agent test/tune loops and deterministic self-play. Combat damage is a deterministic function of attacker/defender stats, terrain defense, and the attacker's HP ratio; reachability and routing use Dijkstra/A* over variable terrain cost; any RNG is seeded so tests and self-play stay reproducible.
- **Presentation layer** — Actors + UMG read from the headless state; never own rules.
- **Data-driven** — `DataTable`/`DataAsset` for units and terrain; agent-generated, human-reviewed.
- **Test suite** — Unreal Automation tests (or a plain C++ harness on the headless module) covering hex math, pathfinding, combat, capture, and win detection.
- **Balance sim harness** — headless AI-vs-AI self-play, N games → win-rate and turn-length distributions → tuning input.
- **Input:** Enhanced Input. **Save/load:** minimal single-slot for the prototype.

4.2 Engine & tooling

- **Unreal Engine 5.8** (last planned UE5 release; stable base).
- **Unreal MCP plugin** (ships in 5.8, Experimental) + **AllToolsets** — editor operations for the Content/UI agent roles. Serial-on-game-thread; off the critical path.
- **Custom MCP toolset** — `place_terrain`, `place_unit`, `set_objective`, `validate_scenario`, authored via `ToolsetDefinition` (Python or C++) so the Content agent builds/edits scenarios in-editor.
- **Reference implementation** — cross-check the agent's grid/pathfinding against a mature template (e.g. *Advanced Turn Based Tile Toolkit*). **License caveat: verify AI-usage terms before an agent reads template code.**

4.3 Build approach

Author the **core in C++ with the agent** (the on-thesis path — hex math, pathfinding, and combat are where agents are strongest and least likely to hallucinate). Keep a bought template installed **only as a reference to check output against**, not as the shipped codebase, so the "agents built the game" claim holds.

4.4 Milestones (7 weeks)

Wk	Goal
	Headless C++ core — §4.11 **rows 1–3**

Wk	Goal
1	(grid and hex math, the §4.8 tables, movement and pathfinding) on top of the already-verified Combat/Repair at `5ffa8d6` — **+ test suite.** Playable via debug commands. Win and tiebreak detection is **row 5**, bundled with the turn loop and landing wk 3 (Q23); this cell used to name it here, one week ahead of the Capture & Fame row it depends on. **UI-scaffolder agent starts UMG widget skeletons in parallel** — the whole game is played through UI, so it can't wait until wk 2.
2	Engine presentation + UI wiring (select/move/attack) onto the wk-1 skeletons, plus the one scenario loading, validating and rendering, **and the §4.10 save/replay format + headless replayer** (Q20, ruled). The format is a *test instrument*, not a feature: T-INT-02's input file is a save, so the week-2 parity gate cannot run without it — and neither can the week-4 self-play logs (T-SAVE-07). **Week 2's command set is exactly** `{Move, Attack}` (§4.9): `Capture`, `Build` and `EndTurn` arrive with §4.11 rows 4–5 in wk 3, so a wk-2 log's entries are all `{turn 1, one side}`. The wk-2 parity and replay gates therefore **run over that subset and re-open when it widens** — T-INT-01/04 and T-SAVE-04 close here, the rest do not (§4.11 rows 9–10) — and **T-INT-04 and T-INT-01 are green at** `e19605e`, over the vendored tree at `a13626f`, and **T-SAVE-04 at** `737f666`, on row 10's part (a) (§3) — all three ahead of this cell rather than behind it, since none of them runs on a command log or needs a rules row: the vendoring the two `T-INT` IDs assert over has landed, and `T-SAVE-04` refuses on the header alone and never applies a command. That is a real gate rather than a placeholder: T-INT-02 replays a *fixed* log, so its divergence surface is the state-mutating math only, and every field of the §4.10 hash is an integer except the transients inside `resolveDamage` — which `Attack` already reaches. What it cannot cover is code that does not exist yet, which is exactly why rows 4–5 re-open it. No save button, slot, or `USaveGame` wrapper here; those are week 5. **Move + attack only** — no capture, no production, no AI opponent.
	Capture + production (§4.11 rows 4–5), then the baseline objective-seeker AI (row 6) → **working vertical slice.** Rows 4–5 add precisely the three commands week 2 lacked, so the wk-2 gates re-run over the complete set, and **this week is where the schedule places their closure**: T-INT-02/03/05 and T-SAVE-01/03/05/06 on rows 4–5, and T-SAVE-02 on row 6, whose determinism gate (T-AI-06) it composes. **T-SAVE-01, T-SAVE-02, T-SAVE-03 and T-SAVE-05 are green at** `ec15be6` (§3), where row 10's part (b) ran over the complete command set rows 4–5 supply. `T-SAVE-07` **is green at** `1ee890e` (§3), on row 10's part (c), which emitted a self-play match in the §4.10 format — ahead of the wk-4 cell that scheduled it rather than behind it. `T-SAVE-06` did not close here: among what it waits on is a vendored replayer, it being asserted jointly with `T-INT-02` (§4.9). `T-INT-02` did not

Wk	Goal
3	close here either: among what it waits on is that replayer, `Replay` being ruled out of vendoring until a bridge consumer exists (§4.9). The in-editor Automation harness both of them also lacked landed at `fed8ae9` in the Stratocracy UE project repo (§3), in no week this table names. Nor did `T-INT-03`: among what it waits on is the bridge's command surface, which is unbuilt (§4.9). Nor did `T-INT-05`: among what it waits on are the real Stratocracy widgets it asserts against, measured absent at `a13626f` (§4.9). **T-UI-05 was scheduled here too** (ruled 2026-08-04), and not as one of those re-runs: it is headless, and it asserts over the per-factory build record, which `Build` reaches and which rows 4–5 supply — the piece landing in the week the thing that consumes it runs, the principle stated below this table. It is **green at `41a1452`** (§3), so it is ahead of this cell rather than behind it; the rows that supply what it asserts over had already landed. AI second pass (utility + threat map) and the custom MCP scenario toolset follow only if the slice lands early; both are off the critical path (§4.2, §4.11).
4	Self-play balance sims and tuning — every match emitted in the wk-2 §4.10 format, so **T-SAVE-07 (harness compatibility) closed ahead of this cell**, at `1ee890e` on row 10's part (c) (§3), rather than here and certainly not in wk 5 — the reason it was placed here holds either way: one format, no dialect drift between a save and a balance log. What this week still owes is the sims and the tuning themselves; scenario polish (additional scenarios only as stretch).
5	UI polish, feedback/juice, **the save-slot UI and its slot I/O** — the §4.10 format and headless replayer landed in wk 2 (Q20), so what remains here is the player-facing surface only: the single `slot0` file, the `USaveGame` wrapper, and whatever §2.11 decides about overwrite-confirm (§4.10 records that surface as unowned); onboarding.
6	Playtest, bug fix, balance lock. LLM-commander stretch **only if on track.**
7	Final polish, **package/build the shippable game** (the deliverable), and write up the agent pipeline + provenance ledger (supporting evidence).

On weeks 2–3 (Q23, ruled). The vertical slice sits in week 3, not week 2, because §4.11's critical path is `1 → 3 → 4 → 5 → 6/8`: the baseline AI (row 6) needs the turn loop (row 5), which needs Capture & Fame (row 4). A week-2 slice *with* an AI opponent would have required building rows 4–6 a week before §2.10 says capture and production land — the document previously asserted both and could not have delivered either. Week 2 therefore delivers move + attack against a static board, which is the honest subset of the slice that rows 1–3 actually support, and §2.10's *"these land wk 3, not wk 1–2"* now describes the schedule rather than contradicting it. **The save/replay half of the same question (Q20) is now ruled — and split rather than moved.** The §4.10 *format and headless replayer* go to **week 2**, where T-INT-02 needs a save file as its input, and are therefore in hand by **week 4** for the self-play logs T-SAVE-07 validates; only the **save-slot UI and slot I/O** stay in week 5. The two rulings run on one principle in opposite directions: each piece lands in the week the thing that consumes it runs, so a milestone that outran its dependencies moved *later* (Q23) and an instrument its own gates outran moved *earlier* (Q20). Stated once, so the table stops drifting: **a format is a test instrument; slot I/O is a feature.** One consequence of that principle needed stating too, because the first application of it promised a week-2 gate this document's own build order could not support: **an integration or replay gate is scoped to the command set of the log it runs on.** It *runs* as soon as that log can be produced and it *closes* only when the log carries

every §4.9 command — so week 2's `{Move, Attack}` pass is a run, not a closure, and §4.11 rows 9–10 now state those two dependency sets separately instead of one. The seam that repairs: row 9 required "rows 1–5 built" while Q23 had just limited week 2 to rows 1–3, so §4.4 and §4.11 described two schedules for the third time. Neither week number moved to fix it; the gate's scope was named. And because a gate that runs green over a subset is not a verified system, **no §3 ledger row flips on a partial pass** (Q29).

The in-editor Automation harness landed off this calendar (ruled 2026-08-06). It landed at `fed8ae9` in the Stratocracy UE project repo (§3), and no cell above gives it a week — wk 3 records that landing beside the two IDs that had waited on it, which is a record rather than a goal cell — and §4.9 records the landing where it recorded the blocker. Both sections declined it in the open, so the absence read as a decision and not as an oversight. What stood between it and a cell held at the landing: the harness by itself closes no acceptance ID (§4.9), and the landing closed `T-DATA-05` because it carried that ID's own subjects with it — the imported §4.8 DataTables and the `UENUM` mirror of the unit type — while the subjects the other editor-pass IDs assert against are unbuilt (§4.9). The widget work is scheduled here — wk 1's UMG widget skeletons, wk 2's select/move/attack wiring onto them. The wiring that submits commands is scheduled here too, in wk 2's `{Move, Attack}` and in the `Capture`, `Build` and `EndTurn` the wk-2 cell places in wk 3. An in-editor import step for the §4.8 tables has no cell; wk 1 names those tables inside the headless core. A `UENUM` mirror of the unit type is named in no cell. And a **vendored** replayer is held out rather than merely uncelled: §4.9 rules `Replay` out of vendoring until a bridge consumer exists, a condition no cell here dates — the *headless* replayer wk 2 schedules is a different artifact. None of those subjects exists at `a13626f` (§4.9), which is a claim about what is built and not about where this table puts it. The harness took no cell and landed anyway, and so did the in-editor import step and the `UENUM` mirror named above, at `fed8ae9` (§3). The principle stated above — the week the thing that consumes it runs — now governs what is still uncelled beside them: the vendored replayer `T-INT-02` waits on, the command surface `T-INT-03` waits on, and the real Stratocracy widgets `T-INT-05`, `T-UI-03` and `T-UI-04` wait on (§4.9).

4.5 Risks & mitigations

Risk	Mitigation
Scope creep (16 scenarios is a commercial game)	Hard MVP line; 1 scenario + flag win is complete
UI underestimated	Treat as core; start week 2
AI rabbit hole	Ship the heuristic; LLM behind a toggle
MCP plugin experimental/flaky	Off the critical path; Perforce checkpoints; manual fallback
Pacing (the original's flaw)	Small maps, decisive damage, turn cap + tiebreak
Agent code quality	Test-first; human review gates; headless module keeps loops fast

Specification outruns the build — **71** written acceptance IDs at this revision (§4.7–§4.11) against **10** verified ledger rows (§3). **Reduced and re-scoped at 2026-08-04, not retired:** two new IDs have been written since `c224825`. `T-TURN-10`, minted into Spec Stub 5 for the per-turn build limit Q8(b) ruled, is **green at `6ccd40b`**, the rebuild at which row 5's widened acceptance set closes in full (§3). `T-UI-05`, minted 2026-08-04 into Spec Stub 8 for the snapshot-fidelity check row 8's landing left ungated, is **green at `41a1452`**, the commit at which the snapshot additions ruled beside it were built and it was implemented — it widened this gap by one when it was written and closed it again there, and those two movements are counted separately rather than netted, as `T-TURN-10`'s were. Row 6's GATE-AI-SMOKE is acceptance that deliberately mints none, so it closes a check without moving this count; the same holds for row 8's GATE-CAP-PARTIAL, and for the presentation block and the snapshot fields ruled beside T-UI-05 on the same day, which mint no ID at all. **62** of the 71 are green: **18** at `c224825`, where rows 1 and 3 closed their full acceptance sets and row 2's headless half passed, **9** at

Risk	Mitigation
`647d4df`, where T-SAVE-01..09 closed row 4, **9** at `ad77b13`, where T-TURN-01..09 closed, **6** at `d8284f1`, where T-AI-01..06 closed row 6, **7** at `9086d6a`, where T-SCN-01..07 closed without closing row 7, **1** at `6ccd40b`, where T-TURN-10 closed and completed row 5's acceptance set, **2** at `7c36303`, where T-UI-01 and T-UI-02 closed without closing row 8, and **1** at `41a1452`, where T-UI-05 closed without closing it either, and, under **the closure convention this document states once, here: an ID whose written text widens closes at the commit at which the widened text is met**, the widened T-INT-01 first closed at `d837fc8` over the vendored tree at `9dec48c` rather than at `b23823f`, where T-INT-04, whose text did not change, first passed over the vendored tree at `99fcb84`; both greens now stand at `e19605e` and are counted there below (§3) — and **1** at `737f666`, where `T-SAVE-04` closed on row 10's part (a) without closing row 10's acceptance set, that row having no ledger row to close (§3), and **4** at `ec15be6`, where `T-SAVE-01`, `T-SAVE-02`, `T-SAVE-03` and `T-SAVE-05` closed on row 10's part (b), run over the command set part (c) requires, without closing that row's acceptance set either, and **1** at `1ee890e`, where `T-SAVE-07` closed on row 10's part (c) over a self-play log written in this format, without closing that row's acceptance set either, and **2** at `e19605e`, where `T-INT-01` and `T-INT-04` re-date over the vendored tree at `a13626f` without closing row 9's acceptance set either — `T-INT-01` because it asserts identity at `rulesCommit`, which moved there, and `T-INT-04` because it compiles the vendored copy, whose `Turn.h` bytes changed — neither on a text change, so both are closure movements rather than widenings (§3), and **1** at `fed8ae9` in the Stratocracy UE project repo, where `T-DATA-05` closed in the editor pass and completed row 2's acceptance set across the repo pair whose crew half is `b1ea992` (Q34; §3) — so every row on the critical path has now landed, row 8 last and on a partial pass, **per acceptance ID as well as per row**. **9 IDs remain unclosed**: T-SCN-08, T-SCN-09 and T-SCN-11, which are written, unblocked and asserting, but ran only part of their fixture sets, and which leave row 7 unflipped; T-UI-03 and T-UI-04, which are written, unblocked and asserting, and for which no in-editor pass exists at either of row 8's two commits — they are the IDs row 8 still lacks, and among what its flip waits on are the real Stratocracy widgets `T-UI-03` and `T-UI-04` assert against, which are measured absent at `a13626f` (§4.9), the in-editor Automation pass those two also lacked having landed at `fed8ae9` (§3); the **3** left in row 9 — T-INT-02, T-INT-03 and T-INT-05, which are scheduled into the editor pass (§4.9), `T-INT-02` waiting on a vendored replayer, `T-INT-03` on the bridge's command surface and `T-INT-05` on those same widgets — its other two having closed and both now credited at `e19605e` — T-INT-01 because it asserts identity at `rulesCommit`, which moved there, and T-INT-04 because it compiles the vendored copy, whose	The † **cut line** (§4.7 head; members marked in §4.11's build-order table, which is authoritative for which side an ID is on) separates the IDs the MVP line above needs from the correctness infrastructure that stands down if the calendar takes it — so a slip drops named suites rather than silently thinning every suite. And the discipline Q20 and Q23 already applied holds for the rest of the table — *each piece lands in the week the thing that consumes it runs* (§4.4), and a gate that runs green over a subset does not flip its ledger row (Q29) — so a slip in rows 7–8 moves everything downstream of them rather than being absorbed by calling a row done on a partial pass. Row 2 was that clause's worked example while its editor half was outstanding, and it has since flipped (§3). Row 7 is the clause's live example: three of its written IDs ran only part of their fixture sets, and its ledger row stands unflipped

Risk	Mitigation
`Turn.h` bytes changed), T-INT-04 first passed at `b23823f`, and T-INT-01 first closed at `d837fc8` on the widening of its own text (§3); and the **1** left in row 10, whose parts (a), (b) and (c) have all since landed — `T-SAVE-04` closed at `737f666`, `T-SAVE-01`, `T-SAVE-02`, `T-SAVE-03` and `T-SAVE-05` at `ec15be6`, part (b) having run over the command set part (c) requires, and `T-SAVE-07` at `1ee890e` over a self-play log written in the §4.10 format — leaving `T-SAVE-06`, among what it waits on being a vendored replayer — it is asserted jointly with `T-INT-02`, whose replay runs **in-engine**, and `Replay` is ruled out of vendoring until a bridge consumer exists (§3)	

4.6 Token budget

AI-agent work is the project's main variable cost, so it is budgeted explicitly. Two surfaces consume tokens: **development-time authoring** (agents writing, testing, and tuning the game across the 7 weeks) and the **runtime LLM commander** (a stretch feature, billed only if it ships). Rates below are Anthropic's official published API prices, **verified against platform.claude.com on 23 Jul 2026** — Sonnet 5 introductory ($2/$10 per M in/out, $0.20/M cache read), Opus 4.8 ($5/$25, $0.50/M cache read), Haiku 4.5 ($1/$5, $0.10/M cache read). *Note: the Sonnet 5 introductory rate holds **through 31 Aug 2026**, then reverts to the $3/$15 standard rate; the 7-week jam (Jul–late Aug 2026) falls entirely inside the introductory window, so these figures apply for the whole project.* As a conservative buffer, estimates carry a **~30% token-overhead margin** on top of raw text length (accounting for tokenizer overhead, tool-call framing, and re-reads). **Prompt caching** (cache reads billed at 0.1× the input rate) is assumed for the stable spec/rules context the agents re-read on every task.

A. Development-time authoring — the bulk. Workhorse model **Claude Sonnet 5** ($2 / $10 per M input/output, $0.20/M cache read), with occasional **Opus 4.8** escalation ($5 / $25 per M) for hard reasoning. The unit of work is one substantial agent task — author or iterate a system together with its tests — spanning many tool calls and file reads.

Line item	Estimate
Tokens per substantial task	~300k input (≈⅔ from cache: ~100k fresh + ~200k cache read) + ~45k output = **345k**. The ~30% overhead margin above is already inside these figures and is not applied again below
Cost per task — Sonnet 5	100k × $2/M + 200k × $0.20/M + 45k × $10/M = $0.20 + $0.04 + $0.45 = **$0.69**
Cost per task — Opus 4.8	100k × $5/M + 200k × $0.50/M + 45k × $25/M = $0.50 + $0.10 + $1.125 = **$1.725** (quoted to the cent as $1.73; **$1.725 is the exact figure**, and every escalation delta in this section is computed from it, not from the rounded display)
Task volume	5 agent roles × ~6 tasks/wk × 7 wk ≈ **210 tasks**
Sonnet-only base	210 × $0.69 ≈ **$145** · 210 × 345k ≈ **72M tokens**
Opus escalation — a line, not a multiplier	~15% of tasks — 15% of 210 = 31.5, i.e. **32** to the nearest whole task — run on Opus *instead of* Sonnet, same tokens at a higher rate. The delta is taken **unrounded**: $1.725 − $0.690 = **$1.035/task**. So 32 × $1.035 = $33.12 ≈ **+$33**. $1.035 is the only escalation delta used anywhere in §4.6
Dev-time subtotal	**≈ 72M tokens · ≈ $178** — unrounded, $144.90 + $33.12 = **$178.02**; rounding first ($145 + $33) reaches the same $178, so the printed figure is stable either way. Downstream lines scale the **unrounded** per-task costs, never this rounded total

The escalation is priced once. It is a *delta* between two models on the same task, so it

belongs inside the subtotal and there is no further uplift after it — an earlier draft folded it into the subtotal silently and then applied it a second time to reach $225, which is why three figures in one table disagreed. Every number above is re-derivable from the two rate lines and the task count, which is the property that made the error visible. The escalation assumes Opus **substitutes** for Sonnet on those tasks; if it instead runs *in addition* (a re-run rather than a substitution) the line is 32 × $1.725 = $55.20 ≈ **+$55** and the subtotal $144.90 + $55.20 = $200.10 ≈ **$200**.

B. Runtime LLM commander — stretch, per match. Model **Claude Haiku 4.5** ($1 / $5 per M) for turn latency; Sonnet 5 as a quality upgrade. Each AI turn serialises the board and its legal moves, asks the model to rank one, then validates and applies it (§2.9).

Line item	Estimate
Tokens per AI turn	~2.5k fresh input + ~1.5k cached rules + ~0.4k output = **4.4k**
Cost per AI turn	2.5k × $1/M + 1.5k × $0.10/M + 0.4k × $5/M ≈ **$0.0047** (Haiku 4.5)
Per match (~20 AI turns)	88k tokens · ≈ **$0.09**
200 self-play + playtest matches	≈ **17.6M tokens** · ≈ **$19** (Haiku 4.5)
Same volume on Sonnet 5	≈ **$37** at the introductory $2/$10 rate this project runs inside · ≈ **$56** at the $3/$15 standard rate that resumes 1 Sep 2026

Headline. Development authoring dominates. Dev-time alone is **≈ 72M tokens · ≈ $178**; the stretch runtime commander adds **≈ 18M tokens** and **$19 (Haiku 4.5)** to **$37 (Sonnet 5)**. The whole jam therefore lands at **≈ 90M tokens** and **≈ $178 without the commander, ≈ $197–$215 with it**. Cost scales linearly with task volume, and the overrun case is stated rather than buried in a wide band. At **1.5× task volume** — 315 tasks rather than 210, of which 15% escalate to Opus (47.25, i.e. **47** to the nearest whole task) — the dev-time line is 315 × $0.69 + 47 × $1.035 = $217.35 + $48.645 = **$265.995**, i.e. **≈ $266**. The overrun is in *agent task volume*; it does not scale the 200-match playtest plan, so part B carries across unchanged and the all-in overrun is **$266 + $19 ≈ $285** with the Haiku 4.5 commander and **$266 + $37 ≈ $303** with Sonnet 5. **$303 is the ceiling.** Each of those is a derivation rather than a round number — as is every figure in the two tables above. (An earlier draft printed **$267** here. That was 1.5 × the *rounded* $178 subtotal, not a re-derivation from the per-task lines, and it sat $1.005 above what its own stated inputs produced ($267 − $265.995; the often-quoted $1.24 came from the retired $1.03 delta) — the second arithmetic fault this table has surfaced by being fully re-derivable, and the reason the escalation delta is now quoted unrounded at $1.035 throughout.)

4.7 Pending-system gate plan — the eight ledger rows that read `*pending*` at 2026-08-01

Each stub below follows the proven Combat-spec shape (§3): Inputs, Formula or state transition, Invariants (one per assertable rule), Determinism, Acceptance test IDs. All rules code is **headless** — `namespace strat`, pure C++17, zero engine dependencies, compiled by the same `python run.py` gate that certified Combat (§3 ledger). That gate detects **one** compiler per run — the first of `g++`, `clang++`, `c++` or `cl` found on PATH — so a green run is green under whichever one it found, not under all four. **"Pure C++17" constrains the sources; it does not name a setting they are compiled under.** That standalone gate at `031ee20` compiles at `-std=c++17` for GCC/Clang and `/std:c++17` for MSVC, while the UBT build that produced `UnrealEditor-StratRules.dll` compiled the same certified bytes at `/std:c++20` — the value carried by every compiler response file that build left beside a vendored `.good.cpp`. That divergence is what §4.9 exists to track. Where a stub needs a rule the GDD does not state, the gate is parameterized on a numbered open question — the Director rules, the gate then pins the ruling. The register below is the single place their extent is stated; nothing else in this document names a range of them, because every range named elsewhere has gone stale within a revision.

The cut line, read before adding a suite. Every acceptance ID in §4.7–§4.11 is either unmarked — the §4.5 MVP line needs it and no human read substitutes for it — or †, correctness infrastructure that does not *close* if the calendar takes it, because its fixtures live on §2.13.7's stretch maps or its only unique coverage is an in-editor Automation pass. The † members are marked in §4.11's build-order table and nowhere else, so that table is authoritative for the split and no count of it is stated here; what remains fixed is the rule that no rules-correctness invariant on the critical path §4.11 states is ever in it — those suites are the game, not the evidence about it.

Shared conventions (Director-owned contract):

- **Coordinates:** axial `(q, r)`, pointy-top hexes (§2.2). Distance is the standard axial hex metric `(|dq| + |dr| + |dq + dr|) / 2` — pure integer math. **Two conventions coexist deliberately, and the conversion between them is part of this contract:**

§2.13's authored maps use odd-r offset `(col, row)`, which is what a human reads off an ASCII grid, while the module uses axial because the distance metric above is only clean in axial. The scenario loader (Stub 7) converts odd-r → axial on import — `q = col - (row - (row & 1)) / 2`, `r = row` — and that conversion is itself gated (T-SCN-05). Neither convention leaks into the other's layer: no authored file stores axial, and no module code stores `(col, row)`.

- **Canonical hex order** (the subject of T-HEX-07): a total order over hexes — **ascending `r`, then ascending `q`**. Used everywhere enumeration order could leak into behavior or bytes: reachable-set enumeration, movement/AI tie-breaks (T-MOVE-04, T-AI-06), scenario serialization and hashing (Stub 7), and the §4.10 canonical state hash.
- **Numbering:** stubs 1–8 below are also the build-order row numbers (Build order table).

```
SPEC STUB 1: Hex grid & math               (Director → Systems Engineer)
Inputs:  map dimensions (Q1); axial coords (q, r).
Functions: neighbors(hex) — the six adjacent hexes in a fixed, documented
         enumeration order, out-of-bounds candidates filtered; distance(a,
b);
         inBounds(hex).
Invariants:
  T-HEX-01  every in-bounds hex has exactly six neighbor candidates in the
fixed
            order; filtering removes only out-of-bounds hexes (§2.2 "six
equal
            neighbours")
  T-HEX-02  distance is a metric: d(a,a)=0; d(a,b)=d(b,a); triangle
inequality
  T-HEX-03  d(a,b)=1  ⇔  b ∈ neighbors(a)
  T-HEX-04  direction fairness: each of the six unit steps has distance
exactly
            1 — no direction is cheap the way diagonals are on squares
(§2.2)
  T-HEX-05  inBounds agrees with the Q1 dimensions; every hex reference
the
            engine or a scenario hands the module is bounds-checked, never
trusted
  T-HEX-06  single distance definition: combat's range checks (T-COMBAT-
06..08 @
            5ffa8d6) consume THIS distance function — Artillery at
distance 1
            cannot counter, per the verified module, with no second metric
to drift
  T-HEX-07  canonical order + determinism: sorting any hex set by (r asc,
q asc)
            is total, stable, and platform-independent; neighbors()
enumeration
            order is fixed across runs and compilers
Determinism: pure integer functions; no state.
Acceptance: T-HEX-01..07.

SPEC STUB 2: Data tables (units/terrain)   (Director → Systems Engineer)
Defined in full in §4.8 (schemas, both-side structs, parity gate).
Invariants
T-DATA-01..06 are specified there once — this stub defers to §4.8 rather
than
duplicate a contract that must never fork.
```

```
SPEC STUB 3: Movement & pathfinding        (Director → Systems Engineer)
Inputs:  unit {move, moveClass}; terrain move costs (§2.3 via Stub 2);
current
         occupancy; start hex.
Transition: reachable set + cheapest paths via Dijkstra over terrain cost
(§4.1);
         executing a move relocates the unit along the chosen path (§2.5).
Invariants:
  T-MOVE-01  reachable set is exact: a hex is in the set ⇔ its cheapest
path
             cost ≤ Move — "the real move set, not an estimate" (§2.5)
  T-MOVE-02  costs per §2.3: Plains 1, Woods 2, Mountains 3,
Town/Bridge/Factory
             1; Water impassable to land — a land path across a Water span
             exists ⇔ it crosses on Bridge hexes (§2.3, "the only hex a
land
             unit crosses Water")
  T-MOVE-03  occupancy: a move never ends on an occupied hex (§2.5, one
unit per
             hex). Pass-through of friendly-occupied hexes: parameterized
on the
             Q3 ruling; until ruled, the gate asserts the conservative
reading
             (occupied hexes block pathing entirely)
  T-MOVE-04  the executed path is minimal-cost; ties between equal-cost
paths are
             broken by canonical hex order, so the route is reproducible
  T-MOVE-05  no zones of control: moving adjacent to an enemy costs
nothing
             extra and freezes nothing (§2.5 — ZOC is cut)
  T-MOVE-06  determinism: same state → identical reachable set and
identical path
  (T-MOVE-07 reserved: Recon's "ignores some terrain cost" (§2.4) —
blocked on
   the Q2 movement-class ruling; no gate is written until the rule
exists.)
Determinism: pure; all tie-breaks canonical.
Acceptance: T-MOVE-01..06 (07 reserved on Q2).
```

```
SPEC STUB 4: Capture & Fame economy        (Director → Systems Engineer)
Inputs:  game state; commands Build{factoryHex, unitId}, Capture{unit};
the §2.7
         income/award values; §2.4 costs via Stub 2.
Transition: income accrual; build-and-spawn; capture progress; kill
awards.
Invariants:
  T-FAME-01  single pool (§2.7): income, kill awards, and spending all
mutate one
             per-side fameTotal; combat awards ALSO accrue a separate
fameCombat
             counter (the §2.8 tiebreak criterion-1 sort key); passive
income
             never touches fameCombat
  T-FAME-02  income: each held factory pays +100/turn, each held town
+25/turn
             (§2.7); accrues at the START of the owner's turn and is
spendable
             in that same turn's economy phase, with NO accrual on turn 1
—
             a side's turn-1 buying power is its STARTING FAME alone — 200
for
             both sides at Normal; §2.9's handicap moves the PLAYER's
opening
             Fame only, to 350 on Easy and 100 on Hard, while the AI opens
on
             200 at every tier — so the gate asserts each side's
configured
             value and never a literal 200 (Q8, ruled)
  T-FAME-03  build: deducts the exact §2.4 cost (Infantry 100, Recon 150,
             Artillery 200, Tank 300); refused if unaffordable; fameTotal
is
             never negative
  T-FAME-04  spawn: on the factory hex if free, else an adjacent free hex,
else
             the build waits (§2.7). A waiting build HOLDS that factory's
slot
             until it spawns; Fame is committed at queue time, never at
spawn
             time, and is not refundable (Q8, ruled). The PER-TURN half of
             §2.7's build limit is deliberately NOT asserted here: this
module
             is handed the turn number rather than owning it (§3), so it
is
             gated in Stub 5 as T-TURN-10
  T-FAME-05  capture: Infantry only (§2.7, §2.4); completes after N turns
of
             holding (N = 1 on the shipped scenario, per-scenario data);
             progress is tile-held and RESETS TO ZERO when the capturing
             Infantry leaves the hex or dies, and never transfers to
another
             unit (Q4, ruled)
  T-FAME-06  a captured objective's income flips to the new owner (§2.7)
  T-FAME-07  kill awards: exactly half the victim's §2.4 cost — Infantry
50,
             Recon 75, Artillery 100, Tank 150 (Q5, ruled). A flag kill
pays a
             flat 500 and ends the match; the flag award REPLACES the
ordinary
             kill award rather than stacking, so a flag Tank pays 500, not
650
             (Q5). No undamaged-strike bonus exists — cut, not priced (Q6,
             ruled) — so the gate asserts its ABSENCE from every award
  T-FAME-08  no Fame cap: fameTotal is unbounded; deployment is throttled
by
             board space only (§2.7)
  T-FAME-09  determinism: same state + command → identical Fame deltas and
state
Determinism: pure state transitions; no RNG anywhere in the economy.
Acceptance: T-FAME-01..09.

SPEC STUB 5: Turn loop & win / tiebreak    (Director → Systems Engineer)
Inputs:  game state; per-unit move and act flags — TWO flags per unit, not
one
         (T-TURN-01); the per-factory record of builds taken this turn
         (T-TURN-10); the turn counter and cap (Q7, stored in the scenario
         file, Stub 7); commands incl. EndTurn{}.
Transition: I-GO-U-GO alternation (§2.1); win/loss/draw evaluation (§2.8);
         start-of-turn repair application (§2.7).
Invariants:
  T-TURN-01  strict alternation; each unit carries TWO INDEPENDENT flags
in
             its own turn — one for its move, one for its act — and may
             move at most once AND act at most once, IN EITHER ORDER
             (Director ruling, 2026-08-03). Moving never consumes the act,
             and acting never consumes the move: a unit that has moved is
             still a legal attacker that turn, and a unit that has
attacked
```

attacked
              may still move that turn. The per-unit sequence is §2.1's to
              state; this gate asserts the two flags and reads no ordering
              constraint into them. The owner takes its units in any order
              it chooses (§2.1). The gate asserts: (a) a move-then-attack
              by one unit COMPLETES; (b) an attack-then-move by the same
              unit COMPLETES, leaving both of that unit's flags spent
              exactly as (a) leaves them — the two orders are NOT
              state-equivalent, since the two attacks are made from
              different hexes, so the assertion is on the flags and not on
              a state match; (c) a second move, or a second act, is
              REFUSED whichever of the two the unit spent first; (d) a
              refused command changes nothing (§4.9) — it sets neither
              flag and moves no unit; and (e) BOTH flags clear at the
              start of the owner's turn — the same moment T-TURN-10's
              per-factory build allowance renews — so a unit that spent
              both last turn moves and acts again on this one
  T-TURN-02   flag death ends the match immediately — Decisive win for the
killer,
              loss for the owner (§2.8)
  T-TURN-03   territorial domination: controlling every factory on the map
at the
              start of your turn ends the match immediately, ranked
Decisive
              (§2.8; factories only, towns excluded)
  T-TURN-04   at the turn cap, the attrition tiebreak resolves in the exact
§2.8
              order: combat Fame → objectives held → surviving HP → draw
  T-TURN-05   mutual-passivity guard: both sides' fameCombat == 0 at the
cap →
              immediate draw, with NO fall-through to objectives held
(§2.8)
  T-TURN-06   criterion 2 (objectives held, X of N) is evaluated only when
both
              sides fought and their fameCombat is equal (§2.8)
  T-TURN-07   result tiers are categorical: Decisive > Marginal > Draw,
regardless
              of Fame totals — Fame is only the sort key inside criterion 1
(§2.8)
  T-TURN-08   repair fires at the start of the unit's turn exactly when the
              verified repairAmount says so (owned Town/Factory, no
adjacent
              enemy, +25% max HP floored, min 1, capped — T-REPAIR-01..07 @
              5ffa8d6); this gate asserts the turn loop calls it at the
right
              moment with the right board facts, nothing more
  T-TURN-09   determinism: the same command sequence from the same scenario
→
              identical result tier and identical state at every step
  T-TURN-10   one build per factory per turn, player and AI alike (§2.7;
Q8(b),
              ruled): a Build naming a factory that has already taken its
build
              this turn is REFUSED, and fameTotal is unchanged by the
refusal —
              Fame is committed at queue time (Q8(c)), and a refused Build
never
              queues. BOTH dispositions of the first build count against
the
              allowance: one that spawned immediately, and one that waits
and
              holds the slot (T-FAME-04). The allowance renews at the start
of
              the owner's turn. This ID lives in Stub 5 rather than Stub 4
              because the check needs the turn number, and the economy
module
              takes the turn as an argument rather than owning it (§3) —
Stub 4
              gates the half it can enforce without a turn, this gates the
half
              it cannot
Determinism: pure state machine; the §4.10 hash is taken from this state.
Acceptance: T-TURN-01..10.

```
SPEC STUB 6: Opponent AI (baseline)        (Director → Systems Engineer)
Inputs:  full game state (the AI cheats at nothing and sees only real
state);
         the §2.9 baseline routine; default buildlist (§2.9).
Transition: economy phase then unit phase, emitting ordinary commands that
the
         rules module validates like any player's (§2.9).
Invariants:
  T-AI-01  legality: every AI command passes the same validation as a
player
           command; zero rejected commands across N self-play games
  T-AI-02  economy phase: at each held factory, if a buildlist unit is
           affordable, one is built — the AI spends and replaces losses
rather
           than hoarding (§2.9)
  T-AI-03  capture behavior: an idle Infantry near an uncaptured,
undefended
           factory/town moves onto it to capture (§2.9) — objectives stay
live
           on both sides
  T-AI-04  attack preference: the enemy flag if in reach, else the best
           expected-damage target; Artillery fires from maximum standoff
(§2.9)
  T-AI-05  strictly-losing-attack guard: the AI never makes an attack in
which
           its unit dies and trades down (§2.9)
  T-AI-06  determinism: same state → same move; every scoring tie is
broken by a
           stated deterministic rule (Q9, ruled): position AND target ties
break
           by canonical hex order — for a target, the hex it occupies —
and
           build ties break by the fixed type priority Infantry > Recon >
           Artillery > Tank — ascending §2.4 COST (100/150/200/300), which
is
           NOT the order §2.4's table prints (Infantry, Tank, Artillery,
Recon)
Determinism: pure function of state; difficulty changes only starting Fame
         (§2.9), never the routine.
Acceptance: T-AI-01..06, plus a self-play smoke run: N headless AI-vs-AI
games
         all terminate at or before the cap with a valid result tier.

SPEC STUB 7: Scenario file & validator     (Director → Content/Scenario
Designer,
                                            Systems Engineer for the
loader)
Format:  one versioned JSON file per scenario; strat::loadScenario parses
and
         validates it headless. Fields:
           formatVersion   int     unknown → refuse load
           scenarioId      string  stable identifier
           scenarioHash    string  hash of the canonical serialization
(fields
                                   in this order, hexes in canonical hex
order).
                                   The order is load-bearing, so NEW
FIELDS
                                   APPEND AT THE TAIL of this list: an
                                   append-only preimage means adding a
field can
                                   never reorder an existing one (the
discipline
                                   that appended `type` last in the combat
                                   addendum, Part A). Serialization order
is not
                                   validation order — a field is validated
after
                                   whatever it references, wherever it
sits here.
           map             object  width/height (Q1) + per-hex terrain Id
                                   (row-major in canonical hex order; Ids
are
                                   §2.3 / Stub 2 row names)
           ownership       array   initial owner of each capturable hex —
a home
                                   factory per side, neutral
factories/towns
                                   unowned (§2.7)
           placements      array   {side, unitId, hex, isFlag} — starting
units;
                                   isFlag is valid only on a Tank (§2.4:
the
                                   flag is "a designated Tank")
           startingFame    object  per side; 200/200 baseline (§2.7). The
                                   difficulty handicap (§2.9) is a match-
setup
                                   parameter applied on top, not a
```

parameter applied on top, not a scenario field
turnCap int the §2.8 cap (value per Q7)
guidedOpening array one entry per side, {side, infantry, objective} — that seat's opening-capture lane (§2.13.1). `infantry` and `objective` are HEX references: the deployment hex of the seat's marked Infantry (§2.11.6 turn 1a) and the neutral Factory hex it walks to. A hex identifies the placement uniquely because T-SCN-02 already forbids two placements sharing one, so this adds no instance-ID concept to the schema. Entries serialize in the module's side enumeration order, not authoring order, so the hash is content-only.
Required on every scenario: §2.13.1 declares the reachability invariant shared by every map, and §2.11.6's turn-2 directive has no other source for the pair. The guidance layer reads this field from the loaded scenario directly. Its two halves sit in two places (ruled 2026-08-04): MARKED is the per-unit Stub-8 snapshot field `isGuidedMarked`, DECLARED DERIVED at that stub on this field; LOCKED THIS TURN is per-unit and per-turn, owned by the guidance layer, and is a member of Stub 8's presentation block rather than of the snapshot.
symmetry enum REQUIRED. `rot180` or `none`, the two values §2.13.1 declares (Q24, ruled). The enum is narrow by SCOPE, not by impossibility, and the difference is load-bearing. An odd-r rectangle has no VERTICAL mirror axis at any dimension (§2.13.1) — but on an ODD row count it does have a HORIZONTAL one, mu(c, r) = (c, H-1-r): with H odd, H-1 is even, so r and H-1-r share a parity, every row keeps its offset and the column is unchanged. 11 x 9 Ferrum Crossing carries that axis geometrically and still declares `none`, because its terrain is not drawn to it. So `mirror` is a claim the schema gives an author no way to STATE, not one the validator could never ACCEPT — two different failure modes, and Q26 ruled which one is intended: the narrow enum stands. Admitting the value could widen nothing: rot180 needs an even H and a horizontal mirror an odd one, their composition would be the vertical mirror that never exists, so at most ONE non-`none` value is well-formed on any given map. Absent or unrecognized is a hard load failure, never a default of `none` — a scenario that forgets to declare must

scenario that forgets to declare must not
                                    silently claim the weakest claim. This is
                                    not a balance or layout field: it is the
                                    AUTHORED CLAIM that T-SCN-09 verifies hex by
                                    hex, and it exists in the schema because
                                    §2.13.1 asks validate_scenario to
                                    machine-verify a declaration the file
                                    previously had no place to make. Appended at
                                    the tail per the scenarioHash policy above;
                                    it moves every scenario's hash, which costs
                                    nothing while no scenario file exists and is
                                    the point of the policy once one does.

Invariants:
  T-SCN-01 exactly one isFlag placement per side, and it is a Tank; the flag
            appears in no buildlist anywhere — "not producible" (§2.4) is a
            scenario/production fact, not a fifth unit row
  T-SCN-02 structural validity: every hex reference is in bounds (T-HEX-05);
            every terrain and unit Id resolves to a Stub-2 row; no two
            placements share a hex (§2.5)
  T-SCN-03 economy validity: each side owns exactly one home factory at start,
            and at least two neutral factories exist in contested ground (§2.7,
            "~4 factories total")
  T-SCN-04 playability: the two flags are mutually reachable by land movement
            (Stub 3 pathing, Bridge rules respected) — a scenario cannot be born
            stalemated
  T-SCN-05 coordinate conversion (Shared conventions): odd-r (col, row) → axial
            (q, r) round-trips for every in-bounds hex of the declared
            dimensions, and the loaded map's adjacency matches the authored
            grid's; no authored file stores axial, and no loaded state stores
            (col, row)
  T-SCN-06 opening-capture lane (§2.13.1): for EACH guidedOpening entry, a land
            path exists from `infantry` to `objective` costing
            <= 2 x Move of the capturing unit row — T-DATA-03's single
            CanCapture row, §2.4 Infantry Move 3, so 6 MP — and crossing NO
            Bridge hex. The ceiling is DERIVED from the loaded table, never a
            literal: a §2.4 Move change re-prices the gate instead of silently
            passing it, and the capturing row is found by CanCapture so §2.7's
            Infantry-only rule and this lane can never name different units.
            Cost counts every hex entered including the objective itself
            (Factory MoveCost 1, §2.3) — the same accounting as T-MOVE-01, so
            the validator and the reach highlight cannot price one lane two
            ways. Excluding Bridge (and Water being land-impassable, §2.3)
            confines the lane to the seat's own bank, which is what makes the
            first lesson uncontested rather than a crossing. Asserting the
            existential over the NAMED hex is deliberate. §2.13.1 states an
            EXISTENTIAL ("at least one Infantry deployment hex must have a
            land path...") and, alongside it, a NAMING requirement (the
            scenario file names that unit, so §2.11.6's turn-1a marked
            Infantry is the one already standing on the lane); quantifying
            over the named hex collapses both into a single check. Written
            the other way round — find ANY qualifying hex, then compare it to
            the named one — the validator passes a map whose qualifying lane
            belongs to a unit turn 1a never marks: the reachability rule
            satisfied to the letter while the marked Infantry stands
            somewhere else, and the guided opening's first directive walking
            the player down a lane nobody priced.

The gate asserts ARRIVAL ONLY: the turn the tile flips is
N-dependent (Q4) and is asserted nowhere here, matching §2.13.1's
"capturing by turn 2, never captured by turn 2."

T-SCN-07 opening-capture naming (structural; no pathing, so this half of the check lands with rows 1-2): exactly one guidedOpening entry per side; `infantry` is the hex of a starting placement of THAT side whose unitId is the CanCapture row and whose isFlag is false; `objective` is a Factory hex that `ownership` leaves neutral (§2.7); and the two entries name DIFFERENT objectives — §2.13.1's "the seat's own neutral," gated at its distinctness floor. Distinctness is now exactly that: a FLOOR, not the requirement. The stronger non-contention property §2.13.1 claims is RULED (Q22) and is asserted by T-SCN-11, which prices a path and therefore lands with row 3 rather than with this check. Two seats can name DIFFERENT objectives and still race each other to one of them — which is what T-SCN-11 exists to refuse and what this invariant, being structural, cannot see.

T-SCN-08 measured, not inferred: the validator COMPUTES and REPORTS each entry's lane cost as an integer, from Stub-3 pathing. The declared-symmetry flag (§2.13.1) is not an input and cannot substitute. Not because the flag is unreliable in principle — a verified rot180 IS an isometry and does imply equal cost between IMAGE lanes (§2.13.1, T-SCN-09) — but for three reasons that hold even when it is right: (i) it is an AUTHORED DECLARATION, and measurement is the only thing that catches the author declaring the wrong one, which is what happened here (the flag said mirrored; the numbers said 3 and 4; the numbers were right, §2.13.1); (ii) a `none`-declared map offers nothing to infer from at all, and `none` is what the SHIPPED map declares (§2.13.2); (iii) even a verified rot180 forces equal lane cost only if the two guidedOpening entries are themselves rho-images, which §2.13.1 does not require and T-SCN-09 does not assert (Q25) — so equal numbers on a symmetric map are a result, not a theorem. And no symmetry argument of any kind prices a lane, because cheapest path is not fewest hexes.

Fixtures:

(a) The Causeway (§2.13.6) PASSES, reporting 3 and 3. Its West lane (1,2)->(1,1)->(2,1)->(3,2) is THREE hexes costing 3 MP through the town; the TWO-hex route over the Mountain at (2,2) costs 4 (§2.3). An implementation that counts hexes instead of summing MoveCost reports 2, and nothing else in this suite catches it.

(b) Longwater March (§2.13.5), rot180 on 13 x 8, PASSES and reports 4 and 4 — COMPUTED equal, never assumed equal.

(c) A scenario whose lanes both cost 7 FAILS the T-SCN-06 ceiling, and the refusal reason CARRIES BOTH MEASURED INTEGERS: an author needs to read the measured 7 against the 6 MP CEILING, not merely "too far" (Determinism, "refuses with a reason"). This is the only "against" in this stub that is not owning-vs-opposing: it compares a MEASUREMENT to a BUDGET, and it SAYS SO at the site — the right-hand term is printed as "the 6 MP ceiling", never as a bare integer, per T-SCN-11's print convention. NAMING THE RELATION IS THE WHOLE OF THE DISAMBIGUATION. Integer order does not separate the two, because a T-SCN-11 refusal whose opposing route is cheaper prints the larger integer first as well; the earlier reading — that this was the one relation whose failing form led with the larger number — was false and is withdrawn.

The reported integers are the source of truth for §2.13.1's lane

```
            table, so a map edit that lengthens a lane surfaces as a
changed
            number rather than a still-green boolean.
  T-SCN-09  declared symmetry is VERIFIED, not trusted — the gate
§2.13.1's
            "machine-verified in axial" clause names, and which no other
            invariant asserts (T-SCN-05 supplies the axial frame and
asserts
            nothing about symmetry).
            `symmetry` == `none` asserts nothing and is always well-
formed.
            `symmetry` == `rot180` asserts, for EVERY in-bounds hex h:
              - terrain(rho(h)) == terrain(h);
              - ownership maps onto itself with the two sides EXCHANGED —
each
                home factory's image is the other side's home, each
neutral
                capturable's image is neutral (§2.7);
              - the placement set maps onto itself with sides exchanged
and
                unitId and isFlag preserved, which is what §2.13.5 and
§2.13.6
                mean by "East is the exact rho-image of West."
            SCOPE per Q25: guidedOpening is NOT bound, so a rot180 map may
name
            lanes that are not each other's image and report unequal costs
            (T-SCN-08 (iii)). Both stretch maps satisfy the stronger
reading as
            drawn, so ruling Q25 either way moves no layout.
            rho RUNS IN AXIAL, after the T-SCN-05 conversion, because no
loaded
            state holds (col, row) to rotate. On an even row count §2.13's
            authored rho(c, r) = (W-1-c, H-1-r) and the 180-degree
isometry are
            the same map, and in axial it is one parity-free affine map:
                rho(q, r) = (W - H/2 - q,  H - 1 - r)
            The EVEN ROW COUNT is a PRECONDITION, not a comparison: on odd
H
            that constant W - H/2 is a half-integer, so no hex has a hex
image
            and the file is REFUSED WITH A REASON before any comparison
runs
            (§2.13.1; Q24, ruled). On 9 x 9 the constant is 4.5 and (1,1)
            rotates to column 6.5 — one refusal, rather than the offset
index
            permutation quietly producing geometrically meaningless
            comparisons.
            Structural: no pathing, so this lands with rows 1-2 (§4.11).
  (T-SCN-10 reserved: verification of a HORIZONTAL mirror declaration on
an odd
   row count. In axial, after the T-SCN-05 conversion, that map is
       mu(q, r) = (q + r - (H-1)/2,  H-1-r)
   with the ODD row count as the precondition — (H-1)/2 is an integer
exactly
   when H is odd, mirroring rot180's even-H precondition, and the two are
   therefore mutually exclusive on one map. It is a true isometry, not an
index
   permutation: it sends (dq, dr) to (dq+dr, -dr), which permutes the
three
   terms of the axial metric and leaves the distance unchanged. RESERVED
BY
   DECISION, not blocked on an answer: Q26 is RULED — the enum stays at
   rot180 | none, so a horizontal mirror is undeclarable and there is
nothing
   for a gate to verify. Nothing is waiting. It stays reserved rather than
   deleted because admitting the value later is purely additive — one enum
   value, this mu, and this invariant — and nothing passing today would
then
   fail. Contrast T-MOVE-07 above, which IS blocked, on the unruled Q2.)
  T-SCN-11  NON-CONTENTION (Q22 ruled; unit set Q28 ruled): for EACH
            guidedOpening entry, the OPPOSING seat's cheapest land path to
            that same `objective` costs STRICTLY MORE MP than the owning
            seat's lane, as reported by T-SCN-08. The opposing route is
            minimised over EVERY CanCapture-row unit that seat deploys
            (Q28), not over that seat's `guidedOpening.infantry` alone:
                min over the opposing seat's Infantry of cost(hex,
objective)
                    >  the owning lane's cost
            PRINT CONVENTION: THE RELATION IS NAMED AT THE SITE, and
            integer order carries no information — neither which relation
            is in play nor pass versus fail, since both relations print
the
            larger integer first on their failing form. Every site says
            what it is comparing; no site relies on which integer is
bigger.
            WHAT A BARE PAIR QUANTIFIES OVER, stated because it was only
            derivable and was read the other way by a careful reader: the
```

right-hand term of a bare "X against Y" is this invariant's opposing term, so it is the MINIMUM over every CanCapture-row unit the opposing seat deploys (Q28) — a set figure, never a cost measured from one named hex. A cost measured from some other named hex is a THIRD quantity; print it with its hex and say what it is — "14 from (1,3) alone, not the set minimum" — so it cannot be mistaken for the set figure. The danger is concrete, not stylistic: the minimising unit can CHANGE under a counterfactual, so a figure taken from the shipped minimiser stops being the minimum. Asymmetry (ii)'s bullet is exactly that case — excluding the Bridges moves West's minimiser from (1,3) to (1,5), and 14 stopped being the minimum without anything on the map moving.

WHETHER THIS SHOULD HARDEN INTO A CLOSED LIST OF PERMITTED FORMS IS Q30, unruled and deliberately left so. An earlier revision attempted that codification and withdrew it: each closure outlawed prose elsewhere in the document that was correct, which is a poor trade for a convention that binds wording only. No invariant, fixture, reported integer or refusal condition depends on any of this.

This is the gate for §2.13.1's "uncontested, not merely reachable" — a promise that until this revision was a property of the drawn map rather than a checkable rule, and one the map failed the first time it was checked (fixture (b)).

IMPLEMENTATION, because the ruled reading looks more expensive than it is: the minimisation is ONE reverse Dijkstra per objective, not one path per unit. Root it at the `objective` with d(objective) = 0 and relax d(h) = min over neighbours n of (MoveCost(n) + d(n)); the result is every hex's cost TO that objective under the T-MOVE-01 accounting, so all of the opposing seat's Infantry are read off one pass and the cost is independent of how many units that seat deploys. That identity holds ONLY because Q21 priced the lane on terrain alone — under an "as deployed" reading each unit sees a different graph and the budget returns to one path per unit.

Both sides of the comparison are priced IDENTICALLY, which is what makes the inequality mean anything: Stub-3 cheapest path, terrain alone with occupancy excluded (Q21, ruled), cost counting every hex entered INCLUDING the objective (Factory MoveCost 1) — the T-MOVE-01 accounting T-SCN-06 already uses, so the validator cannot price the two routes two different ways.

EQUALITY FAILS. "Strictly longer" is the ruled comparison, and a tie is precisely the race the rule exists to forbid; a >= would have passed the one lane in the PRE-FIX set that a human would call contested — West's South lane against East's second Infantry at (9,5), kept as fixture (b). Q28 moved that Infantry to (9,1), so the set AS SHIPPED holds no such lane today: the failing case survives as a fixture, not as a live refusal, and the operator stays > for the next map that needs it.

Three asymmetries with T-SCN-06 are deliberate:

(i) NO CEILING. T-SCN-06 is a budget (<= 2 x Move). This is a comparison: the opposing route may cost anything at all, it must merely cost MORE.

(ii) BRIDGES ARE ALLOWED on the opposing route, and on the shipped map that allowance is LOAD-BEARING rather than merely permitted. T-SCN-06's Bridge-free clause is a property of the GUIDED lane — what makes the first lesson a walk rather than a crossing — not a constraint the enemy is under. Excluding Bridges would make the opposing route NON-EXISTENT on a bisected map (The Causeway, §2.13.6) and pass that map vacuously, so allowing them is the STRICTER reading and it is the one asserted. Measured PER MAP, never quantified over the set:

- Ferrum Crossing (§2.13.2) EXERCISES it, but on ONE of fixture (a)'s TWO opposing routes, not both. West's cheapest route to North (6,2) — 6 MP from (1,3) — runs over the north Bridge (5,1). TWO Bridge-free figures follow and they are DIFFERENT QUANTITIES, so each is labelled where it stands. FROM (1,3) ALONE, not a set minimum: 14 MP, (2,3)(3,4)(3,5)(4,5)(5,6)(6,6)m(6,5)(6,4)v(6,3)v

(2,3)(3,4)(3,5)(4,5)(5,6)(6,6)M(6,5)(6,4)w(6,3)w
(6,2) — around the river's southern end, then up
through the Woods ring. More than double that
hex's own 6.
MINIMISED OVER WEST'S INFANTRY — this invariant's
opposing term (Q28), and the only figure a BARE
pair may carry: 13 MP, from the OTHER West
Infantry (1,5),
(2,6)(3,6)(3,7)(4,7)(5,7)F(6,7)(7,6)T(6,5)
(6,4)w(6,3)w(6,2). Its first five hexes are West's
own guided South lane (§2.13.2): 5 MP to (5,7),
then 8 MP up the east bank. Bridge-free, West's
road to the NORTHERN objective runs through West's
own SOUTHERN one.
EXCLUDING THE BRIDGES MOVES THE MINIMISER, which
is the whole reason the two figures differ. WITH
the Bridges, West's set minimum to North is 6,
achieved by (1,3); (1,5) alone costs 7, not the
set minimum (§2.13.2). WITHOUT them, the set
minimum is 13, achieved by (1,5); (1,3) alone
costs 14, not the set minimum. THE ACHIEVING UNIT
IS NOT THE SAME UNIT under the two readings — that
is the flip, and it is why this bullet's two
figures are two DIFFERENT quantities rather than
one quantity corrected. No "against" is printed in
this comparison, deliberately: these are four
costs of ONE seat's own units, while an "against"
in this stub is the TWO-SEAT inequality a fixture
recomputes (PRINT CONVENTION; Q30).
The 1 MP is the APPROACH, not the Mountain —
(1,5) reaches (5,7) in 5 and (1,3) in 6, while the
tail from (5,7) costs 8 for both, and (1,3)'s 14
is achievable Mountain-free as well. So a figure
measured from the SHIPPED minimiser is NOT the
counterfactual minimum, and that is the trap this
bullet exists to name.
The OTHER opposing route in that fixture is
already Bridge-free and does not move: East's
cheapest route to South (5,7), 6 MP from (9,3), is
(9,4)F(8,5)(8,6)(7,7)(6,7)(5,7), which reaches
column 5 only at the objective itself, on row 7,
below the river's southern end at (5,5) — the
river spans rows 0–5 only. Excluding an edge can
only RAISE a shortest path, and that 6 MP witness
uses no excluded edge, so 6 stands under either
reading. The allowance binds ONE of this map's two
objectives, the northern one.
- The Causeway (§2.13.6) EXERCISES it too, passing
3 against 5 in both seats with the crossing
permitted; exclude Bridges and it reports "no route"
instead of 5 (fixture (c)).
- Longwater March (§2.13.5) is the ONE map with no
Bridge on any opposing route, and only because it
has no Water and no Bridge hexes at all — its
terrain distribution is Water 0, Bridge 0.

WHY THIS IS STATED AS A REASON AND NOT A PERMISSION:
a Bridge-free reading does not FAIL Ferrum Crossing
in EITHER seat, so no gate in this suite catches it.
North still passes, at 5 against 13 — a BARE pair,
so its right-hand term is the SET minimum over West's
Infantry, the (1,5) route above, and NOT the 14
measured from (1,3) alone. Strictly more than the
owning lane on either figure, and either way a number
that describes a walk around the entire river. South
still passes at 5 against 6, exactly as drawn,
because its opposing route never crossed a Bridge.
What the counterfactual changes is ONE margin of the
two: North's opposing figure goes from 6 to 13
against an unchanged owning 5, widening that margin
from 1 MP to 8 MP — a single-digit widening, and an
unremarked one, because this invariant asserts a
strict inequality and NO CEILING (asymmetry (i)).
South's margin stays at 1 MP and never depended on
the allowance at all.
THE SPLIT IS THE REASON. The allowance is what keeps
the NORTHERN opposing route honest, and neither half
of the split is gate-catchable, because no invariant
in this stub reads a MARGIN — only the strict
inequality, which both seats satisfy either way. So
the Bridge is what makes the northern margin a
margin, and this suite would go on reporting green
while that integer stopped meaning anything.

(iii) THE UNIT SET IS BROADER THAN THE LANE, per Q28 (ruled).

"The opposing seat's cheapest Infantry route" ranges over

EVERY CanCapture-row unit that seat deploys, not over
that seat's own `guidedOpening.infantry` alone. The

that seat's own `guidedOpening.infantry` alone. The
narrow reading was available, would have passed the
shipped map unchanged, and was refused: the property
guarded here is a RACE, and a race does not care which
Infantry wins it. T-SCN-06's NAMED-hex quantifier is not
a precedent against this — it names a hex because the
guided lane must be the one turn-1a actually marks,
whereas nothing is being marked on the opposing side and
the only question is who can arrive.
WHAT THE STRICT READING COST, PAID ONCE: one deployment
hex. Ferrum Crossing's East second Infantry was at (9,5),
5 MP from WEST's South objective and therefore tied with
West's own 5 MP lane; it now deploys at (9,1) (§2.13.2)
and the map passes in both seats at 5 against 6. No rule
was weakened, no terrain moved, and the tie survives as
fixture (b) rather than as an exception.
The relocation was forced, not chosen, which is the part
worth knowing before anyone edits that map: East's south
town (7,6) is 2 hexes from South (5,7), so by the
triangle inequality any hex covering that town is within
5 MP of that factory. On Ferrum Crossing a southern
town-capturer IS a racer for West's southern factory —
the conflict is geometric, not a placement slip.
Reported like T-SCN-08: a refusal carries BOTH measured integers,
owning and opposing, so an author reads "5 against 5" rather than
"contested" — and a map edit that shortens an enemy approach
surfaces as a changed number, not a still-green boolean.
Fixtures:
(a) Ferrum Crossing (§2.13.2) PASSES in BOTH seats, reporting
5 against 6 each way: West's South lane 5 from (1,5)
against East's cheapest 6 from (9,3); East's North lane 5
from (9,3) against West's cheapest 6 from (1,3) over the
north Bridge (5,1). A 1 MP margin each way — the thinnest
in the set, on the one map that declares `symmetry: none`.
It is also the fixture that catches ASYMMETRIC PRICING: an
implementation that counts the objective hex on the owning
lane but not on the opposing route reports 5 against 5 and
refuses the shipped map.
(b) THE FAILING FIXTURE IS REAL, NOT CONSTRUCTED. The same map
with East's second Infantry at its PRE-FIX hex (9,5) — one
placement changed, nothing else — must FAIL, reporting
5 against 5: (9,6),(8,7),(7,7),(6,7),(5,7), five cost-1
hexes, and the axial distance from (9,5) to (5,7) is 5, so
no cheaper route exists and no implementation detail can
make the number anything else. This deployment passed every
other invariant in this stub, T-SCN-07's distinctness floor
included; T-SCN-11 is the only check in the suite that sees
it. It also pins the QUANTIFIER and not merely the
comparison: under the Q28 reading REFUSED, (b) passes at
5 against 6, so an implementation that minimises over the
guidedOpening unit alone fails this fixture and nothing
else in the suite.
(c) The Causeway (§2.13.6) passes 3 against 5 in both seats
with the Bridge crossing PERMITTED on the opposing route.
It is the fixture for asymmetry (ii): excluding Bridges
makes both opposing routes NON-EXISTENT on a bisected map
and passes it vacuously, so an implementation that inherits
T-SCN-06's Bridge-free clause onto the opposing side

reports "no route" here instead of 5.
Determinism: pure parse + validation; any failure refuses the whole file with a
reason. scenarioHash is platform-stable by canonical ordering. The
T-SCN-06/08/11 lane costs are Stub-3 path costs and inherit its
determinism (T-MOVE-04's canonical tie-break, T-MOVE-06), so the
reported integers reproduce across runs and compilers. T-SCN-11
compares two such integers and introduces no new source of
nondeterminism.
Acceptance: T-SCN-01..09 and T-SCN-11 headless — the whole written suite.
T-SCN-11 ASSERTS from its first run: Q22 gave it the comparison and
Q28 gave it the unit set, so this stub carries no written-and-blocked
invariant. T-SCN-10 is reserved and UNWRITTEN on Q26, which is a
different state: nothing is asserted, so nothing is waiting.
T-SCN-11 ships with its three fixtures, one of which — (b), the
shipped map's own pre-fix deployment — must FAIL, so the suite
demonstrates refusal and not merely agreement with the repo.
The §4.2 validate_scenario MCP tool wraps the same checks in-editor
for the Content agent; its manual fallback is running the headless
validator on the exported file (MCP stays off the critical path,
§3 guardrails).

SPEC STUB 8: UI binding contract (Director → UI Scaffolder)
Scope: NOT layout or visual design (§2.11's lane) — this is
the contract for how every widget is fed. Widgets bind to the
VIEW-MODEL plus the §4.9 event list, and hold no rules state (§4.1).
The VIEW-MODEL is the rules-produced SNAPSHOT below plus a declared
PRESENTATION BLOCK: per-unit state that no rules field expresses,
held by owners named member by member at the block below. It is
declared here rather than improvised in a widget because T-INT-05
(§4.9) rebuilds from the view-model: state in the block satisfies
it, state in a widget does not.
Snapshot fields (read-only, produced by the rules module). Every field in the
snapshot groups that follow has one of TWO KINDS. A MIRROR — the
unmarked default — equals, unchanged, the module-side value it
names, in the authoritative `GameState`, in the §4.8 tables the
bridge loaded, or in the Stub-7 scenario file it loaded; §4.9 names
those three separately, and a mirror may draw on any of them.
A field marked DECLARED DERIVED is computed from them instead, and
its derivation is stated beside it. Within these groups those are
the only two kinds, and T-UI-05 asserts both:
per-hex {terrainId, owner}
per-unit {id, side, unitId, hex, hp, hpMax, isFlag, hasMoved,
hasActed, captureProgress, isGuidedMarked}
`isGuidedMarked` is DECLARED DERIVED: true exactly on the
placement that the Stub-7 scenario file's
`guidedOpening.infantry` names for that unit's seat —
T-SCN-02 makes a hex identify one placement — and false
on every other unit, computed by the module and never
widget-side. It is a property of the placement, not of
the unit's current hex, so it does not move when the unit
does.
`hasMoved` and `hasActed` are the TWO INDEPENDENT flags
T-TURN-01 asserts, carried into the snapshot as two
fields: one field cannot express a unit that has spent
exactly one of them. Neither is §2.11.1's DONE bit, and
no §2.11 surface reading "has not acted" binds to
either — §2.11.1 states where those bind. The DONE bit
is DERIVABLE FROM NEITHER, and from no pair of them:
Wait and RMB-in-MOVED both reach DONE without spending
the act flag (§2.11.1). It is therefore not a snapshot
field, the module produces no field for it, and it sits
in the presentation block below.
per-factory {hex, owner, hasBuiltThisTurn, buildWaiting,
spawnBlocked}
`hasBuiltThisTurn` is T-TURN-10's per-factory build
allowance and `buildWaiting` the §2.7 build that holds
the factory's slot until it spawns (T-FAME-04); both
mirror state the module already holds, and this exposes
it rather than adding it. The `hex` mirrors the

scenario
                    file's factory placement (Stub 7). `spawnBlocked` is
                    DECLARED DERIVED: true exactly when no hex at or
                    adjacent to the factory is free (board geometry,
§2.7's
                    spawn rule), computed by the module and never
                    widget-side. It and `buildWaiting` are DISTINCT, and
the
                    difference is the case §2.11.5 must display: a boxed-
in
                    factory with nothing queued has `spawnBlocked` true
and
                    `buildWaiting` false, which `buildWaiting` alone
cannot
                    express. Q31 asks whether a player may queue into a
                    boxed-in factory; `buildWaiting` is the field such a
                    ruling would bind to, and nothing here rules it —
today
                    the waiting build is an AI-only path (Q31) and no gate
                    asserts a player-queued one.
         per-side {fameTotal, fameCombat, objectivesHeld X of N,
survivingHP,
                    incomePerTurn}
                    `fameTotal` and `fameCombat` are mirrors. The other
three
                    are DECLARED DERIVED, and each derivation is stated
here
                    so that T-UI-05 clause (b) has one to recompute.
                    `objectivesHeld X of N` is §2.8 criterion 2 in that
                    criterion's own words: "the factories and captured
towns
                    a side owns at the cap, as X of N. Ownership only: a
                    capture in progress (§2.7) counts for nobody until the
                    objective flips." X is that side's count under that
text;
                    N is the same count taken over every factory and
                    capturable town the scenario supplies (§2.11.4).
Citing
                    the criterion mints no rule — the criterion text is
the
                    derivation. GATE-CAP-PARTIAL gates its one edge case,
a
                    capture in progress counting for nobody.
                    `survivingHP` is §2.8 criterion 3 in that criterion's
own
                    words: "total remaining HP of a side's units" — the
sum
                    of `hp` over that side's units in the per-unit group
                    above, and likewise no new rule.
                    `incomePerTurn` is §2.7's rate over that side's held
                    factories (+100 each) and towns (+25 each), computed
by
                    the module so that no surface sums it widget-side
                    (T-UI-03's no-arithmetic clause). It is the STANDING
                    rate, and that is what it reads on turn 1: the amount
                    those holdings will pay at the start of that side's
turn
                    2, not the 0 that Q8(a) pays on turn 1. The field is
the
                    rate the holdings carry and never the accrual of the
                    current turn.
         match   {turn, turnCap, sideToMove, resultTier or null}
Presentation block (NOT produced by the rules module; TWO members, each
with
         its owner named beside it, and neither owner a widget):
         per-unit {done}
                    OWNER: §2.11.1's selection machine, which is a state
                    machine and not a widget.
                    §2.11.1's DONE bit — this unit takes no further
command
                    this turn. Per-turn: it clears when the owner's next
turn
                    begins. It is the selection machine's only per-unit
bit.
                    It is in the view-model rather than in a widget
precisely
                    so that T-INT-05 (§4.9) can rebuild the screen from
the
                    view-model alone.
         per-unit {lockedThisTurn}
                    OWNER: the guidance layer, which is neither the rules
                    module nor a widget.
                    §2.11.6 turn 1a's `Locked this turn.` — this unit is
not
                    selectable under the guided opening while beat 1a is
                    outstanding. It clears when 1a retires, which §2.11.6-
B
                    states is when the marked Infantry's move completes.

```
                    states is when the marked infantry's move completes.
                    Ruled 2026-08-04, which is the question this stub
                    previously left undecided; the guided opening's other
                    half, marked, is the snapshot field `isGuidedMarked`
                    above and is not in this block.
Queries: reachable(unit) → the T-MOVE-01 set; forecast(attacker,
defenderHex) →
         {damage, counterDamage} computed by the verified resolveDamage /
         defenderCanCounter (5ffa8d6).
Invariants:
  T-UI-01  forecast = resolution: the forecast shown before commit is
produced
           by the same strat call that resolves the attack — identical
numbers,
           mechanically (§2.6, §2.11)
  T-UI-02  the reachable-hex highlight displays exactly the T-MOVE-01 set
— the
           UI queries the module and never recomputes movement (§2.5)
  T-UI-03  the live standings scoreboard (§2.11, §2.8) binds 1:1 to
snapshot
           fields — enemy strength destroyed, objectives held X/N,
surviving
           units/HP, turn vs cap — with no widget-side arithmetic
  T-UI-04  the production menu binds to the buildlist derived from the
four
           Stub-2 unit rows plus current fameTotal; the flag never appears
           (T-SCN-01's non-producible clause, enforced at the UI layer
too)
  T-UI-05  snapshot fidelity: the snapshot tells the truth about the state
           the module holds — the authoritative `strat::GameState`, the
§4.8
           tables and the Stub-7 scenario file it loaded (§4.9) — field by
           field, in three parts. (a) MIRRORS: every unmarked field equals
           the module-side value it names, exactly, with nothing widened,
           narrowed, rounded or reordered. (b) DECLARED DERIVED: every
field
           marked DECLARED DERIVED equals the derivation stated beside it
at
           this stub, recomputed inside the check rather than read back
out
           of the snapshot — so a wrong derivation fails here and is not
           merely reproduced. (c) NO OTHER KIND: a snapshot field that is
           neither an unmarked mirror of a named module-side value nor a
           marked field with a stated derivation FAILS this invariant,
which
           is what stops a field entering the snapshot without a contract.
           Asserted by rebuilding the snapshot after each command of a
fixed
           command sequence and comparing every field under (a), (b) and
           (c). The presentation block is NOT in this invariant's subject:
           it has no module-side counterpart and no derivation from one.
           Headless, unmarked (§4.11)
  GATE-CAP-PARTIAL
           a capture in progress contributes zero to "objectives held" for
           either side: raising a unit's captureProgress short of
completion
           leaves both sides' objectivesHeld unchanged. This is §2.8's
           T-CAP-05, which aliases onto no T-TURN- ID; it is a
differential
           read of two fields this snapshot already carries, adding no
field
           and no numbered ID (the GATE-AI-SMOKE precedent, row 6)
Determinism: widgets are pure functions of view-model + events — snapshot
         plus presentation block; asserted end-to-end by T-INT-05 (§4.9).
Acceptance: T-UI-01..02 and T-UI-05 headless (the queries and the snapshot
         builder are headless functions); T-UI-03..04 in-editor
Automation;
         GATE-CAP-PARTIAL headless, on the snapshot rather than on a
widget —
         which is why it carries no † and does not stand down if the
editor
         pass does. A marked ID may not guard a rules invariant (§4.11's †
         note), and T-CAP-05 is one. T-UI-05 was minted 2026-08-04 and is
         GREEN at `41a1452`, where the snapshot additions ruled with it —
         `isGuidedMarked`, the per-factory group and `incomePerTurn` —
were
         built and it was implemented and gated, 34/34 under clang++ and
         MSVC both. The presentation block is declared there and
         deliberately NOT FILLED: both its members have owners that are
not
         the rules module, so no rules-side code fills them, and this
         invariant's subject does not reach them.
```

Open questions (Director rulings owed). Every gap found while writing the §4.7 gates (Q1–Q10), the stage-2 additions (Q11–Q13), the rules- and scenario-side rulings folded in here (Q14–Q20), the two gaps found while gating §2.13.1's opening-capture invariant (Q21–Q22), the milestone contradiction the document knowingly carried (Q23), the two

raised by the §2.13 symmetry correction (Q24–Q25), the one raised by correcting that correction (Q26), the guided opening's one input-gating constraint (Q27), the reading the Q22 ruling exposed the moment its new invariant was measured against the shipped map (Q28), and the ledger-flip criterion exposed by scoping the week-2 parity gate to its command set (Q29), and the quantifier the T-SCN-11 print convention never stated at a print site, found when a hex-scoped figure was read as a set minimum (Q30), and the reachability seam the Q8(c) commitment ruling exposed against §2.11.5's Build-button rule (Q31), and the three §2.13.1 validation checks row 7 found gated under no `T-SCN-` ID (Q32), and the disagreement between an invariant's written text and the check that runs it, found while widening build-order row 9's integration check (Q33), and how Q29's *at one commit* reads for an acceptance set that spans two repositories, found when row 2's editor half closed in the UE project repo (Q34) — so that each question carries exactly one ID across the whole document. **This chain is a provenance record, not a statement of the register's extent: the table below is authoritative for which rows exist, and any count must be taken from it.** The distinction is not pedantry — this paragraph and the preamble beneath it are the register's only two extent-bearing sites, and registering Q31 staled this one while the preamble was updated, which is the same way every pinned range in this document has failed. The **Blocks** column does not name one kind of thing: some open rows name a gate that waits on the ruling, others name only a section, a schema field or a stub, and a few state in terms that nothing is blocked at all. Where a gate *is* named, the Director writes the rule and the gate then pins it. **Two senses of *blocked* meet here and are worth separating.** A row that states *no reading* blocks its gate **outright** — the gate is **written-and-blocked**: it can be written and parameterized on the row, as §4.7's stubs prescribe, but it cannot assert until the ruling lands, which is exactly the state T-FAME-02 and T-AI-06 were in before this revision — which is why the preamble below says none of the open rows blocks a gate *outright*, rather than that none blocks a gate. A row that states a reading can still leave a gate **reserved and unwritten** until the rule exists: Q2 is the current instance, and T-MOVE-07 is blocked on it in the sense that it cannot yet be written, not in the sense that it fails. The last column is not uniform, and the difference matters: **where a reading is stated, it is the conservative one, and it is what ships and what the gates assert** — chosen so that a later ruling loosens behavior rather than invalidating a passing gate. **Q28 was the one row where that convention did not hold**, and how it closed is worth keeping rather than deleting: its conservative reading REFUSED A SHIPPED MAP, so no reading could be stated there without either blocking *Ferrum Crossing* or quietly weakening a rule the Director had just made, and it therefore carried no assumption at all while it was open. It was then ruled the strict way and **the map was corrected instead of the rule** — one deployment hex, no terrain and no rules text (Q28; §2.13.2). The limit of the convention is now known and stated once: **where the conservative reading is not free, this register states no reading and waits. No row now states *no reading*.** The five that did — Q4's interruption semantics, Q5's stacking, Q6, Q8's three sub-questions, and Q9's target- and build-choice ties — were all ruled this revision, and the gates they blocked outright (T-FAME-02, T-FAME-04, T-FAME-05, T-FAME-07, T-AI-06 and the T-CAP- tally suite less T-CAP-05) now assert. **T-CAP-05 is excepted:** it aliases onto no `T-TURN-` ID (§2.8), and its gate home was ruled on 2026-08-02 to be Stub 8's snapshot, where it is `GATE-CAP-PARTIAL`; row 8 has since landed at `7c36303`, where that gate ran and passed under clang++ and MSVC both — on a fixture configured with `captureTurns = 2`, a state the shipped scenario cannot reach at N = 1 (Q4). It mints no numbered acceptance ID, so no §4.5 count moves on its account, and **T-CAP-05 closes there**, ruled 2026-08-04 (Q14). **Seventeen of the thirty-four rows are ruled; the other seventeen remain open but *readable*** — Q1, Q2, Q3, Q10–Q19, Q29, Q30, Q31 and Q32 — each carrying a stated reading rather than a blank, which is why none of them blocks a gate outright. Three are worth naming: Q2 leaves T-MOVE-07 reserved-but-unwritten, Q11's reading is "no undo", and Q30's is partial and deliberately not hardened. *(An earlier draft of this paragraph named only those three and called them the whole remainder — the test behind it looked for rows containing the word "unruled" rather than rows not marked RULED, and so could not see the other twelve.)* **Q33 and Q34 are registered already marked RULED**, so each enters the ruled side and the open list above is unchanged. The convention is unchanged: **where a row states no reading, its gate stays blocked until the Director answers.**

ID	Question	Blocks	Assumption in force until ruled
Q1	Map dimensions. §2.2 and §2.10 never state the prototype map's size or shape.	T-HEX-05 bounds; Stub 7's `map` field	Bounds are per-scenario data, not a global constant; *Ferrum Crossing* ships 11×9 (§2.13.2).
Q2	Movement classes. §2.3's caption says move cost is "per movement class" and §2.4 gives Recon "ignores some terrain cost," but no class set or per-class cost	§4.8's `MoveClass` column; the reserved T-MOVE-07	§2.3's single cost column applies to every land unit; no Recon discount is implemented, and no gate is written until the rule

ID	Question	Blocks	Assumption in force until ruled
	table exists.		until the rule exists.
Q3	Pass-through. §2.5 pins one-unit-per-hex for *ending* a move; it is silent on pathing *through* a friendly-occupied hex.	T-MOVE-03	Blocked — a unit may not path through any occupied hex, friendly or not.
Q4	~~Capture N and interruption.~~ **RULED (this revision).** Pin N (§2.7 says "start N=1–2"), and rule the edge cases: does progress reset if the Infantry leaves or dies mid-capture?	T-FAME-05's exactness; the §2.9 AI capture step	Ruled. N = 1 on the shipped scenario (§2.13.3's recommendation, inside §2.7's stated range), and N is per-scenario data. **Interruption: capture progress is held by the tile and resets to zero the moment the capturing Infantry leaves the hex or dies** — it never transfers to another unit. This is the conservative reading, and it is what keeps the window between arrival and flip a real risk: at N = 1 the Infantry stands on the tile at the end of one turn and the tile flips at the start of the next, so the opponent gets exactly one turn to answer, and killing or displacing the capturer now costs the attacker the whole count rather than a fraction of it. T-FAME-05 unblocked and asserting.
Q5	~~Kill-award exact values.~~ **RULED (this revision).** "~half its Fame cost" (§2.7) is not assertable. Does the flag's +500 stack with the Tank's ordinary kill award?	T-FAME-07	Ruled, and the change request is accepted as proposed: exactly half of each §2.4 cost — **Infantry 50, Recon 75, Artillery 100, Tank 150**, all integers, so no rounding rule is needed anywhere. **The flag's +500 does not stack**: it replaces the victim's ordinary kill award rather than adding to it, so a flag Tank pays **500, not 650**. The choice is nearly free at the cap — §2.8 already states the flag bonus can never appear in a capped tally, since

ID	Question	Blocks	Assumption in force until ruled
			a flag kill ends the match immediately — so it binds the live scoreboard (§2.11) and the balance logs rather than any victory condition. T-FAME-07 unblocked.
Q6	~~Undamaged-strike bonus.~~ **RULED (this revision) — cut.** "A small bonus" (§2.7) has no number, yet it feeds the tiebreak's primary key. Options: (a) price it; (b) keep it in the Fame pool but exclude it from the cap tally until priced; (c) cut it — kills already pay half-cost, and the positional triangle already rewards a clean standoff strike with tempo.	T-FAME-07; the T-CAP- tally suite; the kb economy block	Ruled **(c) — cut**, as the rules author recommended. Kills already pay half-cost and the positional triangle already rewards a clean standoff strike with tempo, so the bonus was paying twice for one thing; cutting it removes an unpriced term from the §2.8 tiebreak's **primary** sort key rather than leaving a number nobody had chosen inside it. Cheaper than (b), which would have required the document to carry two different Fame totals — one for the pool, one for the cap tally — in both the kb economy block and the T-CAP- suite. Beyond §2.7's own bullet, which did cite Q6, the cut reaches **six sites that never cited it** — four here (§2.8's tally definition, §2.11's standings row and its tooltip, the §2.11.6 one-shot toast, and the concept ledger's RPS row, whose *receipt* was that toast and is now the range-2–3 one-shot instead) and two in `kb/rules.md`. Grepping the identifier alone would have found none of them. T-FAME-07 and the T-CAP- tally suite unblocked. T-CAP-05 aliases onto no `T-TURN-` ID (§2.8), so it has a gate of its own rather than an alias: `GATE-CAP-PARTIAL`, ruled separately on 2026-08-02 into

ID	Question	Blocks	Assumption in force until ruled
Q7	~~Turn cap value.~~ **RULED (this revision).** The cap is **per-scenario data**, stored in Stub 7's `turnCap`; *Ferrum Crossing* ships **20** turns.	—	2026-08-02 into Stub 8's snapshot. Ruled. §2.8, §2.11.4 and §2.13.2 now agree, and T-TURN-04 and Stub 7 read a value rather than an example.
	~~Income timing and build limits.~~ **RULED (this revision) — all three.**		Ruled, all three. **(a) Timing:** income accrues at the **start** of the owner's turn and is spendable in that same turn's economy phase — the phase §2.9 already runs first — but there is **no accrual on turn 1**, so turn-1 buying power is the side's starting Fame alone — 200 for both sides at Normal, and for the **player** 350 / 100 once §2.9's Easy / Hard handicap applies — that handicap moves the player's side only, the AI opening on 200 at every tier — so the 200 is a baseline and not a constant. That reading was chosen because it is the one §2.13.2 was already priced on: it prices East's turn-1 Infantry as "100 of the 200 starting Fame", not of 300, so no map number moves. It does correct two sentences that said the opposite — §2.7's "both players have income from turn 1" and its "plus home-factory income from turn 1" — and rewrites the Income bullet beside them. Counting the whole of Q8 rather than its timing clause alone, the row touches **four §2.7 sites** — the factories, Income, build-and-spawn and starting-Fame bullets — and beyond §2.7 it rewrites §2.9's economy phase, T-FAME-02 and T-FAME-04, and the starting Fame and

ID	Question	Blocks	Assumption in force until ruled
Q8	When within the turn does factory/town income pay, and can it fund a build the same turn? Is there a builds-per-factory-per-turn limit (§2.9's AI implies one; the player's rule is unstated)? For a waiting build (§2.7 "the build waits"), is Fame committed at queue time or spawn time, and can it be canceled?	T-FAME-02 and T-FAME-04	starting-Fame and build lines in `ko/rules.md`. **(c) also exposed a seam it did not create:** §2.11.5 disables the Build buttons whenever a factory is boxed in, so the player can never *reach* the waiting-build state this clause prices — for the player, queue time and spawn time are the same instant, and the non-refundable commitment binds only on ordinary builds. The waiting build is therefore an **AI-only path today** (§2.9 builds without the UI). Naming that seam retracts the clause it disproves: §2.7's build-and-spawn bullet and this cell both previously called a boxed-in factory "a commitment to read before spending", which the player cannot reach, and both now say so instead. Whether the Build buttons should be enabled while boxed in — and the commitment shown before the click — is registered as **Q31**, not assumed. **(b) Limit:** one build per factory per turn, player and AI alike, matching what §2.9 already describes for the AI; a waiting build **holds that factory's slot** until it spawns. **(c) Commitment:** Fame is committed at **queue** time and is **not refundable**, so fameTotal moves once and never reverses and no cancel affordance is owed by §2.11. T-FAME-02 and T-FAME-04 unblocked; Stub 4 is no longer gated on an unruled row. Ruled, and split by

ID	Question	Blocks	Assumption in force until ruled
			axis rather than forced onto one rule. **Position and target ties break by canonical hex order** — for a target, the hex it occupies — so the convention already gated for positions extends to targets with no new state and nothing new to remember. **Build ties break by the fixed type priority Infantry > Recon > Artillery > Tank — ascending §2.4 cost**, 100 / 150 / 200 / 300. The key is the cost column, deliberately **not** the order §2.4's table happens to print, which is Infantry, Tank, Artillery, Recon; an earlier draft of this row asserted the two were the same and they are not. A production choice has no board position to sort on, so hex order would have been an arbitrary key there, whereas cost is the one total order the buildlist already carries. Cheapest-first agrees with §2.9's stated buildlist bias on its **first** term, Infantry, but not on its second: §2.9 favours an occasional Tank and this priority ranks Tank last. The two are not in conflict because they answer different questions — the bias governs what the AI *prefers* when it is choosing freely, the priority only what it does when two options have already scored **equal**. Determinism is now a stated rule on every axis rather than an implementation accident. T-AI-06 unblocked.
Q9	~~AI tie-breaks.~~ **RULED (this revision).** §2.9's "best expected-damage target" and pathing choices can tie.	T-AI-06; AI determinism as a stated rule rather than an implementation accident	
	Flag designation. Confirm exactly one flag per side,	T-SCN-01's	§2.4's "Tank

ID	Question	Blocks	Assumption in force until ruled
Q10	designated by the scenario (`isFlag`, Stub 7), with otherwise-standard Tank stats.	exactness; the flag's §4.8 representation	variant" reads as *not producible and nothing else*.
Q11	Undo. The command log makes single-step undo of an uncommitted move nearly free, but the GDD never grants it, and §2.6's forecast-then-commit flow arguably forbids it.	Nothing today — no gate assumes undo exists, and §2.11.1's `Z` binding is explicitly conditional	No undo: a completed move stands.
Q12	Zero-RNG confirmation. §2.6 and §4.1 say any RNG "is seeded"; nothing in §2 actually uses RNG.	§4.10's `seed` field; T-SAVE-01/02 if RNG is ever added	The prototype ships with **none**, so `seed` is a reserved field written as 0.
Q13	Player-facing replay. §4.10's format makes a "watch replay" feature cheap, but §2.10 does not scope one.	The §2.10 scope table; a UX handoff if it is scoped	The format stays internal — saves, gates, and balance logs only.
Q14	Capture-in-progress at the cap. Does a partially captured objective count toward "objectives held" (§2.8 criterion 2)?	T-CAP-05 — the one `T-CAP-` ID with no `T-TURN-` counterpart (§2.8's alias map); its gate home was ruled on 2026-08-02 to be Stub 8's snapshot, where it is `GATE-CAP-PARTIAL`, and row 8 has since landed at `7c36303`, where that gate ran and passed under clang++ and MSVC both, on a fixture configured with `captureTurns = 2` — a state *Ferrum Crossing* cannot reach, since it ships N = 1 on Q4's ruling. Pass 1 of that build awarded partial credit toward `objectivesHeld` and the gate refused it, so this row's stated reading is asserted rather than only written. **T-CAP-05 closes at that commit — ruled 2026-08-04.** The fixture is the configuration the invariant is about, not a stand-in for a map nobody authored: the invariant raises a unit's `captureProgress` *short of completion*, a state that does not exist at N = 1, so its own text entails N ≥ 2. N is per-scenario data (§2.7) rather than a constant, so a rule that governs every	It counts for nobody until the objective flips — §2.7's flip-on-capture wording grants nothing before the flip. Partial credit

ID	Question	Blocks	Assumption in force until ruled
	(§2.5 clause 2).	scenario is properly gated at a value that rule is about, and a synthetic fixture is legitimate exactly where the stub calls for one and counts when it runs — the precedent is **T-SCN-08 (c)**, whose stub asks for a scenario whose lanes both cost 7 and names no map, and which ran and was counted at `9086d6a`. T-SCN-08 itself does not close, on fixtures (a) and (b), which need a stretch map nobody has authored — and that contrast is what carries this ruling: `GATE-CAP-PARTIAL` has **one** written fixture and it ran, under clang++ and MSVC both, so no part of its set is outstanding and Q29, read per ID, is satisfied here where it is not satisfied there. **This rules the closure only, and not Q14's own rules question** — whether a partially captured objective counts toward "objectives held" — which keeps the stated reading in the next column and stays open; the kb victory table	would need a fractional-count rule and would invert T-CAP-05.
Q15	The 5-unit standard starting force (§2.13.1) — 1 Flag Tank, 2 Infantry, 1 Artillery, 1 Recon — has no antecedent outside §2.13, alongside map dimensions and town counts.	§2.13.1; Stub 7's deployment list	As drafted in §2.13.1: one of each producible system live from turn 1, a 550-Fame producible force.
Q16	Recon/Air vs. Water. §2.3 marks Water passable by "sea, air"; §2.4's "Recon/Air" never says whether it *is* air. If Recon crosses Water freely, every bridge chokepoint and *The Causeway*'s lockout premise leaks.	All three §2.13 maps; the terrain schema	Recon is a **land** unit with terrain-cost discounts, and bridges bind it. All three maps are priced on this reading.
	Cross-Water Artillery fire. With LOS blocking a stretch goal (§2.2), bank-to-bank fire across a one-hex river is legal at ship, and the two river maps price it **per map, not alike**: *Ferrum Crossing* prices contested **bank**		

ID	Question	Blocks	Assumption in force until ruled
	control on it — opposite banks are distance 2, inside Artillery range, so fire crosses before units do (§2.13.2, which states as much explicitly: bridge control there is *tempo, not a topological wall*); *The Causeway* prices its **Mountain perches** on it — range 2–3 covers the bridge hex from +40% cover with no counter (§2.13.6). *Longwater March* has no Water and prices nothing on it (§2.13.5).		
Q17	~~Legal. If LOS blocking ships,~~ Water must not block — or those two maps need a redesign pass.	§2.13.2 and §2.13.6 balance, if LOS ever ships	
Q18	Seat-select scope. Is choosing your seat on the shipped map in scope as "scenario data + one menu affordance"?	§2.13.4's replay ceiling — 6 configurations if yes, 3 if no	In scope, as §2.13.4 assumes. It is the cheapest replay lever in the document.
Q19	Factory count as a per-scenario dial. *Longwater March* (§2.13.5) ships **6** factories against §2.7's "typical ~4 factories total" — inside §2.7's "two or more neutral" clause, but above its stated typical. Does §2.7's ~4 describe the shipped map only, leaving count as the match-length dial §2.13.5 uses it as?	§2.13.5; T-SCN-03's economy check, which currently cites "~4 factories total"	Yes — ~4 describes *Ferrum Crossing*; §2.13.5's 6 is a deliberate long-map dial, and T-SCN-03 asserts only the home-plus-two-neutral floor.
	~~Save/replay milestone split.~~ **RULED (this revision).** The row is **split, not moved**: the §4.10 **format +**		Ruled, as §4.11 itself proposed. §4.4's weeks 2, 4 and 5 and the note under the table now describe one schedule, and the distinction the split turns on is stated there rather than left to be re-derived: **a format is a test instrument** and ships with the gates that consume it (T-INT-02 **first runs** wk 2, T-SAVE-07 wk 4); **slot I/O is a feature** and ships with the rest of the UI. Scheduling-adjacent to **Q23** and ruled on the same principle in the opposite direction — a milestone that outran its dependencies moved later; an instrument its own gates outran moved earlier. **Amended this revision, and the amendment is the useful part of the row.** The ruling was written into §4.4 without re-reading §4.11's

ID	Question	Blocks	Assumption in force until ruled
	headless replayer land in **week 2**, and only the **save-slot UI and slot I/O** stay in week 5. Original question: §4.11 showed §4.4's week-5 save/load placement was one week late in one respect — the format and headless replayer are the instrument for the week-2 integration gate (T-INT-02) and the week-4 self-play logs (T-SAVE-07). Split the row?		depends on which [illegible]
Q20		§4.4's milestone table; T-INT-02 and T-SAVE-07 sequencing	"rows 1–5 built" for row 9 while Q23 had just limited week 2 to rows 1–3 — so the document promised a gate, and its instrument, in a week its own build order could not support them. That is the Q23 contradiction class one layer down, and the third time these two sections have disagreed. It is repaired by **scoping, not by moving**: an integration or replay gate is scoped to the command set of the log it runs on, so T-INT-02 **runs** in week 2 over `{Move, Attack}` — which already reaches the whole compiler-divergence class it exists to catch, since `resolveDamage` holds the only non-integer step in the module — and **closes** in week 3 when rows 4–5 supply the remaining three commands. No week number moved. Whether a partial run may flip a §3 ledger row is registered as **Q29** rather than assumed.
			Ruled as drafted: terrain alone. It reproduces §2.13.1's measured 5/5, 4/4, 3/3 and matches how the lane is actually played, since the other four units move too. Accepted consequence: a map can pass while a seat's own unit sits in the lane on turn 1 — the player walks around it, which on *Ferrum Crossing*'s 1 MP of slack may cost a turn and is absorbed by beat 2 being a standing

ID	Question	Blocks	Scope
Q21	~~Opening-capture lane measurement.~~ **RULED.** The lane prices on **terrain alone**, occupancy excluded. The question was whether T-SCN-06 should price it on the board **as deployed** — where, under Q3's blocked-pass-through reading, a seat's own four other starting units can make its own lane unmeasurable? The two readings can disagree by several MP on a crowded deployment.	T-SCN-06's pass/fail and T-SCN-08's reported integers; **T-SCN-11's four opposing routes on the shipped map** — the four cells of §2.13.2's eight-route table that run against a guided lane (East (9,3) and (9,1) → South; West (1,3) and (1,5) → North) — which price on this same convention, as do the table's other four cells: the two guided lanes themselves (T-SCN-06's, not T-SCN-11's) and each seat's second Infantry to its own objective; §2.13.1's three-map lane table if the answer is "as deployed"	**~~Assumption in force until ruled checked after the Q28 deployment move rather than assumed.~~** This row's scope was stated as a property of the drawn deployment, and a deployment has since moved — East's second Infantry (9,5) → (9,1), §2.13.2 — so it was re-measured. It holds, and at a sharper resolution: **no starting unit sits on any of the eight routes *Ferrum Crossing* now prices** — §2.13.2's four-Infantry × two-objective table *is* the whole priced set, and it already holds the two guided lanes (T-SCN-06) alongside T-SCN-11's four opposing routes and the two same-seat routes that fix each seat's cheaper prize. (9,1) is the origin of two of the eight and lies on the interior of none of them, and both stretch lanes were already clear — so an "as deployed" ruling would move **no number on any map as drawn**. This row previously read "ten," adding the two guided lanes to a table that already contained them, and called all eight T-SCN-11's; the corrected split is 2 + 4 + 2. It stays a live question rather than a dead one because of slack — measured per map, not asserted over the set: *Ferrum Crossing* carries 1 MP against T-SCN-06's ceiling and 1 MP against T-SCN-11's inequality; *Longwater March* carries 2 MP and 4 MP; *The Causeway*

ID	Question	Blocks	Assumption in force until ruled
			3 MP and 2 MP. No figure there is new — each ceiling slack is stated in that map's own section (§2.13.5, §2.13.6) and each margin is a subtraction of the owning/opposing pair this register already prints at Q22. So *Ferrum Crossing* is the **tightest map in the set on both gates at once**, and the one where a deployment edit landing in a priced route is likeliest to flip a gate rather than be absorbed — which is a ranking, not the claim that the other two maps cannot be flipped at all. One further consequence, now that Q22 has widened the surface: terrain-only pricing is what lets T-SCN-11 minimise over a whole seat's Infantry in **one** reverse Dijkstra per objective (§4.7 Stub 7). An "as deployed" ruling would give every unit its own graph and return that cost to one path per unit.
	~~Uncontested vs. merely reachable.~~ **RULED.** The validator asserts non-contention: **the opposing seat's cheapest Infantry route to the same objective must cost strictly more than the owning seat's lane.** T-SCN-07's distinctness clause is a **floor beneath** that requirement, not the whole of it: two seats can name DIFFERENT objectives and	T-SCN-07's clause, now a floor; **T-SCN-11**, written, unblocked and asserting since Q28 ruled; §4.11	Ruled — and **measured against all three maps before the invariant was written**, which is what the ruling cost and where it paid. Five of the six lanes cleared as drawn, each printed **owning against opposing** as T-SCN-11 reports it: *Longwater March* 4 against 8 in both seats, *The Causeway* 3 against 5 in both seats, *Ferrum Crossing*'s East lane 5 against 6. **The sixth failed on an exact tie**: West's South lane cost 5, and East's second Infantry at

ID	Question	Blocks	Assumption in force until ruled
Q22	still race each other to one of them, which is exactly ~~what a structural check~~ cannot see. Gated as **T-SCN-11** — T-SCN-10 is reserved-but-unwritten for the horizontal mirror and Q26 keeps it that way, so it was not free to take. Original question: §2.13.1 promises the guided lane is "uncontested, not merely reachable," but states it as a property of the shipped map rather than a checkable rule.	~~Q20 ruled, §4.6~~ row 7's priced half, which gains one full-board pass per `guidedOpening` entry	(9,5) reached that same objective ~~(5,7) in 5 MP flat~~ — 5 against 5, an exact tie. That is the case the rule exists to catch, it was on the map that ships, and every other invariant in the suite passed. **The map was corrected rather than the rule loosened** (Q28): that Infantry now deploys at (9,1), all six lanes pass, and *Ferrum Crossing* reports **5 against 6 in both seats**. The tie is retained as T-SCN-11's fixture (b) — a failing fixture that was authored rather than constructed.
Q23	~~Milestone-vs-build-order contradiction.~~ **RULED (this revision).** §4.11's critical path makes the baseline AI (row 6) depend on row 5, which depends on row 4 Capture & Fame — but §4.4 promises a working vertical slice *with* the baseline AI in week 2, and §2.10 states capture + Fame production "land wk 3, not wk 1–2." The two schedules cannot both hold.	§4.4's week 1–2 milestones; §2.10's IN row; §4.11's critical path; and Q20, which is the same decision for save/replay	Ruled: the vertical-slice milestone moves to **week 3**, and week 2 delivers **move + attack only** (§4.4). §1, §2.10, §4.4 and §4.11 now describe one schedule — §1's "core playable" line is the fourth site and was corrected with them.
Q24	~~Symmetry as a declarable value.~~ **RULED.** The field stays `rot180 | none` with an even-row-count precondition, and `rot180` on an odd row count is a hard refusal. Original question: §2.13.1 previously offered `mirror / rotation / none`, but an odd-r offset rectangle has **no vertical** mirror axis at any dimension, and admits a 180° rotation only when the row count is even. So a vertical `mirror` is a value the validator could never legally accept, and `rot180` is only well-formed against an even-H map. It does, however, have a **horizontal** mirror axis (c ↔ c, r ↔ H−1−r) exactly when the row count is **odd** — the geometry the shipped 11 × 9 map sits on (§2.13.1 fact 1, §2.13.2) — so the field is a choice, not a forced hand. Narrow the field to `rot180 | none` with an even-row-count precondition, and make `rot180` on an odd row count a hard refusal rather	§2.13.1's validation-invariant list; Stub 7's `symmetry` field and T-SCN-08's fixtures (§4.7, `tech-director`-owned)	Narrowed, as §2.13.1 now reads: `rot180 | none`, and `rot180` declared on an odd row count refuses the file with a reason (a wrong dimension is an authoring error, not a balance question). Both stretch maps are redrawn on 8 rows and declare `rot180` (§2.13.5, §2.13.6); *Ferrum Crossing* declares `none`. If the Director rules otherwise, the only consequence is that a bad declaration surfaces as N failed hex comparisons instead of one refusal — no layout moves.

ID	Question	Blocks	Assumption in force until ruled
	than a failed hex comparison? (**Whether a third value should be admitted for the horizontal mirror is Q26**, which owns that question — this row asks only about the narrowing.)		Whether the enum should also carry a horizontal-mirror value is answered at **Q26**, not here.
Q25	~~What a `rot180` declaration binds.~~ **RULED.** Terrain + ownership (sides exchanged) + placements (sides exchanged); `guidedOpening` is **not** bound. Original question: §2.13.1 says declared symmetry is machine-verified but never says over *what*. Terrain only? Terrain + ownership + placements — the reading both stretch maps are actually drawn to (§2.13.5 "every one of those is a ρ-pair"; §2.13.6 "East is the exact ρ-image of West")? And does it bind `guidedOpening`, so the two seats' lanes must themselves be ρ-images?	T-SCN-09's assertion set; and T-SCN-08's fixture (b), whose equal 4 / 4 is a *theorem* if `guidedOpening` is bound and only a *measurement* if it is not	Terrain + ownership (sides exchanged) + placements (sides exchanged), as gated in T-SCN-09. `guidedOpening` is **not** bound: §2.13.1 requires each seat's lane to be its own neutral within 6 MP, never that the two be images of each other, so a `rot180` map may legitimately name non-image lanes and report a split. Costless either way today — both stretch maps satisfy the *stronger* reading as drawn (*Longwater*: ρ(1,2) = (11,5) and ρ(4,1) = (8,6); *Causeway*: ρ(1,2) = (7,5) and ρ(3,2) = (5,5)), so ruling `guidedOpening` in would fail no shipped map, and ruling terrain-only would merely loosen T-SCN-09.
Q26	~~Is a **horizontal** mirror declarable?~~ **RULED — no**, ruled together with Q24 as one decision. The enum stays at two values and **T-SCN-10 stays unwritten**; a horizontal mirror may be added later, purely additively, if a map ever needs one. Original question: Q24 narrowed the enum to `rot180 | none` partly on the ground that `mirror` is a value the validator could never legally accept. That holds for the **vertical** axis at every dimension, but not for the horizontal one: μ(c, r) = (c, H−1−r) is a genuine isometry of an odd-r rectangle whenever the **row**	Stub 7's `symmetry` field (§4.7); the reserved **T-SCN-10**; nothing else — no gate asserts anything about a value the schema does not admit, and	The enum stays `rot180 | none` and T-SCN-10 stays unwritten: adding a value to a REQUIRED enum moves every scenario's hash, and no shipped map needs it — *Ferrum Crossing* declares `none` because its terrain is asymmetric, and both stretch maps are even-H `rot180`. The conservative reading is the narrow one, so a later ruling only ever *widens* the accepted set and *adds* an assertion; nothing that passes today would start failing. The cost of leaving it narrow, stated plainly: an author who genuinely draws a horizontal mirror

ID	Question	Blocks	Assumption in force until ruled
	count is odd — H−1 is then even, so rotation and H−r share a parity, the offset is preserved and the column is unchanged. *Ferrum Crossing* is 11 × **9** and carries that axis geometrically (§2.13.1). So Q24's enum survives, but as a **scope** decision, not an impossibility. Add `mirrorH` with an odd-row-count precondition — the exact counterpart of `rot180`'s even-row-count one — or keep the enum at two values and accept that a true horizontal mirror is undeclarable?	does not admit, and no §2.13 map is drawn to a horizontal mirror	on an odd-H map must declare none, and `none` asserts nothing — a verifiable property is silently discarded rather than an authoring error caught, which is the opposite of the failure mode Q24 was choosing between. Note the two are never in competition: `rot180` needs even H, a horizontal mirror needs odd H, and their composition would be the vertical mirror that never exists, so at most one non-`none` value is well-formed on any given map.
Q27	~~Is gating End Turn during beat 1a presentation or rule?~~ **RULED.** It ships as specified: End Turn is inert during beat 1a until the marked Infantry has moved. Teaching by constraint is accepted here because it guarantees beat 1a retires inside turn 1, which is what makes §2.11.6-B's schedule predictable. Original question: During beat 1a of a guided opening only, End Turn is inert until the marked Infantry (`guidedOpening.infantry`) has moved; hover reads `Move the marked Infantry first.` It is scoped to the first match, dies with `Skip guidance`, and never applies outside the guided window — but it is the only guided-opening constraint that gates a player *input* rather than a selection, which is why it is registered rather than assumed.	Nothing today, and nothing in §4.7: no stub or `T-` ID gates the directive strip. Stub 7 kept the guidance layer out of Stub 8's view-model entirely; since 2026-08-04 it does not — *marked* is the snapshot field `isGuidedMarked` and *locked this turn* is the presentation-block member `lockedThisTurn` (§4.7 Stub 8), and T-INT-05 reaches both. The dependency is internal to §2 — §2.11.6-B's turn-1 row is unconditional in all three branches only if 1a cannot outlive turn 1, so a ruling of "no input gating" would require that row to be re-derived	Ruled: ships as specified. The alternative, recorded because it was weighed: a player who ends turn 1 without moving leaves 1a outstanding, and rule 1 hands the strip to 1b (`End turn.`) — an instruction they have just followed — so the fallback is to let 1a expire silently at the turn boundary like 1b, and §2.11.6-B's turn-1 row gains a footnote.
			Ruled the strict way, knowingly and at a stated price. (b) was available and would have passed the shipped map untouched; it was refused because the property Q22 protects is a **race**, and a race does not care which Infantry wins it. T-SCN-06's named-hex quantifier is

ID	Question	Blocks	Assumption in force until ruled
			hex quantifier is not a counter-precedent — it names a hex because the guided lane must be the one turn-1a marks, whereas nothing is marked on the opposing side and the only question is who can arrive. **The cost was one deployment move, not a weakened rule.** East's second Infantry moved **(9,5) → (9,1)** (§2.13.2); no terrain, factory or town count, lane cost, home-factory-empty rule or turn estimate moved with it, and the relocation was **forced rather than chosen** — East's south town (7,6) is 2 hexes from South (5,7), so by the triangle inequality any hex covering that town races that factory, and the only free southern hexes clearing 5 MP were the Artillery's and the Flag Tank's. *Ferrum Crossing* now reports **5 against 6 in both seats** — 1 MP each way, the thinnest margin in the set. The pre-fix **5 against 5** is kept as T-SCN-11's fixture (b): a failing case that was actually authored, that passes every other invariant in the suite, and that reading (b) would have passed.
Q28	~~Whose Infantry the T-SCN-11 opposing route is measured from.~~ **RULED (this revision).** Reading **(a)**: the opposing route is minimised over **any Infantry that seat owns**, not over its `guidedOpening.infantry` alone. Original question: Q22 ruled that the opposing seat's cheapest Infantry route to a guided objective must cost strictly more than the owning seat's lane, but not over which units "cheapest" ranges — **(a)** every CanCapture-row unit that seat deploys, since either Infantry can race, or **(b)** that seat's own `guidedOpening.infantry` alone, which keeps the comparison lane-against-lane and matches how T-SCN-06 quantifies over a NAMED hex rather than an existential.	Nothing further. **T-SCN-11 is unblocked and asserting** (§4.7 Stub 7), and §4.11 row 7 carries it in its acceptance set	
	Ledger-flip criterion for a partially-scoped gate. §4.11 rows 9–10 now **run** their gates in week 2 over a `{Move, Attack}` log and re-run them over the complete command set in week 3 (Q20, amended). §3's ledger	§3's ledger rows	**Conservative reading in force:** a row flips only when its **full** acceptance set passes over the **complete** §4.9 command set at one commit; a partial pass is reported as a run and never as a closure, and §4.11 rows 9–10 are written in exactly those two parts. Free in the

ID	Question	Blocks	Assumption in force until ruled
Q29	says a row is verified when it cites "the commit and passing test IDs that back it," and §3's two bars require agent-authored tests that pass plus a human sign-off — but neither says whether *passing* means the acceptance set ran over the system's whole input domain. Without a rule, a green week-2 T-INT-02 could flip a proposed ledger row while the log it replays is missing three of the five §4.9 commands.	for §4.9 and §4.10 (both proposed rows, both unwritten today); no gate — every test runs either way, this governs only what may be *claimed* from a run	conservative direction — it can only delay a claim, so a later loosening invalidates nothing that passed. The alternative worth weighing, since the information is real and currently discarded: record partial passes in the ledger as a dated *"green over subset X at commit Y"* line, which reports more without claiming more — that is a §3 presentation decision, not a technical one, which is why it is registered rather than assumed.
Q30	What a T-SCN-11 "against" print quantifies over, and what form a hex-scoped route cost takes. A withdrawn revision of the print convention (§4.7 Stub 7) named **two** relations for an "against" and gave two printed forms — a bare pair for owning-against-opposing, a named ceiling for measured-against-budget — but there are **three** quantities in play, and when this row was filed the third had neither a printed form nor an exclusion. The bare pair's right-hand term is this invariant's opposing term and is therefore the **minimum over every CanCapture-row unit the opposing seat deploys** (Q28); that is derivable from the formula, was never stated at a print site, and was read the other way by a reader looking specifically for it. A cost measured **from one named hex** is neither of the two printed relations, so it has nowhere to go but the bare form, which means something else. That is mechanically how asymmetry (ii)'s Bridge-free counterfactual came to print "5 against 14": 14 is West's Bridge-free cost from **(1,3) alone**, while the set minimum is **13**, from **(1,5)** — `(2,6)(3,6)(3,7)(4,7)(5,7)F(6,7)(7,6)T(6,5)(6,4)w(6,3)w(6,2)`, West's own guided South lane plus 8 MP up the east bank. The two differ	**Nothing computable.** T-SCN-11's inputs, formula, unit set, reported integers, refusal conditions and all three fixtures are identical either way, and no map, lane cost or deployment moves. It governs only what §4.7's **prose** may claim from a measured integer — which is where the error was, and is the one place this suite has no gate.	**Partial reading in force, deliberately not hardened.** Two things are settled and written into the convention: a bare pair is **set-quantified** (the right-hand term is the minimum over every CanCapture unit the opposing seat deploys, per Q28), and a cost measured from some other named hex is printed with its hex and what it is — "14 from (1,3) alone, not the set minimum". Naming a hex beside a bare pair stays legal when that hex **is** the minimiser, which is what fixture (a) does. The counterfactual figures are corrected to this reading: **5 against 13**, margin **8 MP**, with **14 retained** as the explicitly (1,3)-scoped figure it always was. **What is left open is whether this should harden into a closed list of permitted forms.** An earlier revision wrote that closure and it was withdrawn: each version outlawed prose elsewhere in the document that

ID	Question	Blocks	Assumption in force until ruled
	bank. The two differ because **excluding the Bridges moves the minimising unit**: with them permitted West's set minimum is 6, achieved by (1,3), and (1,5) alone costs 7; without them the set minimum is 13, achieved by (1,5), and (1,3) alone costs 14. Should the convention gain a **third printed form** for a hex-scoped cost ("14 from (1,3) alone, not the set minimum"), or should a hex-scoped cost be **forbidden** inside an "against" print entirely?		was correct — a within-seat ranking in §2.13.2, an objective-count comparison in §2.13.5 — and twice broke its own rule inside the passage stating it. That is a poor trade for a convention binding wording only, so the question is left to a human rather than half-answered. Nothing computable turns on it: no invariant, fixture, reported integer or refusal condition depends on any of it.
Q31	Queuing a build at a boxed-in factory. §2.7 says a build that cannot spawn **waits**, and Q8(c) prices that wait as a non-refundable commitment — but §2.11.5 disables the Build buttons while a factory is boxed in, so the player can never create the state the clause prices. Should the buttons be enabled, with the commitment shown before the click, or is the waiting build correctly an AI-only path?	§2.11.5's Build-button rule; §2.7's waiting-build clause; Q8(c)'s commitment, which today binds only on ordinary player builds	§2.11.5 ships as written: Build is disabled while boxed in, so the waiting build is an **AI-only path** (§2.9 builds without the UI) and for the player queue time and spawn time are the same instant. No gate asserts a player-queued waiting build. Registered rather than assumed because Q8(c) ruled on a state the UI makes unreachable — the ruling stands, its player-facing half is simply not exercised yet.
Q32	Three §2.13.1 validation checks that no `T-SCN-` ID asserts. §2.13.1's validation-invariants bullet names *every land-passable hex reaches every factory*, *all deployment hexes are free and land-passable*, and *factory count in the map file equals the count the domination check uses*, for the §4.2 `validate_scenario` tool whose schema is Stub 7's. Building row 7 against that stub found no `T-SCN-` ID that gates any of the three as written. Widen an existing invariant, mint IDs for the three checks, or accept that the tool description is wider than the gated suite?	Nothing today: no `T-SCN-` ID gates the three, so no gate waits on the ruling. A ruling that mints an ID for one of the checks would move §4.5's written-ID count	**The invariants ship as written.** Row 7 implemented T-SCN-02 and T-SCN-04 to their exact written text at `9086d6a` rather than widening them, and minted no ID for the three checks, so the bullet describes more than the suite gates. **Ruled: widen the check, and never narrow the text.** The widened check accounts for every file in

ID	Question	Blocks	Assumption in force until ruled
Q33	~~An invariant whose written text and running check disagree.~~ **RULED (this revision), and registered already ruled.** `T-INT-01` required every file in `Source/StratRules/` to hash-match the recorded crew commit. Two vendored files had no counterpart at that commit to match, so the text was unsatisfiable as written, and the check shipped at `b23823f` did not reach them — measured rather than assumed: a comment appended to `StratRules.Build.cs`, the manifest's `note` altered and its `generator` altered each passed that check undetected (§3). Narrow the text to describe the check that runs, or widen the check until the text is satisfiable and then amend the text to what was actually met?	`T-INT-01`'s invariant text (§4.9), and the commit its green is credited to. No gate waits on the ruling: `T-INT-01` runs on every gate run under either answer, and neither answer mints an acceptance ID.	~~**Assumption in force until ruled**~~ ~~that directory and exempts none; it landed at `d837fc8`~~ with the UE project repo at `9dec48c` (§3), and §4.9's invariant text is amended to state what must hold rather than to describe an implementation. **It mints no acceptance ID:** the amendment widens the coverage of an ID that already exists rather than adding an assertion the suite did not carry, so no §4.5 total moves on its account — what moves is the commit `T-INT-01`'s green is credited to, under the closure convention §4.5 states. The general rule the row settles: **where an invariant's text and its running check disagree, the check moves first and the text is then amended to the requirement that was actually met** — a text edited down to what the code happens to do asserts nothing the code could fail, which is the failure this ledger exists to prevent. It is registered already marked RULED because the Director ruled it in the session that found it; the register carries it so the amendment above has a stated cause rather than an unexplained rewrite. **Ruled: across a repo pair, in the two-repo pinning form this document already uses for `T-INT-01`** — one crew commit and one UE project commit, cited together. Row 2 flips on the pair `b1ea992` and `fed8ae9` (§3).

ID	Question	Blocks	Assumption in force until ruled
Q34	~~How Q29's *at one commit* reads when an acceptance set spans two repositories.~~ **RULED (this revision), and registered already ruled.** Row 2's acceptance set is `T-DATA-01..04, 06` in the crew repo and `T-DATA-05` in the Stratocracy UE project repo, so no single commit in either repo can carry both halves and Q29's condition is unsatisfiable read literally. Read it across a **repo pair**, or leave every two-repo row permanently unflippable?	§3's row 2, and any later row whose acceptance set spans both repos — build-order rows 9 and 10 among them. No gate waits on the ruling: every ID runs either way, and neither answer mints an acceptance ID.	~~What disproves Q29's condition is~~ **Q29's condition is ruled** its own terms is that no fixture went unrun, and that `GATE-DATA-VENDOR` establishes both halves ran against the same data bytes. The conservative reading Q29 states is preserved rather than loosened: a pair whose halves disagree about the data closes nothing. **Which IDs a §4.8 CSV edit re-opens is ruled with it:** such an edit re-opens the **whole** pair — `T-DATA-05` and the headless `T-DATA` IDs together — and the flip re-pins to a new pair when the re-run completes. That records the gates' existing behaviour rather than imposing new behaviour: perturbing `units.csv` without re-importing failed both the parity check, on `Infantry.HP` expected 11 and read 10, and `GATE-DATA-VENDOR`, on a sha256 mismatch, while the suite's other three tests stayed green (§3). Re-opening the headless half alone was the alternative, and it is refused for that reason — the bytes both halves read are one file, so a change to them is a change to both halves' subject. **That rules which IDs re-open and nothing about what this ledger shows in the interval** between the edit and the re-run; the pinned record is untouched either way, since *green at `b1ea992` over the UE tree at `fed8ae9`* stays true of those commits, which is what

ID	Question	Blocks	Assumption in force until ruled
			pinning is for [illegible] registered already, marked RULED because the Director ruled it in the session that found it.

4.8 Data contract — DataTable schemas

Principle: authored once, read twice, proven equal. Each table is one canonical CSV, authored in the crew repo (`data/`). The headless loader parses it directly; the UE project vendors it verbatim beside a manifest recording each file's sha256 and the crew commit it came from, and the editor imports the vendored copy into a `UDataTable` whose row struct derives `FTableRowBase`. `GATE-DATA-VENDOR` asserts that the vendored bytes are the recorded ones, so the two readers are reading one authored file rather than two that resemble each other; it **mints no acceptance ID**, on the `GATE-AI-SMOKE` and `GATE-SAVE-PARSE` precedent (§3). **The manifest's `dataCommit` names the commit the vendored bytes came from, and it advances when and only when those bytes change (ruled 2026-08-06).** A crew commit that does not touch the data directory leaves it where it is, so a `dataCommit` behind the crew commit this document stands at is the **expected** state and not a stale one: it records `b1ea992`, and the three CSVs are byte-identical at that commit and at `c2edae0` (§3). `GATE-DATA-VENDOR` reads `dataCommit` and logs it; what it asserts is the file hashes and not the commit's currency, because the UE project cannot see the crew repo at test time — the same discipline that makes this document cite commits rather than heads everywhere else. A parity gate (T-DATA-05, Unreal Automation) iterates every imported row and asserts it equals the CSV field-for-field. Nothing is authored twice, so the headless sim and the engine can never disagree about a stat. Missing column or unparseable value = hard load failure, never a silent default. **UStruct bool fields carry no `b` prefix, and that is an engine constraint rather than a style choice (ruled 2026-08-06):** `DataTableUtils::GetPropertyImportNames` accepts only `GetName()` and `GetAuthoredName()`, there is no metadata bridge, and `meta=(DisplayName=...)` was tried and failed identically, so a `bCanCapture` field cannot be fed by a `CanCapture` column. The CSV column names and the headless field names are unchanged by that, so the transformation between the two sides stays empty.

Unit schema — `data/units.csv` → headless `strat::UnitDef` → UStruct `FUnitRow`. Exactly four rows (Infantry, Tank, Artillery, Recon; §2.4). The **flag unit is not a row**: §2.4 defines it as "a designated Tank," so flag status is a placement-level field in the scenario file (`isFlag`, §4.7 Stub 7), gated by T-SCN-01 — not a fifth unit type. One representation, one gate.

Column	CSV type	Headless field (`strat::UnitDef`)	UStruct field (`FUnitRow`)	Source
`Id` (row name)	string	`id`	RowName (`FName`)	§2.4
`HP`	int	`hpMax`	`int32 HP`	§2.4 (10/20/8/12)
`Move`	int	`move`	`int32 Move`	§2.4 (3/5/3/7)
`Atk`	int	`atk`	`int32 Atk`	§2.4 (4/8/10/5)
`Def`	int	`def`	`int32 Def`	§2.4 (2/5/1/3)
`RangeMin`	int	`rangeMin`	`int32 RangeMin`	§2.4 (Artillery 2, others 1)
`RangeMax`	int	`rangeMax`	`int32 RangeMax`	§2.4 (Artillery 3, others 1)
`CostFame`	int	`costFame`	`int32 CostFame`	§2.4 (100/300/200/150)
`Type`	enum string	`strat::UnitType type`	`EUnitType Type`	addendum Part A — order fixed: Infantry, Tank, Artillery, Recon
`CanCapture`	bool	`canCapture`	`bool CanCapture`	§2.7 (Infantry only)
`MoveClass`	enum string	*reserved*	*reserved*	**blocked on Q2**

`EUnitType` is a `UENUM` mirroring `strat::UnitType` (`Combat.h`, addendum Part A) with the enumerator order pinned; T-DATA-05 asserts the mirror is exact so an editor-side reorder can never silently reindex the effectiveness table below.

Terrain schema — `data/terrain.csv` → headless `strat::TerrainDef` → UStruct `FTerrainRow`. Exactly seven rows (§2.3).

Column	CSV type	Headless field	UStruct field
`Id` (row name)	string	`id`	RowName
`MoveCost`	int (0 = impassable)	`moveCost`	`int32 MoveCost`
`DefensePct`	int, signed	`defensePct`	`int32 DefensePct`
`PassLand/PassAir/PassSea`	bool ×3	`passLand/Air/Sea`	`bool PassLand/PassAir/Pa`
`Capturable`	bool	`capturable`	`bool Capturable`
`IncomeFame`	int	`incomeFame`	`int32 IncomeFame`
`IsSpawnPoint`	bool	`isSpawnPoint`	`bool IsSpawnPoint`
`IsRepairPoint`	bool	`isRepairPoint`	`bool IsRepairPoint`

Type-effectiveness schema — `data/effectiveness.csv` → `strat::effectiveness` → UStruct `FEffectivenessRow`. A 4×4 matrix, row = attacker type, columns = defender types, values ∈ {0.5, 1.0, 1.5} (§3 spec), **shipping all-1.0** (§2.4 — the triangle stays positional). The verified implementation (`cpp_reference/Combat.good.cpp::effectiveness` @ `5ffa8d6`) hardcodes the neutral stub; this schema is the lever's *data* form, so that if self-play ever asks for a non-1.0 cell, populating it is a CSV edit gated by the existing directional-gate plan (addendum Part A), not a code change. T-COMBAT-09 (neutral stub, 16/16 pairs) continues to pin the shipped state; a non-neutral CSV with T-COMBAT-09 still in the suite is a deliberate, visible gate change the Director must approve — the "do not invent balance values" rule enforced by the pipeline itself.

```
Invariants (the T-DATA set — Stub 2, §4.7):
  T-DATA-01  loaded unit values equal the §2.4 table exactly (4 rows, all
columns)
  T-DATA-02  loaded terrain values equal the §2.3 table exactly (7 rows),
             including Bridge's NEGATIVE defense and Water impassable-to-
land
  T-DATA-03  exactly one unit row has CanCapture == true (Infantry, §2.7)
  T-DATA-04  sanity: all costs > 0; RangeMin <= RangeMax; HP > 0
  T-DATA-05  (editor, Unreal Automation) every imported DataTable row
equals the
             CSV field-for-field, and EUnitType mirrors strat::UnitType
exactly
  T-DATA-06  effectiveness.csv is 4×4, indexed in the pinned type order,
every
             cell ∈ {0.5, 1.0, 1.5}; the SHIPPED file is all-1.0 (re-
asserting
             T-COMBAT-09 at the data layer)
Determinism: pure parse; missing/malformed field = hard fail, no defaults.
Acceptance: T-DATA-01..04, 06 headless; T-DATA-05 in the editor pass.
```

4.9 Headless-module → Unreal integration path

The rules module's value is that it has **zero engine dependencies** (§3, §4.1); integration must add Unreal *around* it without ever adding Unreal *to* it.

1. Module layout — one source, two compilers. The certified headless sources live canonically in the crew repo, and each §4.7 stub joins them as it lands — where the `python run.py` gate runs (§3 ledger), under the single compiler that gate detects: the first of `g++`, `clang++`, `c++` or `cl` found on PATH. The UE project vendors them verbatim into a UBT runtime module, `Source/StratRules/`, via `sync_stratrules.py`, which reads each

source from the git object store rather than the working tree — so identity is true by construction at the moment the script finishes — and records the source commit as `rulesCommit` in `StratRules.manifest.json`. **That module now exists**, at `a13626f` in the UE project repo against `rulesCommit` `e19605e` (§3): the ten modules Combat, Hex, Data, Move, Economy, Turn, Ai, Scenario, Ui and Driver, each as `X.h` and `X.good.cpp`, beside `StratRules.Build.cs`, which is vendored from `ue_module/StratRules.Build.cs` in the crew repo on the same terms as the sources, and `StratRules.manifest.json`, which the sync script generates and which has **no counterpart to be vendored from**: it records `rulesCommit`, and a file's bytes cannot contain the identity of the tree that holds them. `T-INT-01` states what each of those two states requires. Vendored names are unchanged from the crew repo, so the comparison is path-for-path with no rename map. `Driver` is in the set and is not optional: `Ai.good.cpp` links against it. Nothing else is vendored — a UBT module cannot hold a second `main()`, which excludes every `test_*.cpp`, `driver_main.cpp` and `selfplay.cpp`, and the `*.buggy.cpp` files are pass-1 fixtures, not shippable code. **The set is declared, not inferred (ruled 2026-08-05).** `ue_module/vendored_set.json` in the crew repo names it, and both `sync_stratrules.py` and `T-INT-01`'s check read that declaration from the git object store at the commit in question, so neither takes its expectation from the other. The declaration **must partition** the crew's modules: every crew module appears in exactly one of `vendored` and `excluded`, so a module can be neither swept in by a glob nor left out in silence, and the check FAILs a module that appears in neither list and one that appears in both (§3). **Three crew modules exist and are deliberately not vendored (ruled 2026-08-05).** `Save` landed at `737f666`, `Replay` at `ec15be6` and `Balance` at `1ee890e` (§3), and all three stay out of `Source/StratRules/`, where they would be an eleventh, a twelfth and a thirteenth module beside the ten enumerated above. §3 records that no bridge exists — no load mapping, no command surface, no event list, no actor and no widget — so §4.9 part 2 is unbuilt and the bridge consumer that would read them is still hypothetical, while vendoring now would re-date `T-INT-01`'s and `T-INT-04`'s closures. That is a decision rather than an omission, and **nothing re-dated on their account**: after the third module landed `T-INT-01` was green at `d837fc8` and `T-INT-04` at `b23823f`, both still passing 2/2 at `rulesCommit` `d837fc8`, and no UE project commit was made at any of the three landings. Both closures now stand at `e19605e` (§3) — `T-INT-01` because `rulesCommit` moved and `T-INT-04` because `Turn.h`'s vendored bytes changed — and neither moved on these three modules' account, which leaves the decision recorded here as it was made. The enumeration above is correct as it stands, and three modules were left out on purpose. `cpp_reference/selfplay.cpp` is **not** one of the three and is excluded for the different reason stated above — a UBT module cannot hold a second `main()` — and it stays in that exclusion list unchanged. `StratRules` contains **no engine headers, no UObject, no third-party includes** — pure C++17 in `namespace strat`, exactly the base-spec constraint. That constrains the sources and not the setting they are built under: the standalone gate at `031ee20` compiles at `-std=c++17` for GCC/Clang and `/std:c++17` for MSVC, while the UBT build that produced `UnrealEditor-StratRules.dll` compiled the same certified bytes at `/std:c++20` — the value carried by every compiler response file that build left beside a vendored `.good.cpp`. That is the divergence surface this section exists to track. The standalone gate keeps compiling the identical files, so "the engine build works" never substitutes for "the gate passed."

The UBT module is built and linked, and it is not listed in the `.uproject` (ruled 2026-08-06). `Stratocracy.uproject` listed `StratRules` at `a13626f`, and building the `StratocracyEditor` target compiled the vendored crew modules and linked `UnrealEditor-StratRules.dll` — sources UBT had never compiled before (§3). **Listing it made the editor unusable, and the entry was removed at `fed8ae9`.** `Source/StratRules/` contains no `IMPLEMENT_MODULE`, so at `a13626f` the editor aborted at startup with *"The game module 'StratRules' could not be successfully initialized"* and exited: nothing in-editor could run there — not a commandlet, not an Automation test, not the editor. The round that registered it gated that the module compiles and links and never launched the editor, which is the gap that let the defect through. `StratRules` is now a **link dependency of the `Stratocracy` module** instead: UBT still builds it and the game module still links it, and the removal touches no vendored byte, so `T-INT-01` and `T-INT-04` do not re-date (§3). The setting that governs shadowing under UBT is recorded here with the diagnostic that stopped the first attempt: both targets set `DefaultBuildSettings = BuildSettingsVersion.V7`, and `BuildSettingsVersion.V2` onward raises `ShadowVariableWarningLevel` to `Error`, read from the engine's own `TargetRules.cs`. Under that setting C4456 — `Driver.good.cpp` shadowing a local — stopped the first attempt. The build sets `CppCompileWarningSettings.ShadowVariableWarningLevel = WarningLevel.Warning` in `StratRules.Build.cs`, **scoped to this UBT module** and to nothing else, and C4456 still prints, as a warning. **A green UBT build closes nothing.** It compiled and it linked; no editor was launched, nothing was executed, and the build mints no acceptance ID and closes none. `T-INT-04` asserts the **standalone** compile, outside UBT; in-engine compilation is what the editor pass gates, and that pass exists at `fed8ae9` (§3), where it runs `T-DATA-05` and `GATE-DATA-VENDOR` and no in-engine parity check over the vendored module. What the build does add is an MSVC compile **through UBT, under the engine's own flags** — a toolchain configuration these sources had not been built under, and not a compiler they had not seen, the standalone gate having compiled them under clang++ and MSVC both throughout. **A UBT rejection cannot be fixed in place.**

The vendored sources are hash-matched against the crew repo by `T-INT-01`, so an edit in `Source/StratRules/` FAILs the gate rather than fixing the build: the fix goes into the crew repo and the tree is re-vendored. This round is the worked example, and it is not free — both `T-INT` closures re-dated as a result (§3).

2. Bridge — the only code that knows both worlds. The game module (`Stratocracy`) owns: - **Load:** `FUnitRow`/`FTerrainRow`/`FEffectivenessRow` → `strat::UnitDef` / `strat::TerrainDef` / effectiveness table (a mechanical §4.8 mapping), plus `strat::loadScenario` on the shipped scenario asset (§4.7 Stub 7). - **The authoritative `strat::GameState`.** Actors and UMG hold no rules state (§4.1 "never own rules" — here made structural, not aspirational). - **Command in / events out.** Presentation submits commands — `Move{unit, destHex}`, `Attack{unit, targetHex}`, `Build{factoryHex, unitId}`, `Capture{unit}`, `EndTurn{}` — the rules module validates then applies each, and emits an **ordered, deterministic event list**: `Moved(path)`, `Damaged`, `Destroyed`, `Captured`, `Spawned`, `BuildWaiting`, `Repaired`, `IncomePaid`, `MatchEnded(tier)`. Actors and widgets animate and rebind **from events and the view-model only** — §4.7 Stub 8's snapshot plus its presentation block (§4.7). An invalid command returns a rejection reason and changes nothing. - **Threading:** synchronous on the game thread. A full turn resolution is microseconds of integer math; the shipping AI "moves instantly" (§2.8). No async, and no MCP involvement anywhere in the runtime path — the MCP plugin remains editor-only tooling, experimental, off the critical path (§3 guardrails, §4.2, §4.5).

3. Parity gates. The command log format of §4.10 is the instrument: the same recorded match is replayed by the headless harness and by an in-editor Automation test through the UBT-compiled module, and both must land on the same canonical state hash. This is what catches the one genuinely engine-shaped risk — a compiler/CRT divergence in the damage formula's `round` — mechanically instead of by playtest anecdote.

No further spec-stub pass is owed for part 2 (ruled 2026-08-05). The stub below is already full: its invariants carry the bridge's own — `T-INT-02`, `T-INT-03` and `T-INT-05` — beside a Determinism line and an Acceptance line, so what part 2 waits on is not specification. The **in-editor Automation harness** it waited on landed at `fed8ae9` in the Stratocracy UE project repo (§3), so what part 2 waits on now is the bridge's own code — the load mapping, the command surface, the event list, the actor and the widget. The other blocker recorded here was **§4.10's canonical state hash**, `T-INT-02`'s and `T-INT-03`'s subject, which build-order row 10's part (b) built at `ec15be6` (§3). The harness was recorded here among what part 2 was blocked on, and it took no week on §4.4's calendar; both blockers this paragraph named have since been removed, and part 2 is blocked on its own unbuilt code. **What "the editor pass" denotes, stated once here and cited elsewhere (ruled 2026-08-05): the in-editor Automation harness this paragraph names. It is a runner and nothing more, and running it supplies none of the subjects the IDs scheduled into it assert against — those are separate requirements.** The harness is **also not sufficient**, and among what the other editor-pass IDs need besides it are the subjects named here rather than left to be discovered at the pass: `T-INT-02` needs a **vendored replayer**, and `Replay` is ruled out of vendoring until a bridge consumer exists; `T-INT-03` needs the **command surface**, which is part of the unbuilt bridge; `T-INT-05`, `T-UI-03` and `T-UI-04` need **real Stratocracy widgets**; and `T-SAVE-06` is asserted jointly with `T-INT-02`. `T-DATA-05` is not one of them, having closed at `fed8ae9` (§3); what it needed was **imported DataTables and a `UENUM` mirror of the unit type**, and those stay named here because the measurement that follows quantifies over every subject this passage names. None of those subjects exists at `a13626f`, measured in the UE project repo there: the identifier `EUnitType` occurs zero times case-sensitively in the tracked tree; `UENUM` occurs zero times anywhere under `Source/`, a zero controlled against `UCLASS`, which does occur there; and every tracked DataTable and widget asset belongs to the `AdvancedTurnBasedTileToolkit` marketplace content or to the UE template. The stub below carries its own `Inputs` line — a §4.10 replay file and the §4.8 tables imported in-editor — which lists what that stub draws on; the imported tables are not among the DataTable assets measured above. **`T-DATA-05`'s two subjects have since been built** (§3): at `fed8ae9` the `UENUM` mirror `EUnitType` and the imported `DT_Units`, `DT_Terrain` and `DT_Effectiveness` exist, alongside `FUnitRow`, `FTerrainRow` and `FEffectivenessRow` — all of them in the `Stratocracy` module and none in `Source/StratRules/`, which is the constraint stated below. The `a13626f` measurement above is unmoved by that, and it remains the live statement about every other subject this passage names. **Registering the UBT module closes no acceptance ID, and neither does the harness by itself.** One constraint the harness inherits: `T-INT-01` asserts that no file in `Source/StratRules/` is unaccounted for, so **the harness may not live in that directory** — a test file there fails a green ID, confirmed against the running check, where an extra `Sneaky.good.cpp` in that directory FAILs (§3). **Its names vary across this document, and that variance is tracked here rather than being drift (ruled 2026-08-06).** Among the forms this document uses for it are *the editor pass*, *an in-editor Automation pass* and *an in-editor pass*, and §3 records forms from the gate runner's printed output at named commits — at `b23823f`, where the runner prints by name that no in-editor Automation harness exists, and at `41a1452`, where the lines for the IDs that did not run state that no in-editor pass exists at that commit. The denotation is settled by the ruling above and does not turn on which form a section reaches for. A rename would have to reconcile with those commit-pinned §3

records and is deferred to its own round. **No name is changed in this revision.**

```
SPEC STUB: Integration parity              (Director → Systems Engineer /
UI Scaffolder)
Inputs:  vendored StratRules sources + recorded source commit; a §4.10
replay
         file; the §4.8 tables imported in-editor — among this stub's
         invariants, T-INT-02 requires them (ruled 2026-08-06): it replays
a
         command log in-engine to a canonical state hash, and the commands
it
         applies resolve against the unit definitions the bridge maps from
         FUnitRow (part 2 above), the same stats the compiler-rounding
         tripwire in this invariant's own text runs through. T-INT-01 and
         T-INT-04 do not require them: a source-identity check, by
whichever
         mechanism T-INT-01 puts on a given file, and a standalone compile
         load no unit definition. What T-INT-03 and T-INT-05 still need is
         assigned per ID above, and those are different subjects.
Invariants:
  T-INT-01  source identity: every file in Source/StratRules/ is accounted
for
            against the recorded crew commit, and none is unaccounted for.
A
            file that has a tracked counterpart at that commit must hash-
match
            it. A file for which no such counterpart can exist must
instead be
            recomputed from tracked inputs at that commit and equal the
            vendored bytes exactly. Recomputation is not the weaker of the
two:
            it is the strongest check available on a file whose bytes no
stored
            blob can predict. Neither mechanism may take its expectation
from
            the vendored tree or from the vendoring script — the ledger's
            evidence chain survives vendoring
  T-INT-02  replay parity: the same command log replayed headless and in-
engine
            (Automation test) produces the same final canonical state
hash.
            (The tripwire of this stub: an agent that "ports" rather than
vendors
            the module — or a compiler that rounds differently — passes
every
            behavior test in one world and silently diverges in the
other.)
  T-INT-03  rejection safety: an illegal command leaves the state hash
unchanged
            and returns a reason; no partial application
  T-INT-04  no engine deps: StratRules compiles standalone under the
existing
            python run.py gate, using whichever single compiler that gate
            detects — the first of g++, clang++, c++, cl found on PATH.
Any
            one of them compiling clean satisfies this invariant; it does
not
            require all four. The gate run itself is the assert
  T-INT-05  (editor, Automation) presentation statelessness: after any
event
            sequence, rebuilding all widgets/actors from the current view-
model
            alone — §4.7 Stub 8's snapshot plus its presentation block —
            reproduces the same displayed values (nothing lives only in a
            widget). The subject is every member of the view-model, and
the
            presentation block has two: §2.11.1's selection machine is not
a
            widget, so its DONE bit satisfies this gate from the block,
and
            the guidance layer is not a widget either, so its
`lockedThisTurn`
            bit satisfies it from the block on the same footing. The
guided
            opening's other half, `isGuidedMarked`, is a snapshot field
and
            satisfies this gate from there
Determinism: the bridge never reorders, drops, or synthesizes events.
Acceptance: T-INT-01, 04 on every gate run; T-INT-02, 03, 05 in the editor
pass.
```

4.10 Save & replay format

Design choice: a save *is* a replay. Because every system is a deterministic pure function or state machine (§2.6, §4.1, and every §4.7 determinism gate), the cheapest correct save

is not a state snapshot but **the scenario reference plus the ordered command log**. Loading = `loadScenario` + re-apply the log — headless-speed, sub-second. One format, four consumers:

1. **Single-slot save/load** (§4.1's "minimal single-slot," now specified).
2. **The T-INT-02 parity gate** — its input file is a save file.
3. **Balance Analyst self-play logs** (§4.1 harness) — each self-play match is emitted as this same format, so every balance claim in the ledger's evidence chain is a *replayable* artifact, not a summary statistic.
4. **Bug reproduction** — a playtest failure attaches its save; any agent can replay it headless.

File layout — versioned JSON, one file:

Field	Type	Meaning
`formatVersion`	int	This layout's version; unknown = refuse load
`rulesCommit`	string	Crew commit of the rules module that wrote the file (T-INT-01's hash)
`dataHash`	string	Hash of the §4.8 CSV set in effect
`scenarioId` / `scenarioHash`	string	The §4.7 Stub-7 scenario file and its hash
`seed`	int	Reserved; **written as 0** — no RNG ships (§2.6, pending Q12)
`commandLog`	array	The ordered commands of §4.9, exactly as the bridge consumed them, tagged `{turn, side}`
`stateHash`	string	Canonical hash of the resulting state (integrity check)
`result`	string/null	Result tier (§2.8) if the match ended, else null

Canonical state hash. Defined once, in the headless module: serialize the `GameState` in a fixed field order — turn counter, side to move, per-side `fameTotal` and `fameCombat`, objective ownership `{hex, owner}`, per-unit `{id, side, hex, hp, isFlag, hasMoved, hasActed}`, the per-tile capture record `{hex, unitId, turnsHeld}`, the per-factory build allowance `{hex}` (T-TURN-10) and the pending builds `{factoryHex, side, defIndex}` — objective ownership, the per-unit group, the per-tile capture record, the build allowance and the pending builds each walked in the canonical hex order (§4.7 conventions, T-HEX-07), keyed on the hex that group carries, ties within one hex broken by `id` — then hash the bytes. **The build allowance carries `hasBuiltThisTurn` as membership rather than as a field:** the group is walked over the turn module's built-this-turn set, so an entry exists for a factory that has taken its build this turn and none exists for one that has not, each entry emitting a constant marker that carries no information its presence does not. Every emitted value is an **integer** (`eff` and the HP ratio exist only transiently inside `resolveDamage`), so the hash is platform-stable by construction; T-INT-02 proves it across compilers. The module exposes `canonicalStateBytes` beside the digest, so a check can assert the serialisation and not only the digest. **The per-unit turn flags and the per-factory build allowance are carried because a save is accepted mid-turn** (Policies below): at hash time a unit may have spent one of its two flags and a factory may have taken its build for the turn, so a hash without them is identical across states the rules distinguish, and T-INT-02 and T-SAVE-06 would both agree over that difference rather than catch it. **The list above was restated into the groups the modules hold (ruled 2026-08-05)**, when row 10's part (b) implemented it: `captureProgress` and `pendingBuilds` were previously written as members of the per-unit group, while `cpp_reference/Economy.h` holds the first on the tile — progress can never transfer (Q4, T-FAME-05) — and keys the second by the factory hex. **What the hash omits is anything recomputable from the fields it already carries**, since such a value can add no distinction and only one more way for two builds of one state to disagree: §4.7 Stub 8's `spawnBlocked` is that case, being a function of the unit positions this hash carries and the terrain the scenario file fixes, whose own `scenarioHash` is a header field above. `buildWaiting` (T-TURN-10) is the same case, being recomputable from the pending-build group, so the hash carries the pending build and not the flag. That is a narrower test than "derived", and deliberately so — the Policies below call a waiting build and capture-in-progress derived *pending* state because they are a function of the log, and both are hashed, the waiting build as a pending-build entry. **This hash is built**, at `ec15be6` on row 10's part (b), where `T-SAVE-01`, `T-SAVE-02`, `T-SAVE-03` and `T-SAVE-05` closed over it (§3); part (a), which landed before it, defines none of it and carries `stateHash` as an opaque required string. Widening the hash now moves a digest a landed module computes — the cost the

sentence this replaces said would arrive.

Policies (prototype): - **Single slot** — one file (`slot0`), stored via a thin `USaveGame` wrapper holding the JSON so platform save paths are respected. Overwrite-confirm UX is **unowned**: §2.11 specifies no save/load surface, and neither its screen list (§2.11.5) nor its build ranking (§2.11.8) includes one — either §2.11 gains the surface, or the prototype ships silent single-slot overwrite. - **Save points** — a save is accepted between atomic commands during the player's turn. The AI turn resolves synchronously in one call (§4.9), so "mid-AI-turn" is not a reachable save state by construction. - **Mid-match saves** — the log up to the last completed command *is* the save. Derived pending state (a waiting build, capture-in-progress) replays correctly because it is a function of the log. - **Version policy** — mismatched `formatVersion`, `rulesCommit`, `dataHash`, or `scenarioHash` → **refuse load with a reason**. No migration in the prototype; a save is only valid against the exact rules and data that wrote it.

```
SPEC STUB: Save & replay                    (Director → Systems Engineer)
Inputs:  scenario file (Stub 7), command log, §4.8 data set, module
commit.
Invariants:
  T-SAVE-01  round-trip: play N commands → save → load → identical
stateHash
  T-SAVE-02  replay determinism: the same file loaded twice → identical
hashes
             (leans on every §4.7 determinism gate — T-HEX-07, T-MOVE-06,
             T-FAME-09, T-TURN-09, T-AI-06; this is their end-to-end
composition)
  T-SAVE-03  prefix validity: every prefix of a valid log is itself a
valid,
             loadable save (mid-match save falls out of this)
  T-SAVE-04  refusal: any header mismatch (version/rules/data/scenario
hash) →
             load refused with a reason; state untouched
  T-SAVE-05  no partial load: a log with an illegal command at index k is
             refused whole; the pre-load state survives. (The tripwire: an
agent
             that applies-then-validates leaves a corrupted half-loaded
state
             that passes every happy-path test.)
  T-SAVE-06  stateHash stability: hash of a given state is identical
across the
             headless and in-engine builds (asserted jointly with T-INT-
02)
  T-SAVE-07  harness compatibility: a Balance Analyst self-play log
validates
             and replays as a save file — one format, no dialect drift
Determinism: pure; the file contains everything needed to reproduce the
state.
Acceptance: T-SAVE-01..07 headless (05 also exercised in-editor via the
load UI
            path); slot I/O smoke test in the editor pass.
```

4.11 Build order

Rows 1–8 are the §4.7 stubs — the eight ledger rows that read `*pending*` when this table was written, of which **rows 1, 2, 3, 4, 5 and 6 have since flipped** (§3), row 2 last and across a repo pair (Q34). §4.7's heading names the same eight **as at 2026-08-01** and is dated for that reason: it records a state and is not a live count, so it does not move when a row flips and is not out of step with this sentence. Rows 9–10 are the §4.9 and §4.10 systems. Combat, its test suite, Repair, and Type-effectiveness are green at `5ffa8d6` and are prerequisites, not work items. **Rows 1 and 3 are green at `c224825`, row 4 at `647d4df`, row 5 at `6ccd40b` — its rebuild, where the widened `T-TURN-01..10` closes; `T-TURN-01..09` first closed at `ad77b13` — and row 6 at `d8284f1`**, so rows 7–8 depend on landed code rather than on scheduled code, and **row 8 was the critical path's remaining link**, its dependency cell in the table below reading `5, 7` — both landed before it. **Row 8 has since landed too**, at `7c36303`, on a partial pass that leaves its ledger row unflipped, as row 7's does (§3): T-UI-03 and T-UI-04 are the in-editor half and no in-editor pass exists at that commit, and **T-UI-05 was then headless and unimplemented** — minted 2026-08-04, after that commit. **T-UI-05 is green at `41a1452`** (§3), the rebuild at which the snapshot additions ruled beside it were built, so the IDs this row still lacks are `T-UI-03` and `T-UI-04`, and among what the row's flip waits on are the real Stratocracy widgets those IDs assert against, which are measured absent at `a13626f` (§4.9) — the in-editor Automation pass those two also lacked landed at `fed8ae9` (§3) — the row staying unflipped on the widgets. **Row 9's headless half has since landed**, at `b23823f` in the crew repo with the vendored tree at `99fcb84` in the UE project repo, which is what this row's cell means by closing as soon as vendoring lands. **`T-INT-01` and `T-INT-04` are green at `e19605e`**, over the vendored tree at `a13626f`, the standalone gate that asserts `T-INT-04` running under clang++, and not at the landings that first passed them: `T-INT-01` asserts identity at `rulesCommit`, and `rulesCommit` moved there, while `T-INT-04` compiles the vendored copy, whose `Turn.h` bytes changed. Neither ID's text changed there, so both

are closure movements rather than widenings; earlier, `T-INT-01`'s written text was widened at `d837fc8`, over an input that did not exist before it, so it closed there under the closure convention §4.5 states (§3). Each landing stands as the landing it was; what moves is where an ID's green is credited — the treatment row 5 already has, whose `T-TURN-01..10` closes at its rebuild while `T-TURN-01..09` first closed at `ad77b13`. **Row 9's acceptance set does not close**, on the Q29 reading rows 7 and 8 stand on: `T-INT-02`, `T-INT-03` and `T-INT-05` did not run, and among what they wait on besides the harness that has since landed are a vendored replayer, the bridge's command surface and real Stratocracy widgets, one per ID (§4.9). **Row 2 is green:** T-DATA-01..04 and 06 pass headless and T-DATA-05 passed 4/4 in the editor pass at `fed8ae9`, so the full set closes across the repo pair Q34 rules on and the ledger row flips (§3). Its † bullet below priced that flip as the cost of losing the pass; the pass ran, so the mark can no longer fire. Note **row 10's format spec must exist by the row-9 integration pass**, because T-INT-02's input file is a §4.10 save. Rows 9 and 10 therefore state their dependencies in **two parts** — what each gate needs in order to **run**, and what it needs in order to **close** — because both are scoped to the command set of the log they replay, and week 2's log carries only `{Move, Attack}` (§4.4). Reading either row's requirement as a single set is what put a gate in a week this table could not supply it.

#	System (ledger row)	Depends on	Headless?	Acceptance test IDs
1	Hex grid & math (§4.7 Stub 1)	— (Q1 pins bounds)	Yes	T-HEX-01..07
2	Data tables (§4.8, incl. effectiveness CSV)	— (MoveClass column blocked on Q2)	Loader + T-DATA-06 yes; import parity in-editor	T-DATA-01..06 (**T-DATA-05** †)
3	Movement & pathfinding (Stub 3)	1, 2	Yes	T-MOVE-01..06
4	Capture & Fame economy (Stub 4)	3	Yes	T-FAME-01..09
5	Turn loop & win/tiebreak (Stub 5)	4 + verified Combat/Repair @ 5ffa8d6	Yes	T-TURN-01..10
6	Opponent AI (Stub 6)	5	Yes	T-AI-01..06 + self-play smoke
7	Scenario file & validator (Stub 7)	1, 2 for the structural half (T-SCN-01..03, 05, 07, 09); **3 for the priced half** — T-SCN-04, 06, 08, 11 all cost a path, and **T-SCN-11 costs a full-board pass rather than a path**: it minimises over every Infantry the opposing seat deploys (Q28, ruled), which is **one reverse Dijkstra per `guidedOpening` objective** and is therefore independent of that seat's unit count — done naively as one path per opposing unit it scales with the deployment instead	Yes; MCP tool wraps it in-editor, manual fallback stands	T-SCN-01..09, 11 (10 reserved-unwritten on Q26; **T-SCN-08, 09, 11** †)
8	UI binding (Stub 8)	5, 7 (snapshot needs full state)	Contract + queries yes; widgets in-editor	T-UI-01..05 (**T-UI-03, 04** †) + GATE-CAP-PARTIAL
		Run vs close. Vendoring and **T-INT-01/04** depend on no rules row at all — the sync script and the standalone gate run *are* the assert — and close as soon as vendoring lands, which `T-INT-04` first did at `b23823f` over the vendored tree at `99fcb84`, and `T-INT-01` at `d837fc8` over		

#	System (ledger row)	Depends on	Headless?	Acceptance test IDs
9	Presentation bridge & integration — §4.9 (proposed ledger row)	~~INT-01 at d597fc8 over~~ the vendored tree at `9dec48c`; **both greens now stand at `e19605e`**, over the vendored tree at `a13626f`, re-dated there for their own separate reasons (§3). **T-INT-02/03/05** need only the rows behind the log they replay: rows 1–3 for week 2's `{Move, Attack}` log, and they re-open on the `Capture`/`Build`/`EndTurn` that rows 4–5 have since added. They **close on rows 1–5**, which is what this cell used to state as the whole dependency	Source/compile gates yes; replay parity + statelessness in-editor	~~T-INT-01~~ 01..05 (**T-INT-02, 05** †)
10	Save & replay — §4.10 (proposed ledger row)	**Three parts, three dependency sets.** (a) *Format spec + header/version machinery* — **no deps at all; write it first**, and T-SAVE-04 (refusal on any header mismatch) closes on it alone, since it never applies a command. **Part (a) has since landed**, at `737f666`: `T-SAVE-04` is green there under clang++ and MSVC both, beside `GATE-SAVE-PARSE`, which mints no acceptance ID, so this row holds code without closing its set (§3). (b) *Headless replayer* — rows 1–3, plus row 7's **structural** half for the `scenarioId`/`scenarioHash` it loads; it runs T-SAVE-01/02/03/05/06 over week 2's `{Move, Attack}` log. (c) *Closure* — rows **4, 5** complete the command set (T-SAVE-01/03/05/06), row **6** completes T-SAVE-02's determinism composition, and T-SAVE-07 needs row 6's self-play besides. **Part (b) has since landed**, at `ec15be6`, and was run against these closure conditions rather than after them (ruled 2026-08-05, rows 4, 5 and 6 all being green): `T-SAVE-01`, `T-SAVE-02`, `T-SAVE-03` and `T-SAVE-05` are green there under clang++ and MSVC both, beside `GATE-REPLAY-*`, which mint no acceptance ID. `T-SAVE-06` waits on the in-editor Automation harness and `T-SAVE-07` on a self-play log written in this format, so this row holds code without closing its set (§3). **Part (c) has since landed**, at `1ee890e`, on a third registry row whose link set encodes exactly the dependency this cell states — rows 4, 5 and 6: `T-SAVE-07` is green there	Yes, all but slot I/O	T-SAVE-01..07 (**T-SAVE-06** †) + `GATE-SAVE-PARSE`, `GATE-REPLAY-*`, `GATE-BALANCE-*`

#	System (ledger row)	Depends on	Headless?	Acceptance test IDs
		under clang++ and MSVC both, beyond the `BALANCE-*`, which mint no acceptance ID. That log carries the four command kinds row 6's AI emits and not the fifth, `Capture` being outside the AI's vocabulary by design (`T-AI-03`), and `T-SAVE-07` asserts format compatibility rather than command coverage, so its whole written fixture set ran. **`T-SAVE-06` is now the only ID this row lacks**, and among what that ID waits on is a vendored replayer: `T-SAVE-06` is asserted jointly with `T-INT-02`, whose replay runs **in-engine**, so the replayer has to be compiled into the engine and therefore vendored, and `Replay` is ruled out of vendoring until a bridge consumer exists. The in-editor Automation harness this ID also lacked landed at `fed8ae9` in the Stratocracy UE project repo (§3). Both what *the editor pass* denotes and what running it does not supply are stated at §4.9. Slot I/O is week 5 and no headless gate waits on it		

† — **the cut line (§4.7 head).** A marked ID does not *close* if the calendar takes it, and what each mark costs is a **claim**, never a rule: no marked ID guards a rules invariant, so nothing in the game changes behaviour when one stands down. Unmarked is the default and the majority.

- **T-DATA-05** — row 2's only in-editor half, and **closed**, 4/4 at `fed8ae9` in the Stratocracy UE project repo (§3), so the mark prices nothing the calendar can still take. What it priced while it stood: the fallback was a Director read of two frozen tables (4 unit rows × 11 columns, 7 terrain rows × 10), and the cost was row 2's ledger flip. **The mark is kept (ruled 2026-08-06):** a closed ID keeps its †, and no rule about marks in general is minted by that — what a mark costs is a claim and never a rule, as the head of this list says.
- **T-SCN-08, 09, 11** — their fixtures are stretch-map-resident, and §2.13.7 states the condition plainly: if week 4 is consumed by balance, "the set stays on paper." T-SCN-08 then loses fixtures (a) *The Causeway* and (b) *Longwater March*, keeping only the synthetic ceiling refusal (c); T-SCN-11 loses fixture
  c. and keeps its two shipped-map fixtures, including the failing one; and T-SCN-09's asserting branch loses both maps, because `symmetry == none` asserts nothing and the shipped map declares `none`. T-SCN-01..07 stay unmarked — they are what refuses a malformed shipped scenario, T-SCN-04 is the precondition of the flag win §4.5 calls a complete MVP, and T-SCN-06 gates the §2.11.6 opening §2.11.8 ranks must-have. Cost: row 7's ledger flip, on the Q29 reading applied per acceptance ID as well as per row — an ID closes only when its whole written fixture set has run.
- **T-UI-03, 04** — in-editor Automation over widget bindings, where a Director reading the screen is a real check. T-UI-01/02 and T-UI-05 stay unmarked: all three are headless, T-UI-01 is what makes §2.11.3's forecast equal the resolution, and T-UI-05 is the sole assertion that the snapshot tells the truth about the state the module holds — the contract every other ID in the row reads through.
- **T-INT-02, 05 and T-SAVE-06** — the in-editor half of the parity pair (T-SAVE-06 is asserted jointly with T-INT-02). This is the most expensive mark in the list and its cost is a §3 cost rather than a §1 one: without it the shipped engine build is never proven identical to the certified headless module, so the ledger's evidence chain stops at the vendoring hash (T-INT-01) instead of reaching the artifact that ships. T-INT-01/04 and T-SAVE-01..05 stay unmarked on **cost** — §4.9 runs T-INT-01/04 on every gate run and §4.10 takes T-SAVE-01..05 headless, so none of them needs the editor pass in order to be asserted (T-SAVE-05 is *also* exercised in-editor via the load UI path, which adds coverage rather than owning it). **T-INT-03 stays unmarked on the rule, not on cost:** §4.9 does place it in the editor pass, but what it asserts — an illegal command

leaves the state hash unchanged and returns a reason, no partial application — is the bridge behaviour §4.9 contracts ("an invalid command returns a rejection reason and changes nothing"), and a marked ID may not guard a rules invariant. The consequence is stated rather than hidden: an editor pass cut to its marked IDs alone would still owe T-INT-03, so this line thins that pass, it never cancels it.

- **T-SAVE-07 is deliberately unmarked**, against the reading that grouped it with T-SAVE-06. It is headless, and it is **green at `1ee890e`** (§3) on row 10's part (c) — ahead of the week-4 output §4.4 scheduled it on, so no slip in that week can reach it now. The reason the mark was declined stands as written: under §2.13.7's slip condition self-play still runs and only the stretch maps stand down, so cutting it would have bought no calendar.

Critical path: 1 → 3 → 4 → 5 → 6/8. Row 2 runs in parallel immediately. **Row 7 no longer sits beside the chain; it straddles it.** Its structural half (T-SCN-01..03, 05, 07, 09) starts once 1–2 land, but **four of its ten written invariants** — T-SCN-04's flag reachability and T-SCN-06/08/11's opening-capture lane — price a Stub-3 path, so row 7 cannot *close* until row 3 does. T-SCN-11 (Q22 and Q28, both ruled) is the newest of the four and the most expensive: it prices the *opposing* seat's cheapest route to the same objective, minimised over every Infantry that seat deploys, which is one reverse Dijkstra per `guidedOpening` objective — a full-board pass rather than a path, and one whose cost does not grow with the deployment. It **asserts**, as of this revision: nothing in row 7 is written-and-blocked, and the only unwritten invariant is T-SCN-10, reserved on Q26. It ships with a **failing** fixture — the shipped map's own pre-fix deployment, which tied 5 against 5 — so this row's suite proves it can refuse a real scenario and not merely agree with the repo. Row 7 is still not ON the critical path (nothing in the chain waits on it), but scheduling it as "parallel from week 1" would leave its ledger row un-flippable and the §2.11.6 guided opening ungated for however long movement slips: the scenario row flips after movement, not before. 6 and 8 fork after 5. **Row 10 is split, per Q20 (ruled).** Its *format and headless replayer* are not week-5 content but the instrument two earlier gates run on — T-INT-02's input file is a save, and the week-4 self-play logs T-SAVE-07 validates are the same format — so §4.4 now lands them beside the scenario loading they already sit next to, and leaves only the **save-slot UI and slot I/O** at the end. That is the split this section proposed and the Director adopted; §4.4's table and the note under it are the single statement of the schedule, and this paragraph deliberately does not restate the week numbers a second time.